

ASSIGNMENT COVERSHEET

UTS: ENGINEERING & INFORMATION TECHNOLOGY		
SUBJECT NUMBER & NAME 41026 Advanced Software Development	NAME OF STUDENT(s) (PRINT CLEARLY) <i>Lam Andy</i> <i>Hatzidimitriou Nicholas</i> <i>Lim Tristan</i> <i>Hein Htet Aung</i> <i>Barnard Matthew</i>	STUDENT ID(s) 25249309 25471398 25299672 25026511 25563457
STUDENT EMAIL andy.lam@student.uts.edu.au nicholas.b.hatzidimitriou@student.uts.edu.au tristan.j.lim@student.uts.edu.au heinhтет.aung@student.uts.edu.au matthew.k.barnard@student.uts.edu.au		STUDENT CONTACT NUMBER 0450615201 0477559936 0431765886 0421683687 0468962300
NAME OF TUTOR Georges Bou Ghantous	TUTORIAL GROUP 04	DUE DATE 09/09/2026
ASSESSMENT ITEM NUMBER & TITLE Assignment 1 - Release 0 Technical Report		
☒ I acknowledge that if AI or another nonrecoverable source was used to generate materials for background research and self-study in producing this assignment, I have checked and verified the accuracy and integrity of the information used. ☒ I confirm that I have read, understood and followed the guidelines for assignment submission and presentation on page 2 of this cover sheet. ☒ I confirm that I have read, understood and followed the advice in the Subject Outline about assessment requirements. ☒ I understand that if this assignment is submitted after the due date it may incur a penalty for lateness unless I have previously had an extension of time approved and have attached the written confirmation of this extension. Declaration of originality: The work contained in this assignment, other than that specifically attributed to another source, is that of the author(s) and has not been previously submitted for assessment. I have rewritten any material provided by AI or other nonrecoverable sources and where appropriate acknowledged their contribution. I understand that, should this declaration be found to be false, disciplinary action could be taken and penalties imposed in accordance with University policy and rules. In the statement below, I have indicated the extent to which I have collaborated with others, whom I have named. No content generated by AI technologies or other sources has been presented as my own work and I have rewritten any text provided by AI or other sources in my own words. Statement of collaboration:		
Signature of student(s) <i>Andy Lam Tristan Lim Nicholas Hatzidimitriou Hein Htet Aung Matthew Barnard</i> Date 09/09/2026		

Project Overview

Project Title

Faculty Management System

Purpose

This project applies the principles of Agentic AI software into the real world context of a Faculty Management System. We have designed, developed and integrated a microservice based application to support university operations across five domains. These domains are: Academic Staff Management, Teaching and Classroom allocation, Workload and Availability Tracking, Research and Grant Management, and Staff Performance and Professional Development.

Problem

University faculties manage staff, teaching allocations, workloads, research output and professional development mostly through a combination of manual processes and disconnected systems. By doing this, it often creates inconsistent data, duplicated effort and missed opportunities to match staff expertise, availability and workload against the university needs. This project addresses the issue by building a single, integrated faculty management system where each domain is encapsulated by an independent microservice and is driven by AI recommendations where needed.

Project Scope

We have proposed, implemented and integrated one Agentic AI application, consisting of five individually developed features, unified into a single shared homepage, common CSS theme and a shared Docker Compose environment. Each feature has been developed with an independent set of frontend, backend and database microservices with a shared AI-mode service

Individual Features

Student	Feature	Description
Andy Lam	Academic Staff Management	Manage academic staff profiles, qualifications, expertise, employment details, availability and AI-driven qualification check to determine suitability for teaching.

Hein Htet Aung	Teaching, Subject and Classroom Allocation	Manage subjects, classrooms and teaching allocations with AI assisted recommendations based on staff availability, requirements, workload and room capacity.
Tristan Lim	Workload and Availability Management	Track staff's contracted load, work hours and availability with AI suggested rebalancing.
Nicholas Hatzidimitriou	Research and Grant Management	Manage research projects, grant applications and publications, with AI generated research summaries and staff matching on expertise and current workload
Matthew Barnard	Performance and Professional Development	Track performance reviews, development goals and training with AI identified skill gaps and development plans.

Technology Stack

- Frontend: HTMX, CSS, JavaScript
- Backend: Python 3, Flask
- Database: SQLite
- AI Runtime: Ollama running on approved LLMs (Qwen and Llama)
- Containerisation: Docker, Docker Compose
- CI/CD: Github, Github actions, YML workflow files

Architecture Approach

We followed a microservices architecture where each of us owns an independent frontend, backend and database microservice for our individual feature. Cross feature data access (retrieving staff expertise or workload status) is achieved through an exposed REST API, never by direct access to the database, preserving ownership and creating encapsulation. All the services were containerised through a single docker-compose.yml file, alongside a shared, unified Ollama API service.

Project Analysis and Planning

Agile Team Project Approach

The faculty Management System was developed using an Agile, feature based approach. Each team member was responsible for one independent microservice feature, while shared components such as the homepage, CSS, Docker Compose configuration and Ollama AI runtime were developed and integrated at a team level.

Development was completed incrementally, with core CRUD operations and database functionally completed before AI features, cross-service integration, and finally the Agentic

AI loop. Team members worked in parallel through separate repository directions and individual GitHub Actions workflows, then integrated their services through REST APIs and the shared Docker environment.

This approach reduced overlap between team members while still supporting collaboration on shared functionality. Cross-feature dependencies were handled through exposed APIs rather than direct database access, helping maintain clear service ownership and allowing each component to be tested and integrated progressively into the overall system.

Sprint Backlog

To coordinate the implementation of the Faculty Management System within the five feature areas assigned to it, the Release 0 sprint backlog was created. Each team member had to develop their own frontend, backend/API and database microservice, as well as contribute to integration work that was shared throughout the team. The backlog was prioritized with basic CRUD capabilities and database development first, then cross-service interaction, AI-Mode functionality, Docker integration, CI/CD validation and the shared Plan -> Act -> Observe -> Adapt Agentic AI process. This architecture allowed the work to be done in isolation with explicit integration points for the group application.

Sprint Backlog Table

ID	Owner	Type	Backlog Item / Requirement	Priority	Acceptance Criteria	Status
G-01	Group	Project	Establish shared GitHub repository and standard project structure	High	Repository contains individual student directories, shared resources, workflows, Docker Compose and documentation structure	Done
G-02	Group	Integration	Create unified Faculty Management System homepage	High	All five student features are accessible from the shared homepage	Done
G-03	Group	UI	Apply shared CSS theme across individual frontend services	Medium	Individual feature interfaces use the common visual theme	Done

G-04	Group	DevOps	Integrate all services through one Docker Compose configuration	High	Complete Release 0 application starts through shared Docker Compose configuration	Done
G-05	Group	AI	Integrate shared Ollama runtime and approved open-source LLMs	High	Individual AI-Mode features can communicate with the shared Ollama service	Done
G-06	Group	Agentic AI	Implement shared Plan -> Act -> Observe -> Adapt workflow	High	Agentic review workflow executes and records the four stages	Done
G-07	Group	CI/CD	Maintain individual GitHub Actions workflows	High	Each student's microservices are independently built and validated by their workflow	Done
S1-F R01	Andy	Functional	Manage academic staff records	High	Staff records can be created, viewed (including filter and search), updated and deleted	Done
S1-F R02	Andy	Functional	Manage staff qualifications and expertise	High	Qualifications and expertise are stored and retrievable for staff	Done
S1-F R03	Andy	Functional	Generate AI teaching suitability analysis	High	AI generates an analysis based on staff qualifications and expertise	Done
S1-N FR01	Andy	Non-functional	Preserve staff-service ownership	High	Other features obtain staff information through the Staff Management API rather than directly reading its database	Done

S2-F R01	Hein	Functional	Manage subjects	High	Users can create, retrieve, update and delete subject records with validation applied	Done
S2-F R02	Hein	Functional	Manage subject offerings	High	Users can create, retrieve, update and delete subject offerings linked to valid subjects	Done
S2-F R03	Hein	Functional	Manage classrooms	High	Users can create, retrieve, update and delete classroom records containing room capacity and facilities	Done
S2-F R04	Hein	Functional	Manage teaching allocations	High	Teaching allocations can be created, retrieved, updated and deleted through the frontend and backend API	Done
S2-F R05	Hein	Functional	Validate classroom capacity	High	An allocation is rejected when expected class size exceeds the selected classroom's capacity	Done
S2-F R06	Hein	Functional	Detect classroom scheduling conflicts	High	A classroom cannot be allocated to overlapping teaching activities during the same relevant date and time period	Done
S2-F R07	Hein	Functional	Retrieve staff information from Academic Staff Management	High	Staff identity and expertise required for allocation decisions are retrieved through Student 1's REST API	Done
S2-F R08	Hein	Functional	Retrieve availability and workload information	High	Staff availability/workload context is obtained from Student 3 without direct access to its database	Done

S2-F R09	Hein	Functional	Generate AI-assisted staff recommendations	High	AI-Mode ranks suitable staff using allocation requirements, expertise, availability and workload context	Done
S2-F R10	Hein	Functional	Validate AI-generated recommendations	High	Backend review rejects invalid, duplicate, nonexistent or unavailable staff recommendations before displaying accepted results	Done
S2-F R11	Hein	Functional	Support recommendation adaptation	Medium	Previous recommendation IDs and failure observations can be supplied to another AI recommendation cycle	Done
S2-F R12	Hein	Functional	Support human-controlled assignment	High	AI recommendations remain advisory and the user chooses whether to assign a recommended staff member	Done
S2-N FR01	Hein	Non-functional	Maintain microservice separation	High	Frontend, backend/API and SQLite database execute as separate services	Done
S2-N FR02	Hein	Non-functional	Use REST-based cross-service communication	High	Student 2 accesses external staff/workload information using exposed APIs instead of another service's database	Done
S2-N FR03	Hein	Non-functional	Validate untrusted AI output	High	LLM recommendations are checked by deterministic application logic before being accepted	Done

S2-N FR04	Hein	Non-functional	Provide controlled input validation and errors	High	Invalid subjects, classrooms, schedules, class sizes and allocations produce controlled validation responses rather than being persisted	Done
S2-N FR05	Hein	Non-functional	Support containerised local execution	High	Student 2 frontend, backend and database services build and operate within the shared Docker Compose application	Done
S2-N FR06	Hein	Non-functional	Support automated CI validation	High	Student 2 GitHub Actions workflow installs dependencies, executes validation/tests and builds the three Docker images successfully	Done
S3-F R01	Tristan	Functional	Manage workload entries, availability and leave	High	CRUD operations are available for workload, availability and leave records	Done
S3-F R02	Tristan	Functional	Calculate staff workload state	High	Workload rules identify overloaded and underloaded staff using current workload information	Done
S3-F R03	Tristan	Functional	Generate AI workload redistribution recommendations	High	AI can propose redistribution and proposals are checked by the rules engine	Done
S3-F R04	Tristan	Functional	Support human review of AI recommendations	High	Saved recommendations can be individually accepted, rejected or overridden	Done

S3-F R05	Tristan	Functional	Retrieve staff identity from Academic Staff Management	High	Staff directory and reconciliation read live from Student 1's API	Done
S3-N FR01	Tristan	Non-functional	Preserve external staff ownership	High	Staff identity is obtained from Student 1 rather than maintained as an independent master record	Done
S3-N FR02	Tristan	Non-functional	Validate untrusted AI output	High	Every AI proposed reassignment is checked against the rules engine before being shown or saved	Done
S4-F R01	Nicholas	Functional	Manage research projects, grants and publications	High	CRUD operations are provided for the research domain	Done
S4-F R02	Nicholas	Functional	Generate AI research summaries and staff-fit reasoning	High	AI responses use project and staff context retrieved by the service	Done
S4-F R03	Nicholas	Functional	Recommend research staff using cross-service information	High	Staff expertise and workload information are used when generating recommendations	Done
S4-F R04	Nicholas	Functional	The system searches the full faculty staff roster (via the Staff Management service) and recommends new staff for a project, based on expertise	High	Staff and workload information are available for use	Done

S4-N FR01	Nichola s	Non-functi onal	Encapsulate research database access	High	Backend communicates with its database microservice over REST rather than querying SQLite directly	Done
S4-N FR02	Nichola s	Non-functi onal	AI responses are given within reasonable time. Around 3-5 seconds	High	Each call has a 5 second timeout.	Done
S4-N FR03	Nichola s	Non-functi onal	The backend enables CORS so the frontend can call it directly from the browser	High	Frontend returns with no “blocked by CORS”	Done
S5-F R01	Matthe w	Functional	Manage staff performance reviews and development goals	High	Performance/developme nt records support required CRUD operations	Done
S5-F R02	Matthe w	Functional	Manage training and professional development	High	Training and development records can be managed through the feature	Done
S5-F R03	Matthe w	Functional	Generate AI-assisted development recommendati ons	High	AI recommendations are generated from relevant staff performance and development information	Done
S5-N FR01	Matthe w	Non-functi onal	Maintain development-g oal data consistency	High	Invalid combinations such as completed goals below required progress are rejected by validation	Done

Overall Project Plan

Release 0: Agentic AI Foundations, Microservices and DevOps

- Establish the standard project structure, assign ownership of the five distinct features, and create the shared GitHub repository.
- Develop frontend, backend/API, and database microservices for every student.
- Comprehensive database architecture with CRUD functionalities. Subsequently, the cross-service linkages are completed.
- If a particular functionality requires data managed by another microservice, then cross-service REST APIs can be developed.
- Incorporate AI-Mode utilizing a communal Ollama runtime with sanctioned open-source LLMs.
- Collaborative Plan -> Act -> Observe -> Adapt Agentic AI loop incorporated.
- Docker Compose: Encapsulate and manage the entire application.
- Authenticate distinct services with GitHub Actions CI/CD methodologies.
- Conduct comprehensive local integration testing and gather documentation for the technical report and project presentation.

Release 1: MCP, RAG and Grounded AI

- Enhance the Release 0 framework rather than replacing it.
- Present common MCP and RAG services.
- Employ RAG to extract significant context and include it consistently within grounded AI responses.
- Integrate new AI services with the existing microservices and local Docker setup.
- Revise testing, CI/CD, and documentation to correspond with the Release 1 architecture.

Release 2: Multi-Agent Systems, Testing and Cloud Deployment

- Employ Multi-Agent architecture to enhance the Release 1 system.
- Incorporate Planner, Worker, and Reviewer agents, accompanied by human oversight.
- Enhance automated testing and validation throughout the connected application.
- Configure the application for cloud deployment.
- Implement the complete integrated Faculty Management System on AWS.
- Authenticate local and cloud deployment processes and obtain conclusive project artifacts.

Incremental Development Approach

- Each release modifies and enhances upon the features and architecture completed in the previous release.
- The importance of CRUD and microservice capabilities surpasses that of advanced artificial intelligence functionalities.
- Integration, testing, DevOps, and documentation are being incrementally strengthened during the three releases.

- To lessen rework and facilitate continuous integration, a collective architecture is preserved and enhanced throughout the semester.

Academic Staff Management (Andy Lam)

Academic Staff Management - Feature Plan

Description

The academic staff management will provide faculty administrators an overview and control of staff records in creating, reading, updating and deleting records. Staff records along with their personal information will include their qualification, areas of expertise and qualifications. Within this microservice includes an AI-generated suitability analysis for each staff member through the combination of a rule based suitability score and a natural language summary produced by the shared Ollama LLM.

Problem Addressed

With staff information scattered across spreadsheets, this feature will serve as a central place for efficiently managing staff records, track staff capabilities for assigning teaching duties. Additionally, introducing an AI teaching suitability analysis will help faculty administrators make informed decisions related to staffing. This feature will also assist the other microservices in retrieving staff records for other faculty management features.

Frontend Microservice Description

- Will run on a dedicated port as a static HTML/JavaScript application served through nginx.
- Provide a filter and search feature for filtering staff records (search by expertise and filter by department or position).
- Provide a detailed view of staff details, including their personal information, expertise, availability and button for the AI generated summary.
- Contain an Add/Edit form for creating and editing staff records.
- Contain a delete button for deleting a staff record and its attached records.
- Communicate with the backend via REST API calls.
- Adopt the shared CSS theme for consistent visual style.

Backend Microservice Description

- Implemented as a Flask REST API, containing CRUD operations and exposing endpoints.
- Apply input validation and error handling for missing records, database constraint violations and missing records.
- Implement the AI-Mode capability by calling the shared Ollama runtime and computing a suitability score, while using staff expertise and qualifications as a base to formulate a written analysis. The analysis will then be stored against the staff record.

Database Microservice Description

- Will use SQLite, containerised as its own database microservice.
- Store records for staff, departments, qualifications, expertise, availability and AI-generated analysis results.
- Seeded with a minimum of 10 records across each table.
- Will enforce referential integrity with foreign key constraints. Removing a staff member correctly removes their dependent records.

Integration with Team Application

- The feature is to be accessible from the unified home page alongside the other four features and follow the team's shared CSS theme.
- Requires integration into the shared docker compose configuration file and running alongside the other containers including the shared Ollama runtime.
- Exposes endpoints for other microservices to retrieve necessary data (staff, departments, etc).

Academic Staff Management - Risk Management Plan

ID	Risk	Probability	Impact	Risk Level	Mitigation	Contingency
R1	Shared Ollama runtime becomes unavailable causing /generate_analysis to fail	Medium	Medium	Medium	Wrap Ollama call in try/except block with a clear error response.	Ensure CRUD operations are functional and treat AI runtime as secondary.
R2	Staff management goes down causing the other microservice to fail to access its API's	Medium	High	High	Ensure any changes to the API are communicated to the team and keep endpoint paths consistent across releases.	Version the API and notify affected teammates before merging changes.
R3	API_BASE with localhost is hardcoded causing issues with a move to AWS in release 2 deployment.	High	Medium	Medium-High	Move base urls to environment variables and test with docker compose service names.	Allocate enough time to update and retest all interservice URLs.
R4	Summaries generated by LLM are hallucinated but presented as authoritative.	Medium	Medium	Medium	Keep suitability score as the primary auditable metric alongside the LLM	Constrain the prompt further if output is still poor.

					summary.	
R5	Cascading deletes a staff member's history without any method to recover it.	Low	Medium	Low-Medium	Prompt a confirmation before going through with deletion.	Restore the most recent backup for staff.db with the seed if an accidental deletion occurs.
R6	Frontend uses vanilla JS instead of HTMX, deviating from specified frontend technology.	High	Low	Low-Medium	Document deviation explicitly within the report or ask the tutor for clarification.	Refactor to HTMX if necessary in a later release.
R7	Limited automated test coverage could result in hidden bugs or broken functionality in the future.	Medium	Medium	Medium	Use manual test scripts consistently before each release.	Rollback to the last working commit using Git history if regression is found.
R8	Incomplete microservices affect the completion or delivery of the staff management.	Medium	High	Medium-High	Distribute work over weekly milestones and ensure they're met. Prioritise CRUD functions then AI mode after.	Prioritise working CRUD features over AI mode CRUD functions are graded over every release.

Academic Staff Management - Data Design

Conceptual

- A department belongs to a faculty and employs many staff (One to many).
- A staff member belongs to one department and can hold multiple qualifications, expertise, and availability entries (One to many).
- A qualification records a credential a staff member holds (One to many).
- An expertise records a subject or skill a staff member specialises in, with the addition of a skill level (One to many).
- An availability entry is a time frame, which a staff member is available to teach (One to many).
- An AI analysis is a system generated summary of a staff member's general teaching suitability (One to many).

Logical

Entity	Key Attribute	Relationship
Departments	department_id (PK), department_name, faculty	One department to Many staff

Staff	staff_id (PK), name, email, phone, department_id (FK), position, employment_type, status	Many staff to one department One staff to many qualifications, expertise, availability, ai analysis
Staff Qualifications	qualification_id (PK), staff_id (FK), qualification_name, institution, year_obtained	Many qualifications to one staff
Staff Expertise	expertise_id (PK), staff_id (FK), expertise_area, skill_level	Many expertise to one staff
Staff Availability	availability_id (PK), staff_id (FK), day, time_slot, availability_status	Many availability to one staff
Staff AI analysis	analysis_id (PK), staff_id (FK), generated_summary, suitability_score, date_generated	Many ai analysis to one staff

Physical

Implemented via SQLite, staff.db generated from schema.py

```
CREATE TABLE departments (
    department_id INTEGER PRIMARY KEY,
    department_name TEXT NOT NULL,
    faculty TEXT NOT NULL
);
```

```
CREATE TABLE staff (
    staff_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT NOT NULL,
    phone TEXT,
    department_id INTEGER NOT NULL,
    position TEXT NOT NULL,
    employment_type TEXT NOT NULL,
    status TEXT NOT NULL,
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);
```

```
CREATE TABLE staff_qualifications (
    qualification_id INTEGER PRIMARY KEY,
    staff_id INTEGER NOT NULL,
    qualification_name TEXT NOT NULL,
    institution TEXT,
    year_obtained INTEGER,
    FOREIGN KEY (staff_id) REFERENCES staff(staff_id) ON DELETE CASCADE
);
```

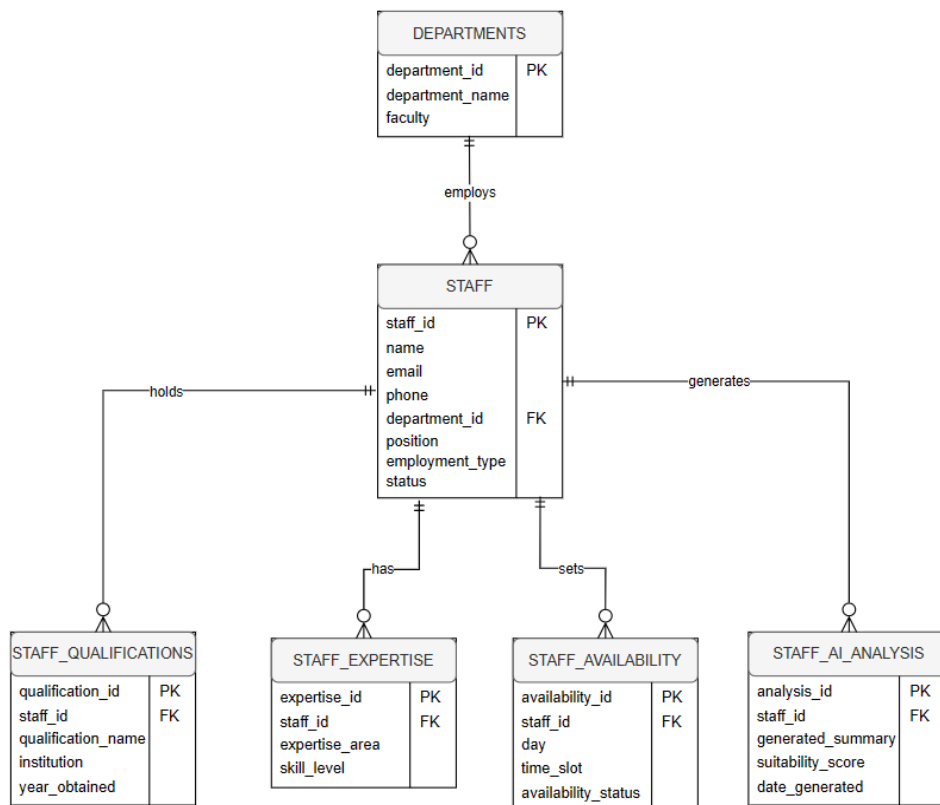
```
CREATE TABLE staff_expertise (  
    expertise_id INTEGER PRIMARY KEY,  
    staff_id INTEGER NOT NULL,  
    expertise_area TEXT NOT NULL,  
    skill_level INTEGER NOT NULL,  
    FOREIGN KEY (staff_id) REFERENCES staff(staff_id) ON DELETE CASCADE  
);
```

```
CREATE TABLE staff_availability (  
    availability_id INTEGER PRIMARY KEY,  
    staff_id INTEGER NOT NULL,  
    day TEXT NOT NULL,  
    time_slot TEXT NOT NULL,  
    availability_status TEXT NOT NULL,  
    FOREIGN KEY (staff_id) REFERENCES staff(staff_id) ON DELETE CASCADE  
);
```

```
CREATE TABLE staff_ai_analysis (  
    analysis_id INTEGER PRIMARY KEY,  
    staff_id INTEGER NOT NULL,  
    generated_summary TEXT NOT NULL,  
    suitability_score REAL NOT NULL,  
    date_generated TEXT NOT NULL,  
    FOREIGN KEY (staff_id) REFERENCES staff(staff_id) ON DELETE CASCADE  
);
```

- PRAGMA foreign keys = ON set on every connection as SQLite doesn't enforce foreign key constraints on default.
- Use INTEGER PRIMARY KEY for primary keys which auto increment.
- Suitability score uses REAL to store a decimal score.
- ON DELETE CASCADE to delete dependent records.
- NOT NULL to enforce an input and no records have major fields which are empty.

Entity Relationship Diagram



Teaching, Subject and Classroom Allocation (Hein Htet Aung)

Teaching, Subject and Classroom Allocation - Feature Plan

Description

The Teaching, Subject and Classroom Allocation tool offers faculty administrators a unified interface for overseeing subjects, subject offerings, classrooms, and teaching assignments. This functionality incorporates conventional CRUD operations with business rule verification and AI-driven staff suggestions. Teaching allocations link a subject offering to a classroom, a designated teaching period, and optionally, a designated instructor. For assignments requiring personnel allocation, AI-Mode assists the administrator by prioritizing qualified staff according to subject requirements, staff expertise, and availability, as well as workload data provided by other faculty microservices. Distributions continue to depend on human judgment.

Problem Addressed

The distribution of teaching allocations necessitates data that is frequently stored across several university systems, including subject requirements, classroom capacity, faculty qualifications, staff availability, and workload. When these records are examined manually, there exists a potential for scheduling conflicts, inappropriate room assignments, and erroneous staff allocations. This functionality integrates the distribution procedure with fixed validation standards that are enforced during the process prior to record retention. Artificial intelligence is employed where decision-making is crucial, especially in selecting appropriate educators. Application-level validation enforces scheduling and allocation limits.

Frontend Microservice Description

- Functions as a static HTML and JavaScript interface sent over nginx.
- The CSS theme that the team has concurred upon is utilized, and a link to the Faculty Management System homepage is included.
- Provides separate tabs for:
 - Subjects
 - Subject Offers
 - Classrooms
 - Teaching Allocations
 - AI Recommendations
- Support CRUD operations for the managed records.
- Teaching Allocation forms include:
 - subject offering
 - optional assigned staff member
 - classroom
 - teaching day
 - date range
 - start and end time
 - class type
 - expected class size
 - allocation status
- Shows the classroom's capacity and verifies the expected class size against the chosen classroom prior to submission.
- The AI Recommendations section enables users to solicit prioritized suggestions by choosing an allocation that necessitates personnel assignment.
- AI results are conveyed in the subsequent formats: burden details, personnel identification, proficiency, evaluation, and the justification for the suggestions.
- The final staff allocation is an ultimate user-control action and is not doable by AI.

Backend/API Microservice Description

- Deployed using Python and Flask
- Exposes REST endpoints for:
 - Subjects
 - Subject Offers
 - Classrooms
 - Teaching Allocations
 - AI staff recommendations
- Facilitates interactions among the frontend, Student 2 database service, and associated faculty microservices.
- Separates business rule verification from data storage.
- Teaching allocation validation includes:
 - required fields
 - valid teaching days
 - valid class types and statuses
 - valid date and time formats
 - start time before end time
 - classroom capacity

- classroom scheduling conflicts
 - staff existence and availability where applicable
- Utilizes REST APIs to obtain staff and expertise information from the Academic Personnel Management microservice.
- Acquires context regarding workload and availability from the Workload and Availability Management microservice.
- It does not directly access the databases of other students, therefore preserving microservice ownership and encapsulation. However, the prior report indicates that the design stipulates the team utilizes REST for inter-service communication rather than direct database access.

AI-Mode Description

- Employing the standard ollama runtime alongside an authorized gwen model.
- Formulates suggestions for the allocation of staff assignments.
- Builds the LLM context from:
 - subject requirements
 - teaching allocation information
 - staff expertise
 - staff availability
 - workload information
- Utilizes external prompt files rather than hard-codes all prompt commands in program logic.
- Directs the model to make three most suitable recommendations to the user.
- It examines content created by AI before presenting it on the user interface. It also considers if any content is unreliable.
- Rejects recommendations that:
 - contain duplicate staff IDs
 - reference staff not supplied to the model
 - reference staff who no longer exist
 - select unavailable staff
- Facilitates an adaptive recommendation process by providing prior failure rationales and historical recommendation identifiers during the regeneration of ideas.
- AI suggestions serve solely as guidance and do not instantaneously transform into instructional responsibilities.

Database Microservice Description

- Operates as an independent Flask microservice database utilizing SQLite.
- The Student 2 backend interacts with the database through REST API requests, rather than through direct access to the sqlite database.
- Stores four primary tables:
 - Subjects
 - SubjectOffers
 - Classrooms
 - TeachingAllocations
- 'Subjects' table stores subject code, name and required expertise.
- 'SubjectOffers' table stores the subject offering, semester, year and expected enrolment.

- 'Classrooms' table stores room location, capacity, room type and facilities.
- 'TeachingAllocations' table stores:
 - subject offer
 - assigned staff ID
 - classroom
 - teaching day
 - date range
 - start/end time
 - class type
 - expected class size
 - allocation status
- Employs foreign keys among domestically owned entities to ensure referential integrity.
- The assigned_staff_member is stored as an external reference instead of a local foreign key, as the definitive staff record is managed by the Academic Staff Management microservice.
- Database tables are populated with a minimum of ten entries as stipulated by the project requirements.

Integration with Team Application

- Accessible from the shared Faculty Management System homepage.
- Access the Faculty Management System website in a singular manner.
- Employs the standard Team CSS theme.
- Student 2 has compartmentalized the frontend, backend, and database into distinct containers, all linked by a unified docker-compose.yml file.
- Integrates with:
 - Student 1 Academic Staff Management for staff records and expertise
 - Student 3 Workload and Availability Management for workload and availability
 - shared Ollama service for AI recommendation generation
- It interacts with services through REST APIs instead than duplicating data maintained by others.
- Functions as a component of the comprehensive Faculty Management System implemented locally.

Teaching, Subject and Classroom Allocation - Risk Management Plan

ID	Risk	Probability	Impact	Risk Level	Mitigation	Contingency
----	------	-------------	--------	------------	------------	-------------

R1	Shared Ollama runtime or LLM becomes unavailable, causing AI recommendations to fail.	Medium	Medium	Medium	Keep CRUD and allocation validation independent of AI-Mode. Handle Ollama/API failures without crashing the backend.	Allow normal subject, classroom and allocation management to continue while AI-Mode is unavailable. Restart Ollama or the affected container.
R2	Qwen returns malformed, incomplete or invalid JSON that cannot be parsed as a recommendation response.	Medium	Medium	Medium	Constrain the prompt to a strict JSON structure and validate the returned response before use.	Reject the invalid response with a controlled error and allow the user to generate recommendations again. Improve retry/fallback handling in a later release.
R3	Academic Staff Management or Workload and Availability services become unavailable, preventing retrieval of staff expertise, workload or availability.	Medium	High	Medium-High	Access external data through dedicated REST clients with timeouts/error handling and keep locally owned CRUD functionality independent.	Disable or return an error for affected recommendation/all location operations while allowing local subject, classroom and allocation management to remain available.

R4	The LLM recommends an invalid, duplicate, nonexistent or unavailable staff member.	Medium	High	High	Treat AI output as untrusted. Verify recommended IDs against the candidate list and current staff/availability information before returning recommendations.	Reject invalid recommendations, record the failure reason and request another recommendation cycle where appropriate.
R5	Incorrect allocation validation allows a room that is too small or creates an overlapping classroom booking.	Low-Medium	High	Medium-High	Keep capacity and scheduling checks in deterministic backend business rules and cover critical rules with automated tests.	Reject conflicting allocations when detected, correct the validation rule and rerun affected tests before merging.
R6	API endpoint or data-format changes in another student's microservice break cross-service integration.	Medium	High	Medium-High	Keep service interfaces clearly defined and coordinate endpoint/schema changes between dependent team members before merging.	Update the relevant client module and integration tests, or temporarily revert to the last compatible commit.

R7	Docker Compose configuration, service naming or environment variables prevent Student 2 services from connecting to other containers.	Medium	Medium	Medium	Use Docker service names and environment variables for internal service URLs and validate images through CI before integration.	Inspect container logs/network configuration, rebuild affected services and revert configuration changes if required.
R8	Limited automated testing allows regressions in CRUD or allocation business rules to reach the integrated application.	Medium	Medium	Medium	Maintain backend/data base tests for critical rules and run them through the Student 2 GitHub Actions workflow.	Roll back to the latest working commit, reproduce the failure locally, add a regression test and fix before re-merging.

Teaching, Subject and Classroom Allocation - Data Design

Conceptual Design

- Subject - Subject Offers: One-to-many
 - A subject can have many subject offers while each subject offer must be linked to only one subject.
- Subject Offer - Teaching Allocations: One-to-many
 - A subject offer can have multiple teaching allocations while each teaching allocation belongs to one subject
- Classroom - Teaching Allocations: One-to-many
 - A classroom can be used by many teaching allocations across different schedules while each teaching allocation uses one classroom
- Teaching Allocation - Staff member: Many-to-one external relationship
 - A teaching allocation may optionally be assigned to one academic staff member. A staff member may be associated with many teaching allocations. However, staff information is stored and owned by the Academic Staff Management microservice rather than stored locally.

- Each teaching allocation portrays a scheduled teaching plan by combining a subject offer, classroom, day and date range, time period, class type and expected class size.
- Classroom capacity is used when validating whether a classroom is usable for the expected class size of a teaching allocation. (i.e. Expected class size $\leq$ Classroom Capacity)

Logical Design

Entity	Key Attributes	Relationship
Subjects	subject_code (PK), name, required_expertise	One Subject can have many Subject Offers.
Subject Offers	offer_id (PK), subject_code (FK), semester, year, expected_enrollment	Many Subject Offers belong to one Subject. One Subject Offer can have many Teaching Allocations.
Classrooms	classroom_id (PK), building, floor, room_number, capacity, room_type, facilities	One Classroom can be referenced by many Teaching Allocations across different schedules.
Teaching Allocations	allocation_id (PK), offer_id (FK), assigned_staff_member , classroom_id (FK), day, date_range, start_time, end_time, class_type, expected_class_size, allocation_status	Many Teaching Allocations belong to one Subject Offer and one Classroom. assigned_staff_member optionally references a staff member owned externally by the Academic Staff Management microservice.

The local foreign key implementation on assigned_staff_member is deliberately omitted since the staff records are maintained by the Academic Staff Management microservice. Student 2 only preserves the external staff identity and acquires official staff details using the Staff Management REST API.

Physical Design

SQLite is used for the implementation of the Teaching, Subject and Classroom Allocation database in allocation.db and initialized with database/init_db.py pragma foreign_keys=on; Enable foreign-key support. The present implementation contains 4 tables, subject, subject_offers, classroom and teaching_allocations.

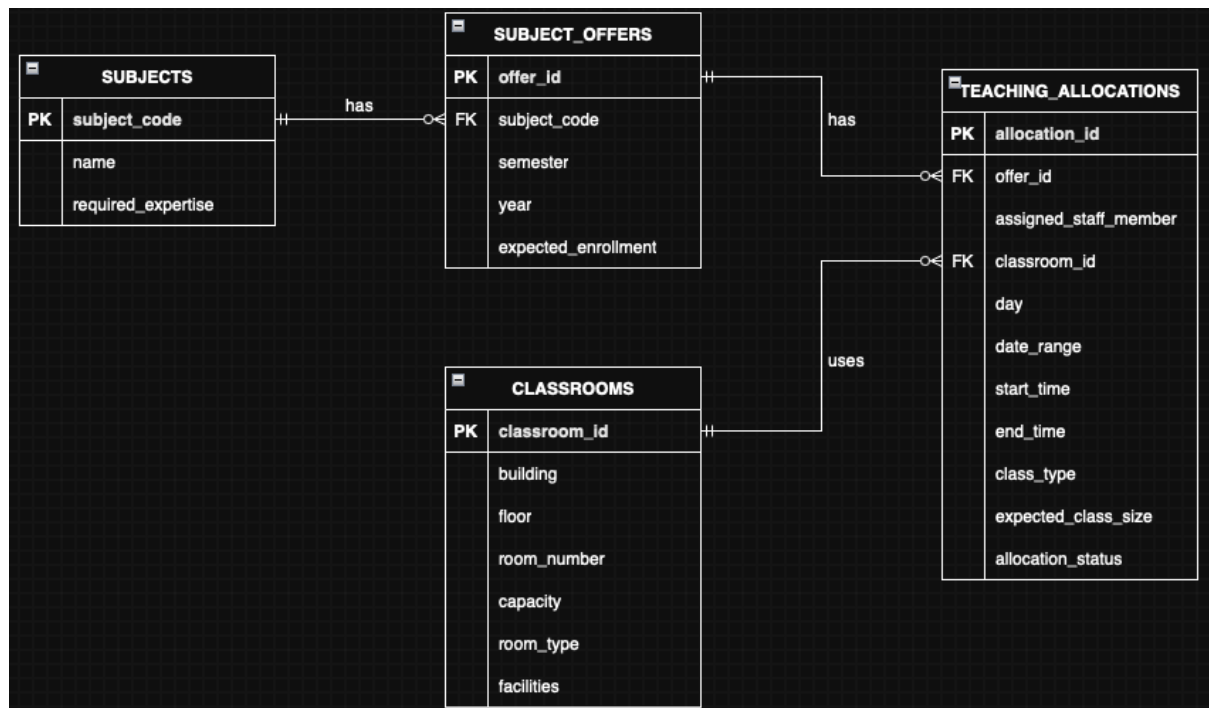
```
CREATE TABLE subjects (  
    subject_code TEXT PRIMARY KEY,  
    name TEXT NOT NULL,  
    required_expertise TEXT NOT NULL  
)
```

```
CREATE TABLE subject_offers (  
    offer_id TEXT PRIMARY KEY,  
    subject_code TEXT NOT NULL,  
    semester TEXT NOT NULL,  
    year TEXT NOT NULL,  
    expected_enrollment INTEGER NOT NULL,  
    FOREIGN KEY (subject_code)  
        REFERENCES subjects(subject_code)  
        ON DELETE RESTRICT  
)
```

```
CREATE TABLE classrooms (  
    classroom_id TEXT PRIMARY KEY,  
    building TEXT NOT NULL,  
    floor TEXT NOT NULL,  
    room_number TEXT NOT NULL,  
    capacity INTEGER NOT NULL,  
    room_type TEXT NOT NULL,  
    facilities TEXT NOT NULL  
)
```

```
CREATE TABLE teaching_allocations (  
    allocation_id INTEGER PRIMARY KEY AUTOINCREMENT,  
    offer_id TEXT NOT NULL,  
    assigned_staff_member INTEGER,  
    classroom_id TEXT NOT NULL,  
    day TEXT NOT NULL,  
    date_range TEXT NOT NULL,  
    start_time TEXT NOT NULL,  
    end_time TEXT NOT NULL,  
    class_type TEXT NOT NULL,  
    expected_class_size INTEGER NOT NULL,  
    allocation_status TEXT NOT NULL,  
    FOREIGN KEY (offer_id)  
        REFERENCES subject_offers(offer_id)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE,  
    FOREIGN KEY (classroom_id)  
        REFERENCES classrooms(classroom_id)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
)
```

Entity Relationship Diagram



Note: assigned_staff_member is an external reference to a staff record owned by the Academic Staff Management microservice and is therefore not represented as a local foreign-key relationship.

Workload and Availability Management(Tristan Lim)

Workload and availability Management – Feature Plan

Description

The workload and availability management feature gives a faculty workload coordinator a single, live view of every academic's contracted hours, teaching and research commitments, availability and leave. Hours are recomputed from individual workload entries rather than stored as a single figure, so overloaded and underloaded status always reflects the current data. An AI agent can propose a redistribution of an overloaded staff member's work, with every proposal checked against the underlying rules engine before it is shown.

Problem Addressed

Workload is currently tracked informally, making it difficult to see who is overloaded, who has spare capacity or where a scheduling conflict has occurred. For example, an approved leave overlapping a teaching commitment. This feature centralises the tracking, applies caps automatically, and uses AI assistance to suggest a fairer redistribution of work before allocations are locked in, while keeping a human in control of the final decision.

Frontend Microservice Description

- Runs a static HTML / JavaScript application served through nginx on its own port, which follows the shared CSS theme
- Provides a computed workload overview across all staff, filterable by status and department
- Provides a staff detailed view of activity breakdown and clashes as well as a colour coded weekly availability calendar for one staff member
- Reads the staff directory live from student 1 (Academic Staff Management) service, including a health check and a reconciliation view that joins the two services' data on staff_id
- Provides the full CRUD forms for workload entries, availability and leave requests
- Provides CSV export across nine datasets and a separate printable report page that can be saved as a PDF
- Provides an AI mode tab which has a health check, rebalance workflow, saved recommendations queue, and a free text question form.
- Communicates with the backend via fetch() calls, receiving ready to render HTML fragments rather than JSON

Backend Microservice Description

- Implemented as a FLASH REST API on its own port, with CRUD endpoints for every table driven by a dedicated rules engine module
- Applies input validation and consistent error handling
- Implements AI mode by calling the shared Ollama runtime
- Reads staff identity from the Student 1 service over the network rather than duplicating it

Database Microservice Description

- Uses SQLite, containerized as its own database microservice
- Stores seven tables
- Seeded with at least 10 records in every table
- Staff_id values and staff names are aligned with student 1 services.

Integration with Team Application

- Accessible from the unified home page alongside the other four features and follows the team's shared CSS theme
- Integrated into the shared docker-compose.yml as three services
- Exposes and uses related endpoints for other microservices to read workload status.

Workload and availability Management – Risk management Plan

ID	Risk	Probability	Impact	Risk Level	Mitigation	Contingency
R1	The local LLM hallucinates activity	High	Medium	High	Validate every AI proposal against the	Reject the proposal automatically and display

	names or proposes reassignments that exceed a target's cap.				rules engine before it is shown or saved.	the reason. Fall back to larger model if available
R2	Staff_id values in this feature do not match the student 1 roster, making cross-service join useless.	High	High	High	Align seed data to the student 1 roster and add a reconciliation endpoint to verify the join	Expose any mismatch explicitly rather than silently joining incorrect records
R3	The shared Ollama container fails to start because its published port conflicts with a locally install Ollama	Medium	Medium	Medium	Do not publish the Ollama container's port. Every service reaches it by container name over the internal network	Document the fix for the team so no student is blocked bringing up the full stack
R4	Incomplete Microservices affect delivery by the release deadline	Medium	High	Medium	Build incrementally in a fixed order so a partially complete feature still demonstrates CRUD and the rules engine	Prioritise CRUD and the rules engine over AI-mode

Workload and Availability Management – Data Design

Conceptual

- A staff member (id owned by Student 1) has many workload entries, one per teaching, research, professional development, service or admin commitment (one to many)
- A staff member has many availability slots, each a recurring or one-off block of time marked available, preferred or unavailable (one to many)
- A staff member has many leave records, each with a type, date range, and approval status (one to many)
- A staff has one workload profile per semester, recording their contracted fraction and cap (one to one, per semester)

- A workload rule applies to either a specific department or to all staff and is used to resolve the cap for a profile (one to many)
- A workload alert is raised against a staff member when their computed load or a scheduling conflict breaches a rule (one to many)
- A rebalance recommendation is generated in response to an alert and may reference a specific staff member the work would move to (one to many, with an optional self-referencing link to another staff member)

Logical

Entity	Key Attribute	Relationship
Staff Workload profile	Profile_id (PK), staff_id, staff_name, department, semester, contracted_fraction, max_weekly_hours, current_total_hours, status	One profile per staff member per semester
Workload Entry	entry_id (PK), staff_id, activity_type, description, hours_per_week, start_date, end_date, semester, source_service	Many entries to one staff member
Availability Slot	Slot_id (PK), staff_id, day_of_week, start_time, end_time, availability, is_recurring	Many slots to one staff member
Leave Record	Leave_id (PK), staff_id, leave_type, start_date, end_date, reason, approval	Many leave records to one staff member
Workload Rule	Rule_id (PK), name, applies_to, max_total_hours, warning_threshold	One rule to many profiles
Workload Alert	Alert_id (PK), staff_id, alert_type, severity, message, status, date_raised	Many alerts to one staff member
Rebalance Recommendation	Rec_id (PK), alert_id (FK), staff_id, suggested_action, target_staff_id, rationale, decision_status	Many recommendations to one alert, optional link to a target staff member

Physical

Implemented via SQLite, workload.db

CREATE TABLE staff_workload_profile (

```
profile_id INTEGER PRIMARY KEY,  
staff_id INTEGER NOT NULL,  
staff_name TEXT NOT NULL,  
department TEXT NOT NULL,  
semester TEXT NOT NULL,  
contracted_fraction REAL NOT NULL,  
max_weekly_hours REAL NOT NULL,  
current_total_hours REAL NOT NULL DEFAULT 0,  
status TEXT NOT NULL DEFAULT 'ok'  
);
```

```
CREATE TABLE workload_entry (  
entry_id INTEGER PRIMARY KEY,  
staff_id INTEGER NOT NULL,  
activity_type TEXT NOT NULL,  
description TEXT,  
hours_per_week REAL NOT NULL,  
start_date TEXT NOT NULL,  
end_date TEXT NOT NULL,  
semester TEXT NOT NULL,  
source_service TEXT DEFAULT 'manual'  
);
```

```
CREATE TABLE availability_slot (  
slot_id INTEGER PRIMARY KEY,  
staff_id INTEGER NOT NULL,  
day_of_week TEXT NOT NULL,  
start_time TEXT NOT NULL,  
end_time TEXT NOT NULL,  
availability TEXT NOT NULL DEFAULT 'available',  
is_recurring INTEGER NOT NULL DEFAULT 1  
);
```

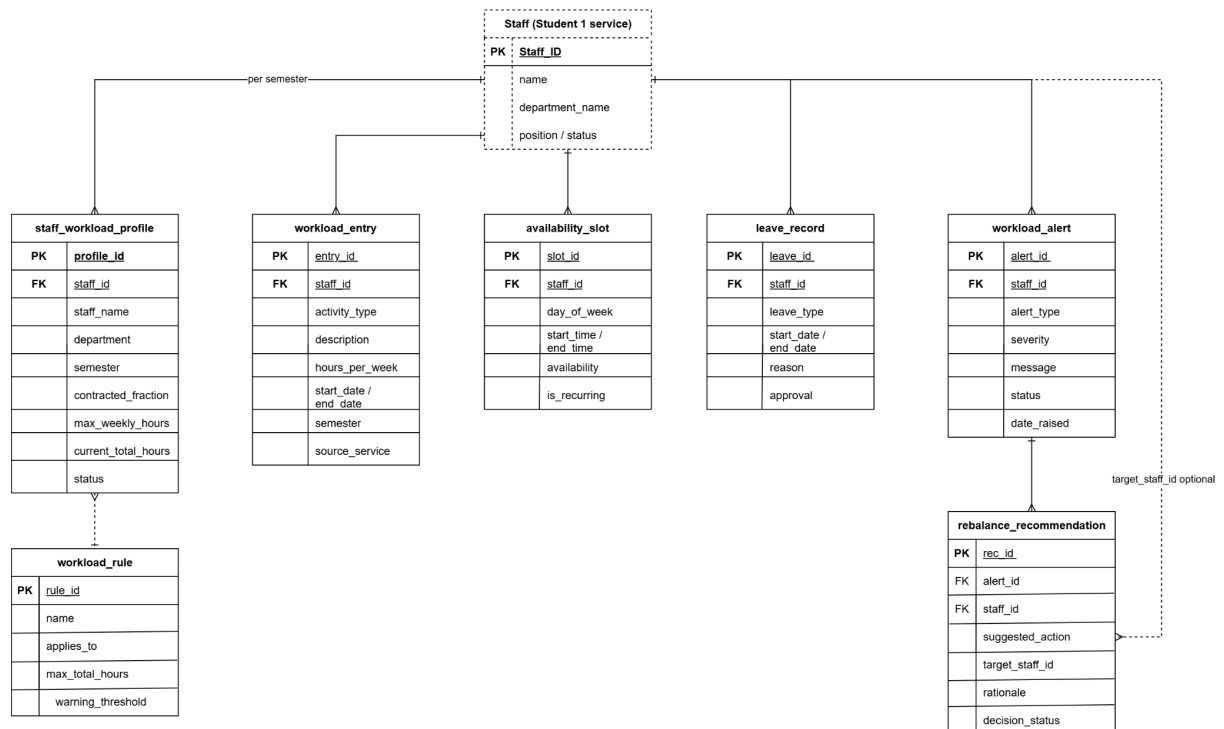
```
CREATE TABLE leave_record (  
leave_id INTEGER PRIMARY KEY,  
staff_id INTEGER NOT NULL,  
leave_type TEXT NOT NULL,  
start_date TEXT NOT NULL,  
end_date TEXT NOT NULL,  
reason TEXT,  
approval TEXT NOT NULL DEFAULT 'pending'  
);
```

```
CREATE TABLE workload_rule (  
rule_id INTEGER PRIMARY KEY,  
name TEXT NOT NULL,  
applies_to TEXT NOT NULL DEFAULT 'all',  
max_total_hours REAL NOT NULL,
```

$$);$$
$$);$$
$$);$$

- No foreign key is defined between tables. Staff_id is a soft reference resolved against the student 1 service rather than a local staff table.

Entity Relationship Diagram



Research and Grant Management (Nicholas Hatzidimitriou)

Research and Grant Management: Feature Plan

Description

Research and Grant Management will provide faculty admins with a central hub to create, read, update and delete research projects, update grant applications and publications. A research project will link to a grant, publications and many assigned staff. Within this microservice, three AI mode capabilities are offered: generated research summary for a project, a rationale explaining why currently assigned staff suit the project and an AI recommendation of new staff for a research project based on expertise and workload data retrieved from other faculty microservices.

Problem Addressed

Since research funding, deadlines and staff expertise are typically tracked across disconnected records, this microservice will create a central hub for managing projects and grant applications, while tracking funding, deadlines and research outputs produced. Moreover, the AI staff matching capability will allow faculty admins to identify well suited staff for research opportunities based on their expertise and workload rather than relying on manual cross checking.

Frontend Microservice Description

- It will run on a dedicated port as a Flask application via HTMX instead of a static page.
- Provide a detailed view of a project. Including its grants, publications and assigned staff. Staff names are updated live via the Staff Management microservice.
- Allows for the addition of new research projects.
- AI insights tab with three actions: generate research summary, rationale behind current staff suitability and recommendation for new staff
- Communicate with the backend via REST API calls
- Adopt a shared CSS theme

Backend Microservice Description

- Implemented as a Flask REST API that contains CRUD operations and exposing endpoints
- Apply input validation and error handling for missing records, required fields and invalid foreign references. (if a grant is created against a project that doesn't exist)
- Implement AI capability by calling the shared Ollama runtime for all three features, using feature specific prompts.
- Implement dedicated client modules to retrieve staff expertise from the Staff Management microservice and workload and availability status from Workload Management microservice. This ensures that staff recommendations are based on real data instead of hallucinated by the AI.

Database Microservice Description

- Uses SQLite, containerised as its own database microservice that exposes its data over a REST API instead of a shared file.
- Store records of research projects, grants, publications, project staff assignments, grant alerts and AI generated analysis.
- Seeded with 10 records

Implementation with Team application

- The entire feature will be accessible from the unified homepage and follow the team's shared CSS theme.
- Requires integration into the shared docker-compose.yml configuration file, running alongside the other containers.
- Consumes endpoints exposed by the Staff Management and Availability Management microservices.

Research and Grant Management: Risk Management

ID	Risk	Probability	Impact	Risk Level	Mitigation	Contingency
R1	The shared ollama run time goes down / becomes unavailable causing AI-mode endpoints to fail	Med	Med	Med	Wrap it in a try/except returning a 503 error instead of crashing the entire thing.	Ensure CRUD operations remain functional and independent of AI mode.
R2	Staff Management or Workload Service become unavailable, breaking the recommend new staff feature	Med	Med	Med	Staff_client.py and workload_client.py catch failures and display "unknown" placeholders.	The feature still remains usable but doesn't display real time data. Only displays whichever data is available.
R3	Naming mismatches between route handlers and server-layer functions	High	Med	Med	Adopt consistent naming across routes / services. Add	Use previous Git commit if naming mismatch gets pushed to main

					CI syntax to catch mismatches	
R4	AI generated staff recommendations misrepresent a staff's expertise or availability	Med	Med	Med	Prompts explicitly instruct the model to reason only with the evidence provided (explicitly from expertise (Andy) and workload data (Tristan) and recommend candidates from the list.	Presented AI output should be clearly labelled as a recommendation for an overseer to review and not an automated decision.

Research and Grant Management: Data Design

Conceptual

- A research project can have many grant applications (1 to many)
- A research project can have many publications (1 to many)
- A research project can have many staff assigned to it, each with a role (1 to many, via linking record)
- A grant can have many deadline/status alerts (1 to many)
- AI analysis record is generated for either a research project or staff member, a project or staff member can have any AI analysis records (1 to many)

Logical

Entity	Key Attributes	Relationship
ResearchProjects	projectID (PK), title, description, department, status, startdate, endDate, leadStaffID	One researchprojects has many grants, publications, and projectstaff records has ResearchAIAnalysis
Grants	grantID (PK), projectID (FK), fundingBody, amountRequested,	Many grants belong to one ResearchProject, one Grant

	amountAwarded, applicationDeadline, status	can have many GrantAlerts
Publications	publicationID (PK), projectID (FK), staffID, title, publicationType, journalOrVenue, datePublished	Many publications to one ResearchProject
ProjectStaff	projectStaffID (PK), projectID (FK), staffID, role	Many staff-assignments to one ResearchProject
GrantAlerts	alertID (PK), grantID (FK), alertType, dueDate, status	Many alerts to one grant
ResearchAIAnalysis	analysisID (PK), projectID (FK, nullable), staffID (nullable), generatedSummary, recommendedStaffMatches, matchRationale, dateGenerated	Many ResearchAIAnalysis to one project (optional)

Physical

Implemented via SQLite, student4.db SQL:

```
CREATE TABLE ResearchProjects (
  projectID INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  description TEXT,
  department TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('proposed', 'in progress', 'completed', 'on
hold')),
  startDate DATE,
  endDate DATE,
  leadStaffID INTEGER
);
```

```
CREATE TABLE Grants (
  grantID INTEGER PRIMARY KEY AUTOINCREMENT,
  projectID INTEGER NOT NULL,
  fundingBody TEXT NOT NULL,
  amountRequested REAL NOT NULL,
  amountAwarded REAL,
  applicationDeadline DATE NOT NULL,
```

```
    status          TEXT NOT NULL CHECK (status IN ('applied', 'rejected', 'awarded',
'completed', 'in progress')),
    FOREIGN KEY (projectId) REFERENCES ResearchProjects(projectID)
);
```

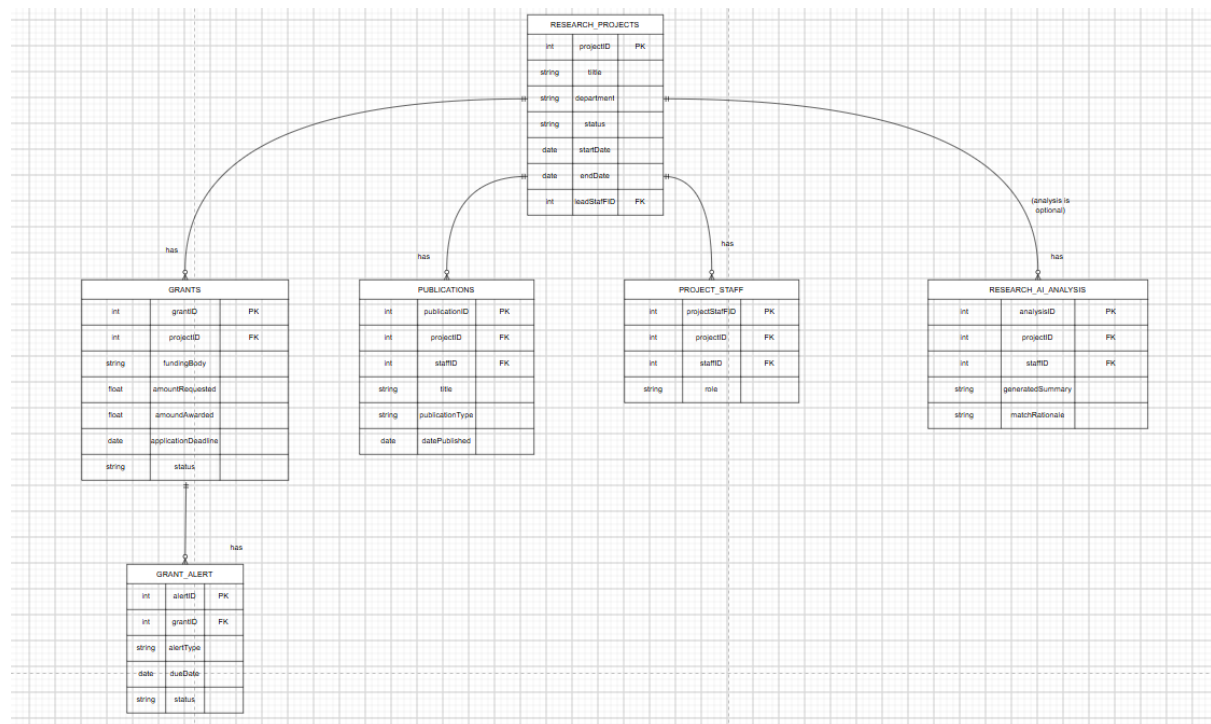
```
CREATE TABLE Publications (
    publicationID    INTEGER PRIMARY KEY AUTOINCREMENT,
    projectId        INTEGER NOT NULL,
    staffID          INTEGER,
    title            TEXT NOT NULL,
    publicationType  TEXT NOT NULL CHECK (publicationType IN ('journal', 'conference',
'book chapter', 'report')),
    journalOrVenue  TEXT,
    datePublished   DATE,
    FOREIGN KEY (projectId) REFERENCES ResearchProjects(projectID)
);
```

```
CREATE TABLE ProjectStaff (
    projectStaffID  INTEGER PRIMARY KEY AUTOINCREMENT,
    projectId        INTEGER NOT NULL,
    staffID          INTEGER NOT NULL,
    role            TEXT NOT NULL CHECK (role IN ('lead', 'co-investigator', 'research assistant',
'contributor')),
    FOREIGN KEY (projectId) REFERENCES ResearchProjects(projectID)
);
```

```
CREATE TABLE GrantAlerts (
    alertID         INTEGER PRIMARY KEY AUTOINCREMENT,
    grantID         INTEGER NOT NULL,
    alertType       TEXT NOT NULL CHECK (alertType IN ('deadline approaching', 'missing
documents', 'status change', 'overdue')),
    dueDate        DATE NOT NULL,
    status         TEXT NOT NULL CHECK (status IN ('active', 'dismissed', 'resolved')),
    FOREIGN KEY (grantID) REFERENCES Grants(grantID)
);
```

```
CREATE TABLE ResearchAIAalysis (
    analysisID      INTEGER PRIMARY KEY AUTOINCREMENT,
    projectId        INTEGER,
    staffID          INTEGER,
    generatedSummary TEXT,
    recommendedStaffMatches TEXT,
    matchRationale  TEXT,
    dateGenerated   DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (projectId) REFERENCES ResearchProjects(projectID)
);
```

Entity Relationship Diagram



Performance and Professional Development (Matthew Barnard)

Performance and Professional Development: Feature Plan

Description

The Performance and Professional Development feature provides faculty administrators with a central system for managing staff performance reviews, development goals, training programs and staff training. It also provides AI-assisted professional development recommendations based on a staff member's performance, goals, current training and available training programs. Recommendations are stored for users to accept, reject or modify.

Problem Addressed

Performance reviews, professional development goals and training records can become separated across different systems, making it difficult to track staff development and identify appropriate training. This feature centralises these records and uses AI assistance to generate development recommendations while keeping the final decision under human control.

Frontend Microservice Description

- Runs through Nginx on port 8005.

- Provides access to performance reviews, development goals, training programs, staff training and development recommendations.
- Communicates with the backend through the Nginx proxy.
- Allows users to review and change the status of AI-generated recommendations.

Backend Microservice Description

- Implemented using Python and Flask on port 5005.
- Provides CRUD operations for all five managed resources.
- Retrieves staff profiles and expertise from the Student 1 Staff Management service.
- Applies validation before sending data to the database service.
- Implements AI-Mode through the shared Ollama runtime.

AI Mode Description

- Uses qwen2.5:1.5b through the shared Ollama runtime.
- Builds staff context from performance reviews, development goals, current training, available training programs and Student 1 staff information.
- Validates the generated recommendation before saving it.
- Makes a second model attempt if the first result is invalid, followed by a grounded training fallback if required.
- Saves valid recommendations in DevelopmentRecommendations with a default status of Pending.

Database Microservice Description

- Uses SQLite through a separate Flask database service on port 5105.
- Database file is performance_dev.db.
- Stores five tables: PerformanceReviews, DevelopmentGoals, TrainingPrograms, StaffTraining and DevelopmentRecommendations.
- Each table is seeded with 10 records.
- Staff IDs remain external references because staff ownership belongs to Student 1.

Integration with Team Application

- Runs as three separate frontend, backend and database containers.
- Integrated into the shared Docker Compose configuration.
- Uses Student 1's Staff Management REST API for staff information and expertise.
- Uses the shared Ollama service for AI recommendations.
- Performance and Professional Development does not currently integrate with Workload and Availability.

Performance and Professional Development: Risk Management

ID	Risk	Probability	Impact	Risk Level	Mitigation	Contingency
R1	Shared Ollama service becomes unavailable, preventing AI recommendations	Medium	Medium	Medium	Keep CRUD operations independent of AI and handle model failures gracefully	Continue normal feature operation and restore Ollama when available

R2	AI produces malformed or unsupported recommendation output	Medium	Medium	Medium	Validate JSON, recommendation type, goal ID and required fields before persistence.	Retry the model once and use the grounded fallback if the second result fails.
R3	Student 1 Staff Management becomes unavailable	Medium	Medium	Medium	Use a dedicated REST client with timeout/error handling.	Continue locally owned operations and restore staff context when the service becomes available.
R4	Inconsistent development goal status and progress are stored	Medium	High	High	Apply cross field validation so Completed goals require 100% progress.	Reject the update and return a validation error to the user.
R5	AI recommends duplicate or already enrolled training	Medium	Medium	Medium	Validate recommendation against available training data and existing records	Allow the user to reject the recommendation and strengthen duplicate validation in a future release.

Performance and Professional Development: Data Design

Conceptual

- A staff member can have many performance reviews.
- A staff member can have many development goals.
- A training program can have many staff training enrolments.
- A staff member can enrol in many training programs through StaffTraining.
- A development goal can have many development recommendations.
- Staff IDs are external references to Student 1 rather than local foreign keys.
- A recommendation may optionally relate to a specific development goal.

Logical

Entity	Key Attributes	Relationship
Performance Reviews	reviewID (PK), staffID, reviewDate, reviewerID, rating, feedback, status	Many reviews to one external staff member

Development Goals	goalID (PK), staffID, title, description, targetDate, progress, status	Many goals to one external staff member
Training Programs	trainingID (PK), title, description, provider, startDate, endDate, skillArea	One training program to many StaffTraining records
Staff Training	staffTrainingID (PK), staffID, trainingID (FK), enrolmentDate, completionDate, status	Many enrolments to one training program
Development Recommendations	recommendationID (PK), staffID, goalID (FK), recommendationType, recommendation, rationale, dateGenerated, status	Many recommendations to one optional development goal

Physical

Implemented using SQLite through performance_dev.db. The schema is defined in schema.py and each table is seeded with 10 records through seed_data.py.

```
CREATE TABLE IF NOT EXISTS PerformanceReviews (
    reviewID INTEGER PRIMARY KEY AUTOINCREMENT,
    staffID INTEGER NOT NULL CHECK (staffID > 0),
    reviewDate TEXT NOT NULL,
    reviewerID INTEGER NOT NULL CHECK (reviewerID > 0),
    rating REAL CHECK (rating BETWEEN 1 AND 5),
    feedback TEXT,
    status TEXT NOT NULL CHECK (
        status IN ('Draft', 'Scheduled', 'Completed', 'Acknowledged')
    )
);
```

```
CREATE TABLE IF NOT EXISTS DevelopmentGoals (
    goalID INTEGER PRIMARY KEY AUTOINCREMENT,
    staffID INTEGER NOT NULL CHECK (staffID > 0),
    title TEXT NOT NULL CHECK (length(trim(title)) BETWEEN 1 AND 120),
    description TEXT,
    targetDate TEXT,
    progress REAL NOT NULL DEFAULT 0 CHECK (progress BETWEEN 0 AND 100),
    status TEXT NOT NULL CHECK (
        status IN ('Planned', 'In Progress', 'Completed', 'On Hold', 'Cancelled')
    )
);
```

```

CREATE TABLE IF NOT EXISTS TrainingPrograms (
    trainingID INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL CHECK (length(trim(title)) BETWEEN 1 AND 160),
    description TEXT,
    provider TEXT,
    startDate TEXT,
    endDate TEXT,
    skillArea TEXT,
    CHECK (endDate IS NULL OR startDate IS NULL OR endDate >= startDate)
);

```

```

CREATE TABLE IF NOT EXISTS TrainingPrograms (
    trainingID INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL CHECK (length(trim(title)) BETWEEN 1 AND 160),
    description TEXT,
    provider TEXT,
    startDate TEXT,
    endDate TEXT,
    skillArea TEXT,
    CHECK (endDate IS NULL OR startDate IS NULL OR endDate >= startDate)
);

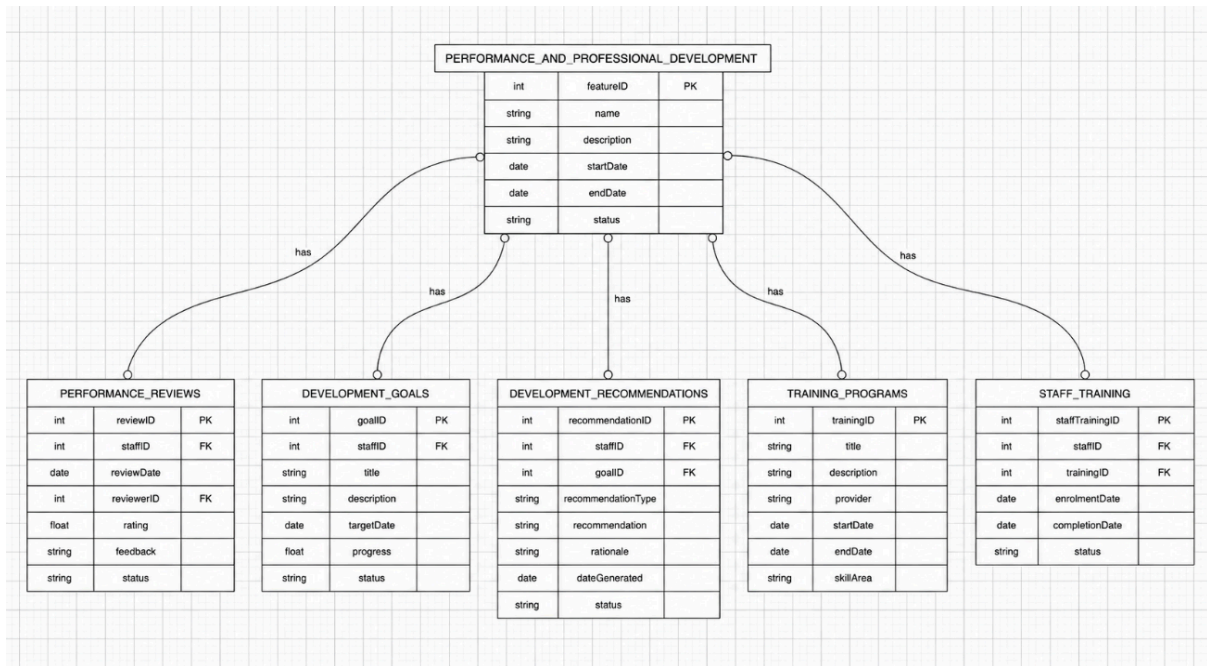
```

```

CREATE TABLE IF NOT EXISTS DevelopmentRecommendations (
    recommendationID INTEGER PRIMARY KEY AUTOINCREMENT,
    staffID INTEGER NOT NULL CHECK (staffID > 0),
    goalID INTEGER,
    recommendationType TEXT NOT NULL CHECK (
        recommendationType IN ('Training', 'Goal', 'Mentoring', 'Experience')
    ),
    recommendation TEXT NOT NULL,
    rationale TEXT,
    dateGenerated TEXT NOT NULL,
    status TEXT NOT NULL CHECK (
        status IN ('Pending', 'Accepted', 'Rejected', 'Modified')
    ),
    FOREIGN KEY (goalID) REFERENCES DevelopmentGoals(goalID)
    ON UPDATE CASCADE ON DELETE SET NULL
);

```

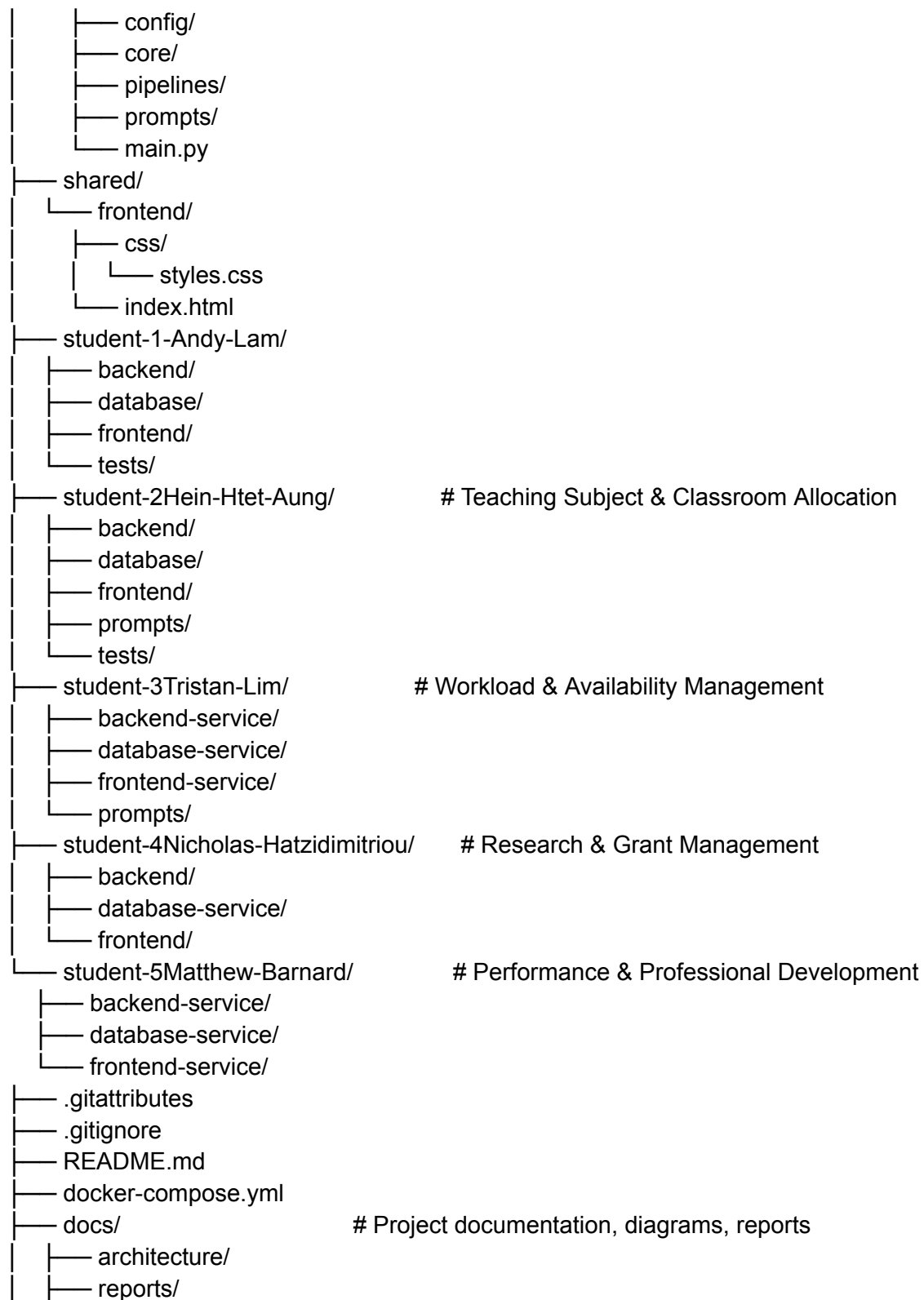
Entity Relationship Diagram



Repository Structure

faculty-management-system-ASD/





Repository Structure evidence from GitHub

▼ .github/workflows

-  student-1-AndyLam.yml
-  student-2-HeinHtetAung.yml
-  student-3-TristanLim.yml
-  student-4-NicholasHatzidimitriou.yml
-  student-5-MatthewBarnard.yml

▼ ai-services

▼ agentic_loop

> collectors

> config

> core

> pipelines

> prompts

 .env.example

 main.py

 requirements.txt

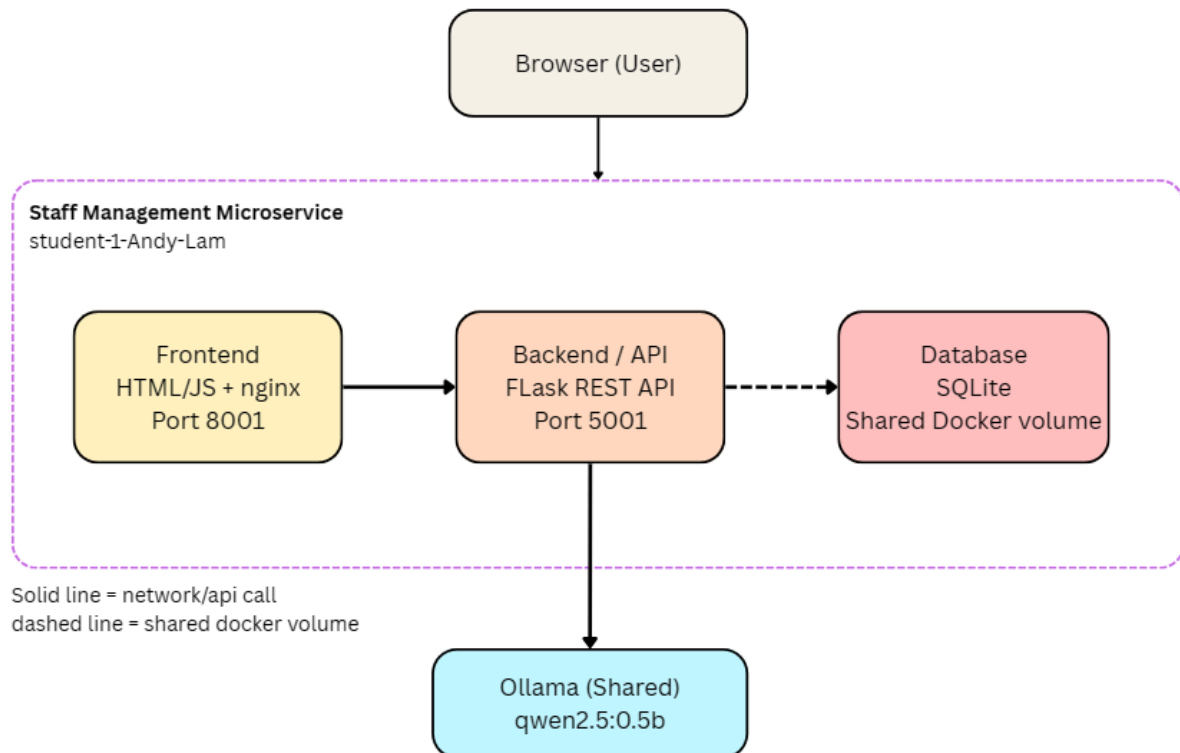
 agentic_loop.py

-
- ✓  shared/frontend
 - >  css
 -  index.html
 - ✓  student-1-Andy-Lam
 - >  backend
 - >  database
 - >  frontend
 - >  tests/manual
 -  Dockerfile
 - ✓  student-2Hein-Htet-Aung
 - >  backend
 - >  database
 - >  frontend
 - >  prompts/allocation
 - >  tests
 -  Dockerfile

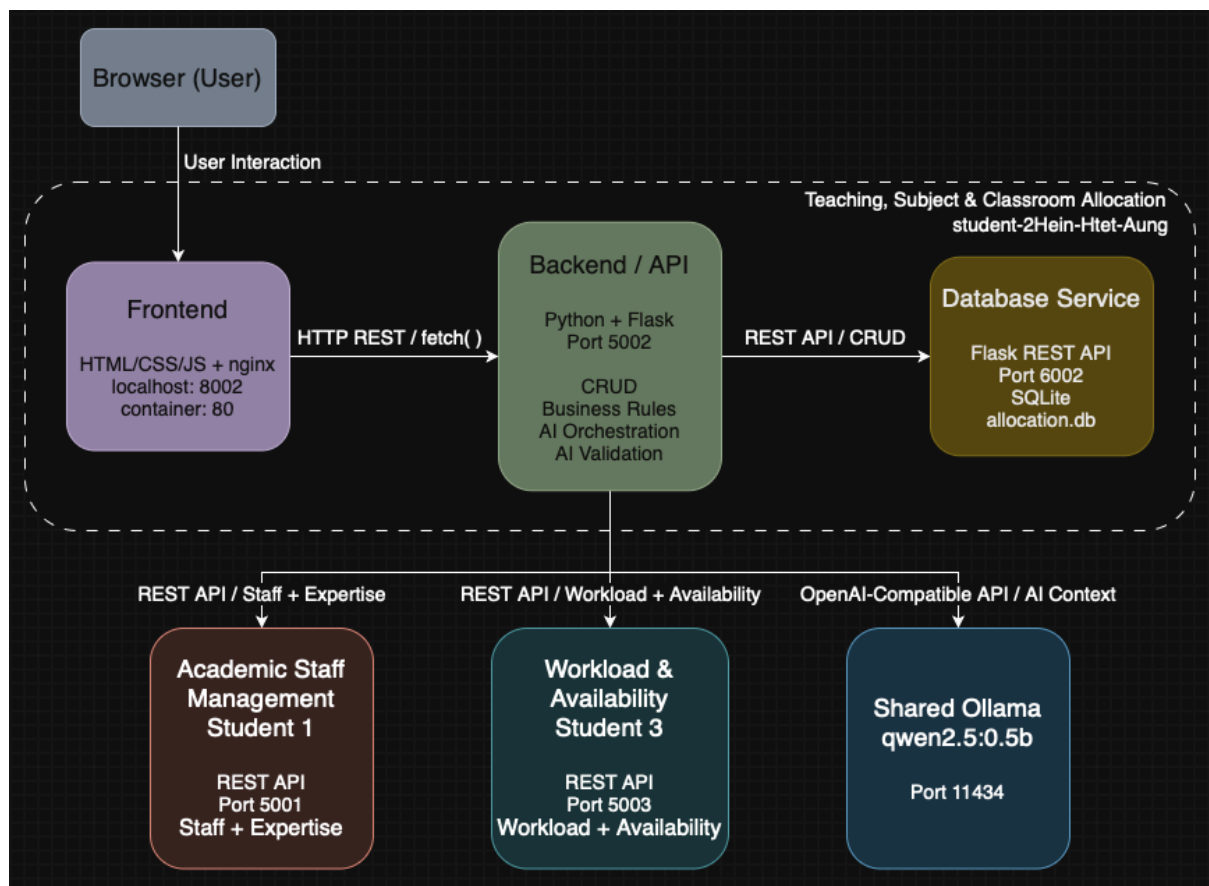
- ✓  student-3Tristan-Lim
 - >  backend-service
 - >  database-service
 - >  frontend-service
 - >  prompts/service
 -  Dockerfile
- ✓  student-4Nicholas-Hatzidimitriou
 - >  backend
 - >  database-service
 - >  frontend
 -  .gitignore
- ✓  student-5Matthew-Barnard
 - >  backend-service
 - >  database-service
 - >  frontend-service
 -  .gitignore
 -  README.md
 -  docker-compose.yml
 -  .gitattributes
 -  .gitignore
 -  README.md
 -  docker-compose.yml
- ✓  docs
 - >  architecture
 - >  reports

Individual Software Architecture

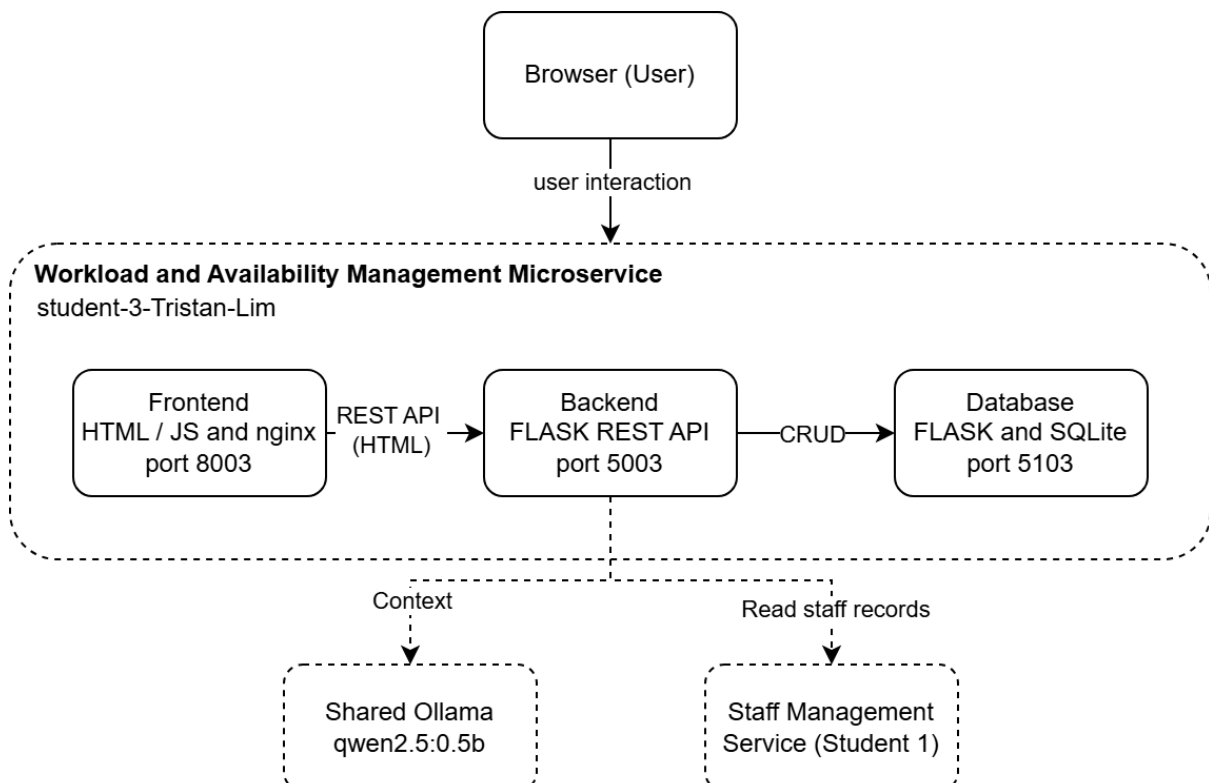
Academic Staff Management (Andy Lam)



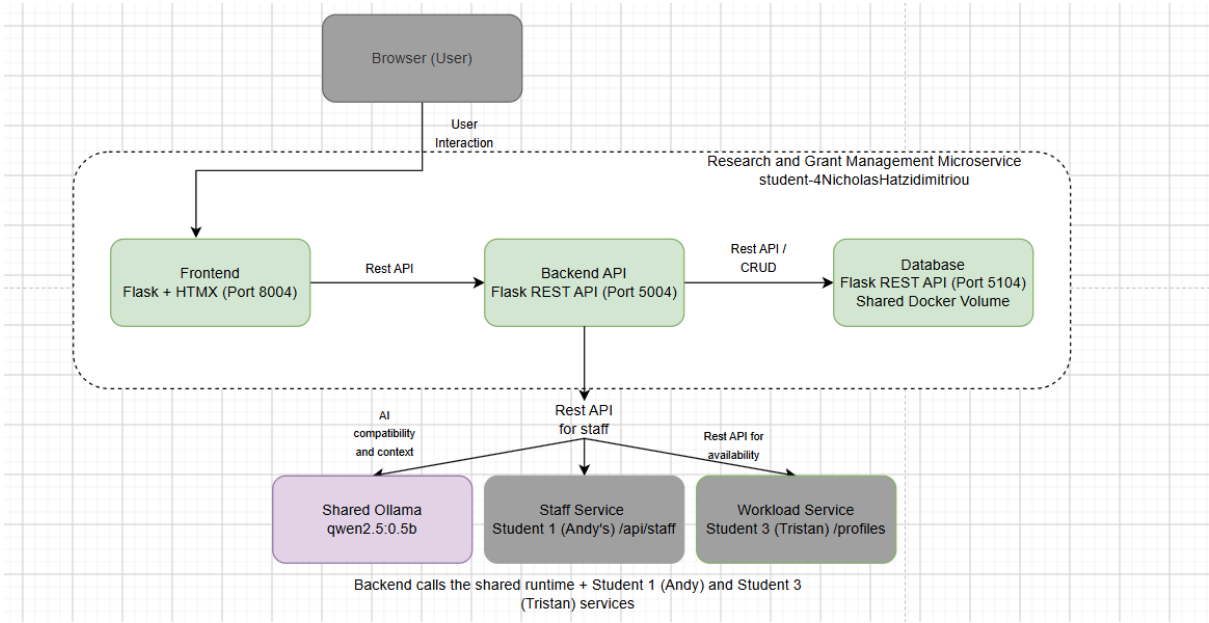
Teaching, Subject and Classroom Allocation (Hein Htet Aung)



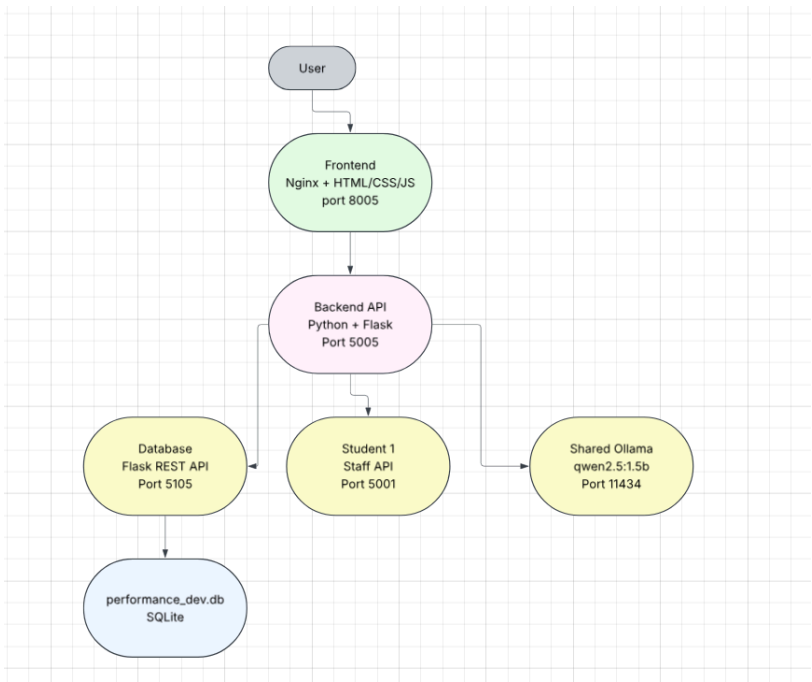
Workload and Availability Management (Tristan Lim)



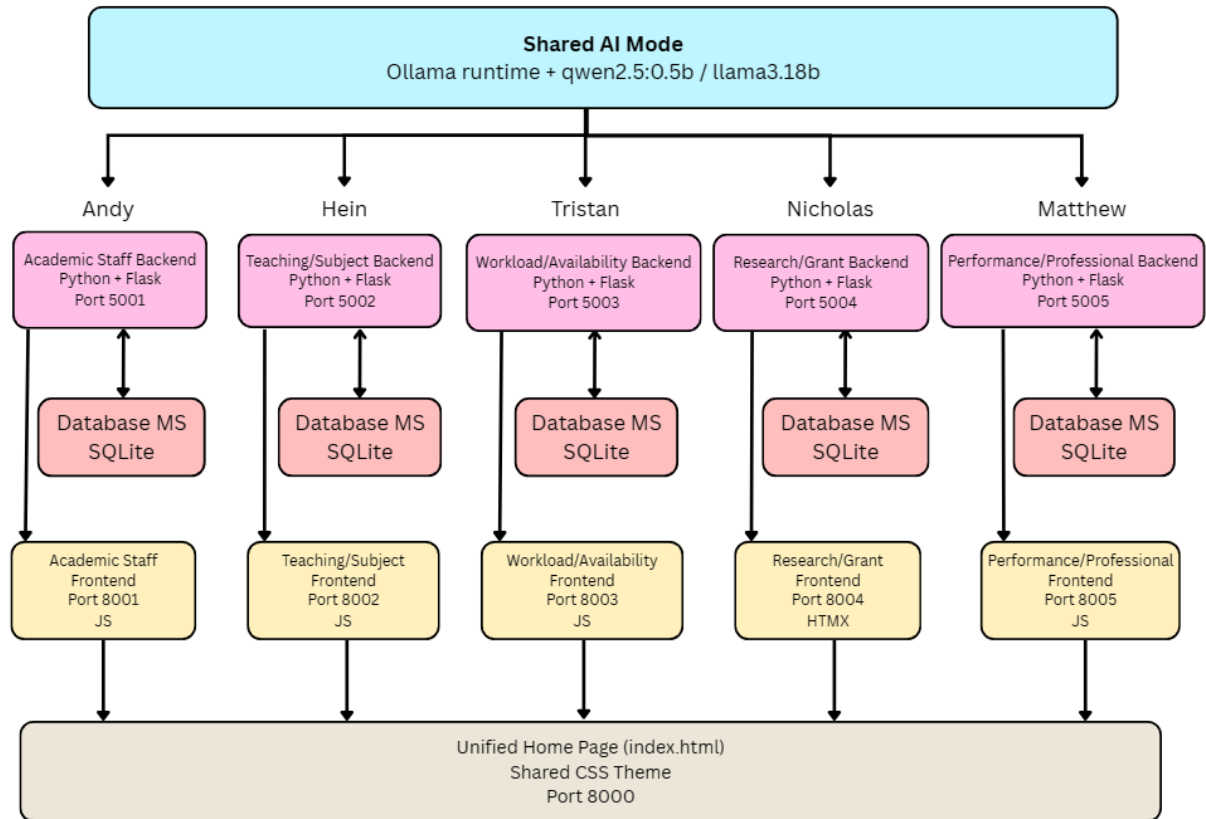
Research and Grant Management (Nicholas Hatzidimitriou)



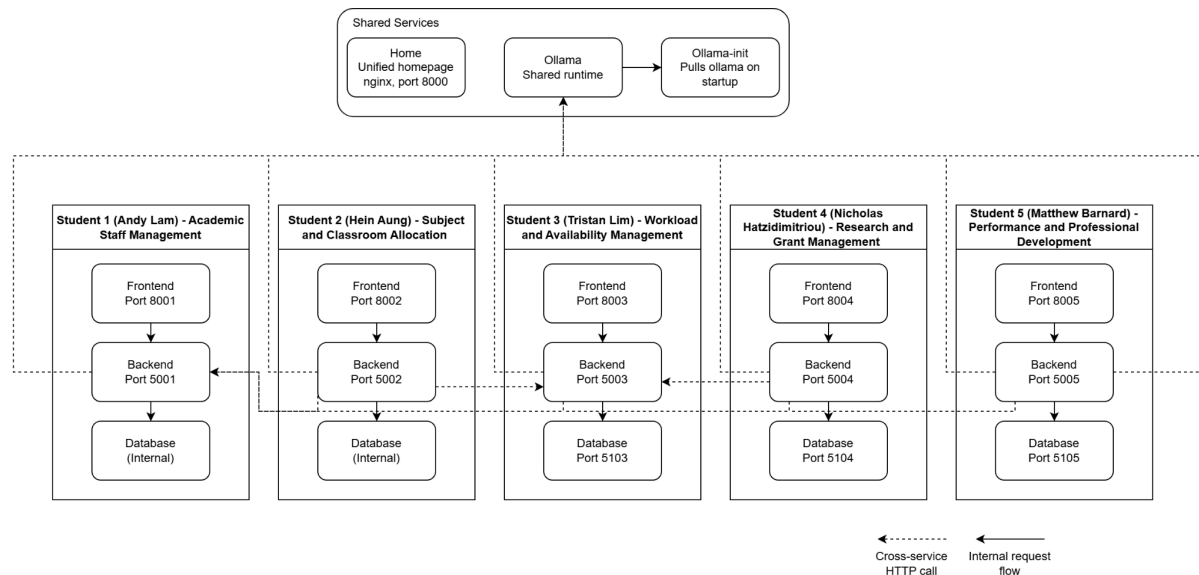
Performance and Professional Development (Matthew Barnard)



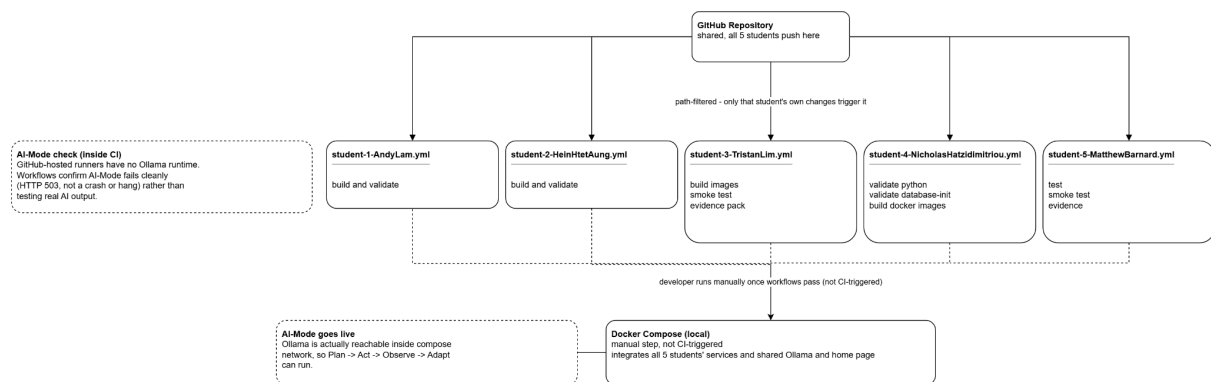
Integrated Software Architecture



Docker Compose Architecture

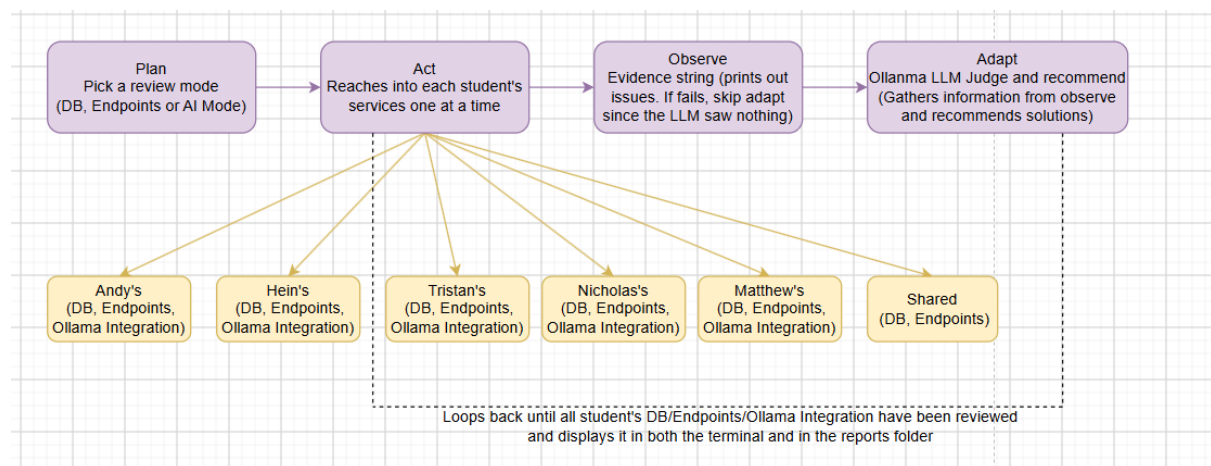


DevOps Pipeline Architecture



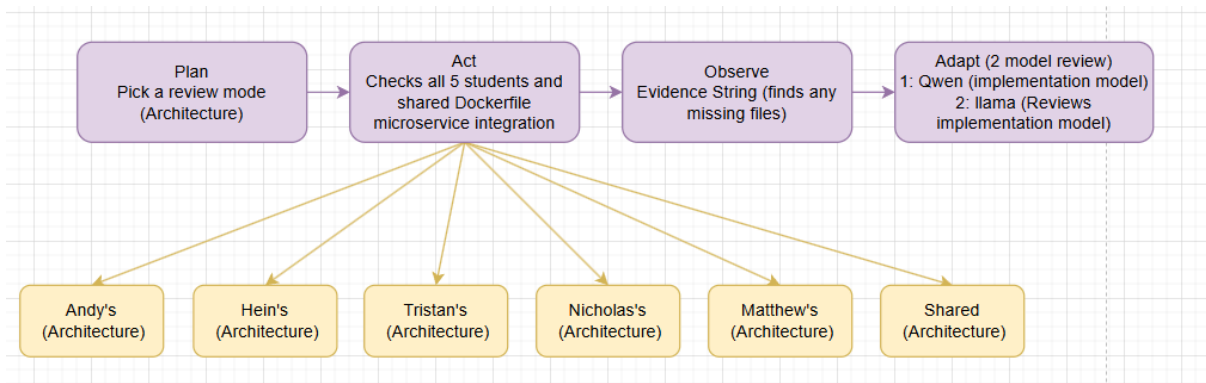
Agentic AI Workflow Diagram

For the Agentic AI Workflow Diagram, it is split into two separate ones. The first image below shows the Plan - Act - Observe - Adapt function for reviewing Database, Endpoints and Ollama Integration. It follows this simple plan: Pick a plan (DB or endpoints) → Act begins (Reaches into student 1 (Andy's DB, Endpoints or Ollama Integration)) → Observes any issues → Creates recommendations on what to adapt. Once one student is done, it repeats it for the other four. This ensures that actions can be taken for each student. It also creates a Markdown file in the reports folder outlining what observation occurred and the recommendations applied (does this inside the terminal as well)



Compared to the first image, the Architecture workflow diagram runs an entire architectural analysis on the codebase and reviews it as a whole without having to complete one student architecture before going on to the next. The adaptation model does this:

- **Qwen Implementation:** Proposes initial architectural analysis
- **Llama Implementation:** Independently checks the analysis against the same evidence to flag any unsupported claims or catch missed claims.



GitHub Actions Workflows

Student 1 (Andy Lam)

```

name: Student 1 CI

on:
  push:
    branches: [ main, student-1-* ]
    paths: [ 'student-1-Andy-Lam/**' ]
  pull_request:
    paths: [ 'student-1-Andy-Lam/**' ]

jobs:
  build-and-validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install backend dependencies
        run: pip install -r student-1-Andy-Lam/backend/requirements.txt

      - name: Check backend imports correctly
        run: python -c "import sys; sys.path.insert(0, 'student-1-Andy-Lam/backend'); import app"

      - name: Build database Docker image
        run: docker build -t staff-database student-1-Andy-Lam/database

      - name: Build backend Docker image
        run: docker build -t staff-backend student-1-Andy-Lam/backend

      - name: Build frontend Docker image
        run: docker build -t staff-frontend student-1-Andy-Lam/frontend
  
```

- The workflow independently builds and validates the Staff Management microservices (database, backend, frontend) when changes within this feature occur. This workflow is independent of the other microservices.
- The workflow runs on pushes to main or any branch prefixed student-1-* including for pull requests.
- With paths, this workflow only triggers when inside the student-1-Andy-Lam/ keeping it independent.
- actions/checkout@v4 checks out the repository so workflow can access source code.
- actions/setup-python@v5 installs python 3.11 on the runner.

- Backend dependencies are installed through `pip install -r student-1-Andy-Lam/backend/requirements.txt`.
- The three images (database, backend and frontend) are built via `docker build -t staff-(microservice) student-1-Andy-Lam/(microservice)`.

Summary

- Backend python dependencies install cleanly
- `app.py` has no import-time errors
- The three dockerfiles (database, backend and frontend) build successfully in isolation.

Student 2 (Hein Htet Aung)

```
You, 30 minutes ago | 1 author (You)
1  name: Student 2 - Teaching, Subject and Classroom Allocation
2
3  on:
4    workflow_dispatch:
5
6    push:
7      branches:
8        - main
9        - 'student-2-*'
10     paths:
11       - 'student-2Hein-Htet-Aung/**'
12       - 'shared/frontend/css/styles.css'
13       - 'docker-compose.yml'
14       - '.github/workflows/student-2-HeinHtetAung.yml'
15
16     pull_request:
17       paths:
18         - 'student-2Hein-Htet-Aung/**'
19         - 'shared/frontend/css/styles.css'
20         - 'docker-compose.yml'
21         - '.github/workflows/student-2-HeinHtetAung.yml'
22
23     env:
24       STUDENT_DIR: student-2Hein-Htet-Aung
25       DATABASE_PORT: '6002'
26       BACKEND_PORT: '5002'
27       FRONTEND_PORT: '8002'
28
29     jobs:
30       test:
31         name: Unit and code validation
32         runs-on: ubuntu-latest
33
34         steps:
35 > - name: Check out repository--
36
37 > - name: Set up Python 3.11--
38
46 > - name: Install Student 2 dependencies--
47
52 > - name: Validate Python syntax--
53
61 > - name: Validate backend import--
62
76 > - name: Verify AI prompt files--
77
83 > - name: Run Student 2 automated tests--
84
90
91 build:
92   name: Build Docker images
93   needs: test
94   runs-on: ubuntu-latest
95
96   steps:
97 > - name: Check out repository--
98
99
100 > - name: Build database Docker image--
101
106 > - name: Build backend Docker image--
107
113 > - name: Build frontend Docker image--
114
120 > - name: Confirm Docker images exist--
121
128
129 smoke-test:
130   name: Runtime and API smoke tests
131   needs: build
132   runs-on: ubuntu-latest
133
134   steps:
135 > - name: Check out repository--
136
137 > - name: Build Student 2 Docker images--
138
154 > - name: Create isolated CI network--
155
158 > - name: Start database service--
159
166 > - name: Wait for database service--
167
185 > - name: Start backend service--
186
198
199 > - name: Wait for backend service--
200
217 > - name: Start frontend service--
218
225 > - name: Wait for frontend service--
226
244 > - name: Verify seeded database records--
245
275 > - name: Verify backend resource endpoints--
276
306 > - name: Verify Subject CRUD lifecycle--
307
402 > - name: Verify invalid Subject validation--
403
428 > - name: Verify classroom capacity validation--
429
471 > - name: Verify back-to-back scheduling rule--
472
509 > - name: Verify frontend files are served--
510
536 > - name: Collect container logs--
537
553 > - name: Create CI evidence summary--
554
603 > - name: Upload CI evidence--
604
610 > - name: Clean up CI containers--
611
You, 37 minutes ago
```

Note: Since the code for the workflow is too long to be screenshotted. Only the test names are displayed in the screenshots rather than the code for the whole test. The whole file could be viewed in the github repo.

- The process is run on `.github/workflows/student-2-HeinHtetAung.yml` and checks the feature Teaching, Subject and Classroom Allocation one by one.
- It is triggered on appropriate pushes and pull requests, and also enables manual run via ``workflow_dispatch``.
- This procedure accepts Python 3.11, installs backend and database requirements, verifies Python syntax and backend imports, reviews Student 2 prompt materials, and runs automated tests.
- It produces the Docker images for the Student 2 database, backend/API and frontend.
- The three containers of Student 2 are executed in a separate runtime smoke-test operation in an isolated docker-network, and verifies that the database, backend and frontend services are reachable.
- Seeded DB data for smoke test, REST API availability, full Subject CRUD lifecycle, managing invalid input in a controlled way, monitoring classroom capacity, and proper back-to-back scheduling behavior.
- The running nginx service checks the frontend files to ensure that the interface and javascript assets are served successfully.
- Collect container logs and CI evidence summary and post as item to GitHub Actions
- The last workflow merged into the main branch and it ran correctly.

Student 3 (Tristan Lim)

```
github > workflows > ! student-3-TristanLim.yml
1  name: student-3-TristanLim
2
3  on:
4    workflow_dispatch:
5    push:
6      branches:
7        - main
8        - student-3-tristan-workload-and-availability-management
9      paths:
10       - "student-3Tristan-Lim/**"
11       - ".github/workflows/student-3-TristanLim.yml"
12    pull_request:
13      paths:
14       - "student-3Tristan-Lim/**"
15
16  env:
17    STUDENT_DIR: student-3Tristan-Lim
18    DATABASE_PORT: "5103"
19    BACKEND_PORT: "5003"
20    FRONTEND_PORT: "8003"
21
22  jobs:
23
24    # ----- build ----
25
26    build-images:
27      name: Build microservice images
28      runs-on: ubuntu-latest
29
30      steps:
31        - uses: actions/checkout@v4
32
33        - name: Validate Python syntax
34          run: |
35            python3 -m compileall -q "$STUDENT_DIR/backend-service" "$STUDENT_DIR/database-service"
36            echo "PASS all Python modules compile"
37
38        - name: Build database-service
39          run: docker build -t s3-db:ci "$STUDENT_DIR/database-service"
40
41        - name: Build backend-service
42          # Build context is the student directory so the sibling prompts/ tree
43          # is included in the image.
44          run: docker build -t s3-be:ci -f "$STUDENT_DIR/backend-service/Dockerfile" "$STUDENT_DIR"
45
46        - name: Build frontend-service
47          run: docker build -t s3-fe:ci "$STUDENT_DIR/frontend-service"
48
49        - name: List built images
50          run: docker images --filter "reference=s3-*:ci"
51
52    # ----- smoke test ----
53
54    smoke-test:
55      name: Run and validate services
56      runs-on: ubuntu-latest
57      needs: [build-images]
58
59      steps:
60        - uses: actions/checkout@v4
61
62        - name: Build images
63          run: |
64            docker build -t s3-db:ci "$STUDENT_DIR/database-service"
65            docker build -t s3-be:ci -f "$STUDENT_DIR/backend-service/Dockerfile" "$STUDENT_DIR"
66            docker build -t s3-fe:ci "$STUDENT_DIR/frontend-service"
67
68        - name: Start services
69          run: |
70            docker network create s3-net
71            docker run -d --name s3-db --network s3-net -p ${DATABASE_PORT}:5103 s3-db:ci
72            docker run -d --name s3-be --network s3-net -p ${BACKEND_PORT}:5003 \
73              -e DATABASE_SERVICE_URL=http://s3-db:5103 \
74              -e OLLAMA_BASE_URL=http://127.0.0.1:11434/v1 \
75              -e LLM_TIMEOUT=5 \
```

```

76     s3-be:ci
77     docker run -d --name s3-fe --network s3-net -p ${FRONTEND_PORT}:80 \
78     -e BACKEND_URL=http://localhost:${BACKEND_PORT} s3-fe:ci
79
80 - name: Wait for services
81 run: |
82     wait_for() {
83         name="$1"; url="$2"
84         for i in $(seq 1 20); do
85             code=$(curl -s -o /dev/null -w "%{http_code}" --max-time 5 "$url" || true)
86             if [ "$code" = "200" ]; then echo "PASS $name ready ($url)"; return 0; fi
87             echo "    waiting for $name ($i/20, last: ${code:-none})"
88             sleep 2
89         done
90         echo "FAIL $name never became ready"
91         docker logs "$3" || true
92         return 1
93     }
94
95     wait_for "database-service" "http://localhost:${DATABASE_PORT}/health" s3-db
96     wait_for "backend-service" "http://localhost:${BACKEND_PORT}/health" s3-be
97     wait_for "frontend-service" "http://localhost:${FRONTEND_PORT}/index.html" s3-fe
98
99 - name: Verify seed data meets the ten record minimum
100 run: |
101     python3 - <<'PY'
102     import json, os, sys, urllib.request
103
104     PORT = os.environ["DATABASE_PORT"]
105     TABLES = [
106         "staff_workload_profile", "workload_entry", "availability_slot",
107         "leave_record", "workload_rule", "workload_alert",
108         "rebalance_recommendation",
109     ]
110
111     failed = False
112     for table in TABLES:
113         with urllib.request.urlopen(f"http://localhost:{PORT}/{table}") as response:
114             count = len(json.load(response))
115             ok = count >= 10
116             failed |= not ok
117             print(f"'PASS' if ok else 'FAIL' {table}: {count} records")
118
119     sys.exit(1 if failed else 0)
120     PY
121
122 - name: Validate workload analysis endpoints
123 run: |
124     check() {
125         name="$1"; url="$2"; expect="$3"
126         code=$(curl -s -o /tmp/body -w "%{http_code}" --max-time 20 "$url" || true)
127         if [ "$code" != "$expect" ]; then
128             echo "FAIL $name -> $code (expected $expect)"; head -c 400 /tmp/body; echo; return 1
129         fi
130         echo "PASS $name -> $code"
131     }
132
133     check "analysis (all staff)" "http://localhost:${BACKEND_PORT}/analysis" 200
134     check "analysis (staff 1)" "http://localhost:${BACKEND_PORT}/analysis?staff_id=1" 200
135     check "analysis (unknown)" "http://localhost:${BACKEND_PORT}/analysis?staff_id=999" 404
136     check "clashes" "http://localhost:${BACKEND_PORT}/clashes" 200
137     check "calendar" "http://localhost:${BACKEND_PORT}/calendar?staff_id=1" 200
138     check "calendar (no staff)" "http://localhost:${BACKEND_PORT}/calendar" 400
139     check "alerts" "http://localhost:${BACKEND_PORT}/alerts" 200
140     check "profiles" "http://localhost:${BACKEND_PORT}/profiles" 200
141
142 - name: Validate CRUD operations
143 run: |
144     BASE="http://localhost:${BACKEND_PORT}"

```

```

146 echo "CREATE"
147 code=$(curl -s -o /dev/null -w "%{http_code}" -X POST "$BASE/availability?staff_id=99" \
148 | -d "staff_id=99&day_of_week=Mon&start_time=09:00&end_time=12:00&availability=available&is_recurring=1")
149 [ "$code" = "201" ] || { echo "FAIL create -> $code"; exit 1; }
150 echo "PASS create -> 201"
151
152 ROW_ID=$(curl -s "http://localhost:${DATABASE_PORT}/availability_slot?staff_id=99" \
153 | python3 -c "import json,sys; print(max(r['slot_id'] for r in json.load(sys.stdin)))")
154 echo "created slot_id=$ROW_ID"
155
156 echo "READ"
157 code=$(curl -s -o /dev/null -w "%{http_code}" "$BASE/availability/$ROW_ID/record")
158 [ "$code" = "200" ] || { echo "FAIL read -> $code"; exit 1; }
159 python3 -c "
160 import json, urllib.request
161 row = json.load(urllib.request.urlopen('$BASE/availability/$ROW_ID/record'))
162 assert row['day_of_week'] == 'Mon', row
163 assert row['availability'] == 'available', row
164 "
165 echo "PASS read -> 200"
166
167 echo "UPDATE"
168 code=$(curl -s -o /dev/null -w "%{http_code}" -X PUT "$BASE/availability/$ROW_ID?staff_id=99" \
169 | -d "staff_id=99&day_of_week=Fri&start_time=13:00&end_time=17:00&availability=preferred&is_recurring=1")
170 [ "$code" = "200" ] || { echo "FAIL update -> $code"; exit 1; }
171 python3 -c "
172 import json, urllib.request
173 row = json.load(urllib.request.urlopen('$BASE/availability/$ROW_ID/record'))
174 assert row['day_of_week'] == 'Fri', row
175 assert row['availability'] == 'preferred', row
176 "
177 echo "PASS update -> 200"
178
179 echo "DELETE"
180 code=$(curl -s -o /dev/null -w "%{http_code}" -X DELETE "$BASE/availability/$ROW_ID?staff_id=99")
181 [ "$code" = "200" ] || { echo "FAIL delete -> $code"; exit 1; }
182 code=$(curl -s -o /dev/null -w "%{http_code}" "$BASE/availability/$ROW_ID/record")
183 [ "$code" = "404" ] || { echo "FAIL record still present -> $code"; exit 1; }
184 echo "PASS delete -> 200, record gone"
185
186 - name: Validate alert recomputation is idempotent
187 run: |
188   BASE="http://localhost:${BACKEND_PORT}"
189   count() { curl -s "http://localhost:${DATABASE_PORT}/workload_alert" \
190   | python3 -c "import json,sys; print(len(json.load(sys.stdin)))"; }
191
192   curl -s -X POST "$BASE/alerts/recompute" -o /dev/null --max-time 60
193   first=$(count)
194   curl -s -X POST "$BASE/alerts/recompute" -o /dev/null --max-time 60
195   second=$(count)
196
197   echo "alerts after first recompute: $first"
198   echo "alerts after second recompute: $second"
199   [ "$first" = "$second" ] || { echo "FAIL recompute duplicated alerts"; exit 1; }
200   echo "PASS recompute is idempotent"
201
202 - name: Validate AI-Mode degrades cleanly without an LLM runtime
203 run: |
204   # No Ollama on the runner. AI-Mode must report the failure rather than
205   # returning a 500 or hanging.
206   code=$(curl -s -o /tmp/ai -w "%{http_code}" --max-time 90 \
207   | "http://localhost:${BACKEND_PORT}/ai/health" || true)
208   if [ "$code" != "503" ]; then
209     echo "FAIL expected 503 when the LLM is unreachable, got $code"
210     head -c 400 /tmp/ai; echo
211     exit 1
212   fi
213   grep -q "LLM unreachable" /tmp/ai || { echo "FAIL unexpected body"; cat /tmp/ai; exit 1; }
214   echo "PASS AI-Mode returned 503 with a clear message"

```

```

216 - name: Verify prompts are packaged in the backend image
217   run: |
218     docker exec s3-be sh -c "ls /app/prompts/service/implementation/system_prompt.txt \
219     /app/prompts/service/review/rebalance_review_prompt.txt"
220     echo "PASS prompt files present in the image"
221
222 - name: Verify frontend assets
223   run: |
224     for path in /index.html /tabs/normal.html /tabs/ai-mode.html /js/api.js /js/config.js; do
225       code=$(curl -s -o /dev/null -w "%{http_code}" "http://localhost:${FRONTEND_PORT}$path")
226       [ "$code" = "200" ] || { echo "FAIL $path -> $code"; exit 1; }
227       echo "PASS $path -> 200"
228     done
229     curl -s "http://localhost:${FRONTEND_PORT}/js/config.js" | grep -q "BACKEND_URL" \
230     || { echo "FAIL backend URL not injected"; exit 1; }
231     echo "PASS backend URL injected by entrypoint"
232
233 - name: Service logs
234   if: always()
235   run: |
236     echo "===== database-service ====="; docker logs s3-db 2>&1 | tail -30
237     echo "===== backend-service ====="; docker logs s3-be 2>&1 | tail -30
238     echo "===== frontend-service ====="; docker logs s3-fe 2>&1 | tail -30
239
240 - name: Stop services
241   if: always()
242   run: |
243     docker rm -f s3-db s3-be s3-fe || true
244     docker network rm s3-net || true
245

```

```

248 evidence-pack:
249   name: Generate CI evidence
250   runs-on: ubuntu-latest
251   needs: [smoke-test]
252
253   steps:
254     - uses: actions/checkout@v4
255
256     - name: Generate reports
257       run: |
258         mkdir -p reports
259
260         cat > reports/report.json <<EOF
261         {
262           "workflow": "student-3-TristanLim",
263           "student": "Tristan Lim",
264           "feature": "Workload and Availability Management",
265           "services": ["frontend-service", "backend-service", "database-service"],
266           "run_id": "${{ github.run_id }}",
267           "run_number": "${{ github.run_number }}",
268           "commit_sha": "${{ github.sha }}",
269           "branch": "${{ github.ref_name }}",
270           "generated": "$(date -u +%Y-%m-%dT%H:%M:%S2)"
271         }
272         EOF

```

```

274 cat > reports/report.md <<EOF
275 # Student 3 CI Report - Workload and Availability Management
276
277 | Field | Value |
278 | --- | --- |
279 | Workflow | student-3-TristanLim |
280 | Run ID | ${ github.run_id } |
281 | Commit | ${ github.sha } |
282 | Branch | ${ github.ref_name } |
283 | Generated | $(date -u +%Y-%m-%dT%H:%M:%SZ) |
284
285 ## Validated in this run
286
287 - All Python modules compile
288 - frontend-service, backend-service and database-service images build
289 - All three services start and report healthy
290 - Every database table holds at least ten records
291 - Workload analysis, clash detection and calendar endpoints respond
292 - Full CRUD cycle: create, read, update, delete
293 - Alert recomputation is idempotent
294 - AI-Mode degrades cleanly when no LLM runtime is reachable
295 - Prompt files are packaged inside the backend image
296 - Frontend assets served and backend URL injected at container start
297
298 ## Not covered here
299
300 The Ollama runtime is not available on a GitHub Actions runner, so the
301 Frontend -> Backend -> Ollama -> LLM path and the
302 Plan -> Act -> Observe -> Adapt cycle are demonstrated locally instead.
303 EOF
304
305 cat > reports/run-view.md <<EOF
306 Run URL:
307 https://github.com/\${ github.repository }}/actions/runs/\${ github.run\_id }
308 EOF
309
310 cat reports/report.md
311
312 - uses: actions/upload-artifact@v4
313   with:
314     name: student-3-ci-report
315     path: reports/

```

- The workflow independently builds and validates the Workload and Availability Management microservices belonging to student-3 so it does not run for other student's changes.
- Builds all three images with the backend build from the student directory so its Dockerfile can copy the sibling *prompts/* tree into the image.
- Starts all three containers and runs live checks
- Confirms AI-mode fails cleanly when no Ollama runtime is reachable
- Generates a CI evidence report and uploads it as a build artefact

Student 4 (Nicholas Hatzidimitriou)

```
.github > workflows > ! student-4-NicholasHatzidimitriou.yml
1  name: Student 4 - Nicholas Hatzidimitriou (Research & Grant Management)
2
3  on:
4    push:
5      branches: [main, "student-4-*"]
6      paths:
7        - "student-4Nicholas-Hatzidimitriou/**"
8        - ".github/workflows/student-4-NicholasHatzidimitriou.yml"
9    pull_request:
10     branches: [main]
11     paths:
12       - "student-4Nicholas-Hatzidimitriou/**"
13
14  jobs:
15    validate-python:
16      name: Validate Python syntax
17      runs-on: ubuntu-latest
18      steps:
19        - name: Checkout repository
20          uses: actions/checkout@v4
21
22        - name: Set up Python
23          uses: actions/setup-python@v5
24          with:
25            python-version: "3.11"
26
27        - name: Install backend dependencies
28          working-directory: student-4Nicholas-Hatzidimitriou/backend
29          run: pip install --no-cache-dir -r requirements.txt
30
31        - name: Compile-check backend
32          working-directory: student-4Nicholas-Hatzidimitriou/backend
33          run: python -m compileall -q .
34
35        - name: Install database-service dependencies
36          working-directory: student-4Nicholas-Hatzidimitriou/database-service
37          run: pip install --no-cache-dir -r requirements.txt
38
39        - name: Compile-check database-service
40          working-directory: student-4Nicholas-Hatzidimitriou/database-service
41          run: python -m compileall -q .
42
43        - name: Install frontend dependencies
44          working-directory: student-4Nicholas-Hatzidimitriou/frontend
45          run: pip install --no-cache-dir -r requirements.txt
46
47        - name: Compile-check frontend
48          working-directory: student-4Nicholas-Hatzidimitriou/frontend
```

```
.github > workflows > ! student-4-NicholasHatzidimitriou.yml
```

```
49 |   run: python -m compileall -q .
50 |
51 | validate-database-init:
52 |   name: Validate database initialises and seeds correctly
53 |   runs-on: ubuntu-latest
54 |   needs: validate-python
55 |   steps:
56 |     - name: Checkout repository
57 |       uses: actions/checkout@v4
58 |
59 |     - name: Set up Python
60 |       uses: actions/setup-python@v5
61 |       with:
62 |         python-version: "3.11"
63 |
64 |     - name: Install database-service dependencies
65 |       working-directory: student-4Nicholas-Hatzidimitriou/database-service
66 |       run: pip install --no-cache-dir -r requirements.txt
67 |
68 |     - name: Run init_db.py and seed.py
69 |       working-directory: student-4Nicholas-Hatzidimitriou/database-service
70 |       run: |
71 |         mkdir -p /tmp/ci-data
72 |         DATA_DIR=/tmp/ci-data python init_db.py
73 |         DATA_DIR=/tmp/ci-data python seed.py
74 |
75 |     - name: Verify seeded record counts
76 |       working-directory: student-4Nicholas-Hatzidimitriou/database-service
77 |       run: |
78 |         python -c "
79 |         import sqlite3
80 |         conn = sqlite3.connect('/tmp/ci-data/student4.db')
81 |         tables = ['ResearchProjects', 'Grants', 'Publications', 'ProjectStaff', 'GrantAle
82 |         for t in tables:
83 |             count = conn.execute(f'SELECT COUNT(*) FROM {t}').fetchone()[0]
84 |             assert count >= 10, f'{t} has only {count} records, expected at least 10'
85 |             print(f'{t}: {count} records OK')
86 |         "
87 |
88 | build-docker-images:
89 |   name: Build Docker images
90 |   runs-on: ubuntu-latest
91 |   needs: validate-python
92 |   steps:
93 |     - name: Checkout repository
94 |       uses: actions/checkout@v4
```

```
95 |
96 |     - name: Build backend image
97 |       run: docker build -t student4-backend:ci ./student-4Nicholas-Hatzidimitriou/backend
98 |
99 |     - name: Build frontend image
100 |       run: docker build -t student4-frontend:ci ./student-4Nicholas-Hatzidimitriou/fronte
101 |
102 |     - name: Build database-service image
103 |       run: docker build -t student4-database:ci ./student-4Nicholas-Hatzidimitriou/databa
```

- Workflow independently builds and validates the Research and Grant Management microservices (frontend, backend, DB-service) when changes occur.
- The workflow is split into three jobs: validate-python, validate-database-init and build-docker-images.
- Workflow runs on pushes to main / any branch associated with student-4
- Validate-python installs dependencies and runs on python -m against all 3 services to catch syntax errors
- Validate-database-init does more, running init_db.py and seed.py to see if every table contains at least 10 records
- Build-docker-images: builds all 3 docker images (for frontend, backend and database-service) to confirm that each is valid and provides a working image.

Student 5 (Matthew Barnard)

```
name: Matthew - Performance and Professional Development

on:
  push:
    paths:
      - "student-5Matthew-Barnard/**"
      - ".github/workflows/student-5-MatthewBarnard.yml"
  pull_request:
    paths:
      - "student-5Matthew-Barnard/**"
      - ".github/workflows/student-5-MatthewBarnard.yml"
  workflow_dispatch:

jobs:
  test:
    name: Unit and API tests
    runs-on: ubuntu-latest
    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
          cache-dependency-path: |
            student-5Matthew-Barnard/backend-service/requirements.txt
            student-5Matthew-Barnard/database-service/requirements.txt

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install -r student-5Matthew-Barnard/backend-service/requirements.txt
          python -m pip install -r student-5Matthew-Barnard/database-service/requirements.txt

      - name: Test database service
        working-directory: student-5Matthew-Barnard/database-service
        run: python -m unittest discover -s tests -v

      - name: Test backend and AI endpoint
        working-directory: student-5Matthew-Barnard/backend-service
        run: python -m unittest discover -s tests -v
```

```
smoke-test:
  name: Build and smoke-test containers
  needs: test
  runs-on: ubuntu-latest
  steps:
    - name: Check out repository
      uses: actions/checkout@v4

    - name: Validate Compose configuration
      run: docker compose -f student-5Matthew-Barnard/docker-compose.yml config

    - name: Build all three feature services
      run: |
        docker build -t student5-database:ci student-5Matthew-Barnard/database-service
        docker build -t student5-backend:ci student-5Matthew-Barnard/backend-service
        docker build -t student5-frontend:ci student-5Matthew-Barnard/frontend-service

    - name: Start isolated test stack
      run: |
        docker network create student5-ci
        docker run -d --name student5-database --network student5-ci -p 5105:5105 \
          -e PORT=5105 -e DATABASE_PATH=/app/data/performance_dev.db student5-database:ci
        docker run -d --name student5-backend --network student5-ci -p 5005:5005 \
          -e PORT=5005 -e DATABASE_SERVICE_URL=http://student5-database:5105 \
          -e OLLAMA_BASE_URL=http://127.0.0.1:11434/v1 -e LLM_TIMEOUT=5 student5-backend:ci
        docker run -d --name student5-frontend --network student5-ci -p 8005:80 \
          -v "$PWD/shared/frontend/css:/usr/share/nginx/html/css:ro" student5-frontend:ci
```

```

- name: Wait for services
  run: |
    wait_for() {
      name="$1"; url="$2"; container="$3"
      for attempt in $(seq 1 20); do
        code=$(curl -s -o /dev/null -w "%{http_code}" --max-time 5 "$url" || true)
        if [ "$code" = "200" ]; then echo "PASS $name ready"; return 0; fi
        sleep 2
      done
      echo "FAIL $name did not become ready"
      docker logs "$container" || true
      return 1
    }
    wait_for database http://localhost:5105/health student5-database
    wait_for backend http://localhost:5005/health student5-backend
    wait_for frontend http://localhost:8005/ student5-frontend

- name: Verify seed counts and all resource endpoints
  run: |
    python - <<'PY'
    import json
    import urllib.request

    expected_tables = {
      "PerformanceReviews", "DevelopmentGoals", "TrainingPrograms",
      "StaffTraining", "DevelopmentRecommendations",
    }
    with urllib.request.urlopen("http://localhost:5105/health") as response:
      counts = json.load(response)["tableCounts"]
    assert set(counts) == expected_tables, counts
    assert all(count >= 10 for count in counts.values()), counts

    resources = [
      "performance-reviews", "development-goals", "training-programs",
      "staff-training", "development-recommendations",
    ]
    for resource in resources:
      with urllib.request.urlopen(f"http://localhost:8005/api/{resource}") as response:
        rows = json.load(response)
        assert len(rows) >= 10, (resource, len(rows))
        print(f"PASS {resource}: {len(rows)} records")

```

steps:

```
- name: Verify CRUD through the frontend proxy
run: |
    created=$(curl -fsS -X POST http://localhost:8005/api/development-goals \
    -H 'Content-Type: application/json' \
    -d '{"staffID":1,"title":"CI verification goal","status":"Planned","progress":0}')
    goal_id=$(python -c 'import json,sys; print(json.load(sys.stdin)["goalID"])' <<< "$created")

    updated=$(curl -fsS -X PUT "http://localhost:8005/api/development-goals/$goal_id" \
    -H 'Content-Type: application/json' -d '{"progress":25,"status":"In Progress"}')
    python -c 'import json,sys; row=json.load(sys.stdin); assert row["progress"] == 25 and row["status"] == "In Progress"'

    code=$(curl -s -o /dev/null -w "%{http_code}" -X DELETE \
    "http://localhost:8005/api/development-goals/$goal_id")
    [ "$code" = "204" ] || { echo "FAIL delete returned $code"; exit 1; }
    code=$(curl -s -o /dev/null -w "%{http_code}" \
    "http://localhost:8005/api/development-goals/$goal_id")
    [ "$code" = "404" ] || { echo "FAIL deleted record returned $code"; exit 1; }
    echo "PASS create, read/update, and delete lifecycle"

- name: Verify frontend and graceful AI failure
run: |
    for asset in /app.js /styles.css /css/styles.css; do
        code=$(curl -s -o /dev/null -w "%{http_code}" "http://localhost:8005$asset")
        [ "$code" = "200" ] || { echo "FAIL $asset returned $code"; exit 1; }
    done
    code=$(curl -s -o /tmp/ai-response -w "%{http_code}" --max-time 15 \
    http://localhost:8005/api/ai/health || true)
    [ "$code" = "503" ] || { echo "FAIL expected AI health 503 without Ollama, got $code"; cat /tmp/ai-response; exit 1; }
    grep -q "Ollama is unavailable" /tmp/ai-response
    echo "PASS frontend assets and explicit AI unavailable response"

- name: Show service logs
if: always()
run: |
    docker logs student5-database 2>&1 | tail -30 || true
    docker logs student5-backend 2>&1 | tail -30 || true
    docker logs student5-frontend 2>&1 | tail -30 || true
```

```
- name: Stop test stack
if: always()
run: |
    docker rm -f student5-frontend student5-backend student5-database || true
    docker network rm student5-ci || true
```

evidence:

name: Package CI evidence

needs: smoke-test

runs-on: ubuntu-latest

steps:

- name: Create evidence summary

```
run: |
    mkdir -p evidence
    cat > evidence/student-5-ci-summary.md <<EOF
    # Student 5 Release 0 CI evidence

    - Run: https://github.com/${{ github.repository }}/actions/runs/${{ github.run_id }}
    - Commit: ${{ github.sha }}
    - Branch: ${{ github.ref_name }}
    - Generated: $(date -u +%Y-%m-%dT%H:%M:%SZ)
```

This run validated Python tests, three Docker image builds, service health, ten-or-more seed records per table, all five resource endpoints, a complete CRUD lifecycle through the frontend proxy, frontend assets, and graceful behaviour when Ollama is unavailable on the CI runner.

The shared software-development agentic loop is intentionally outside this base-feature evidence and must be documented from genuine loop executions. EOF

- name: Upload evidence artifact

uses: actions/upload-artifact@v4

with:

name: student-5-release-0-evidence

path: evidence/

- Runs automatically when Student 5 files change and can also be manually triggered.
- Uses Python 3.11 and installs the backend and database dependencies.
- Runs database and backend/AI endpoint tests.
- Validates the Student 5 Docker Compose configuration.
- Builds the frontend, backend and database Docker images.
- Starts an isolated three-container stack and checks service health, seeded records and CRUD operations.
- Confirms the AI service fails cleanly when Ollama is unavailable.
- Generates and uploads a Student 5 CI evidence report as a build artifact.

Implementation Summary

Academic Staff Management (Andy Lam)

The academic staff management feature consists of CRUD functions relating to academic staff records and an AI summary analysis of a staff member's suitability for general teaching. The implementation follows the project's microservices architecture of a frontend, backend, database microservice, which is integrated with the shared Ollama AI-mode runtime.

Academic Staff Management - Database Microservice

- SQLite database called staff.db containing 6 tables: departments, staff, staff_qualifications, staff_expertise, staff_ai_analysis, staff_availability.
- Foreign key constraints are enforced and deleted records are cascaded on dependent tables. An example is removing a staff record, removing their qualifications, expertise, availability and AI analysis records.
- All tables begin seeded with 10 records each.

Academic Staff Management - Backend Microservice

- Built using Flask, exposing a REST API on port 5001.
- API's include:
 - GET staff, staff by id, staff qualifications, staff expertise, staff availability, staff by expertise, staff by position/department and departments
 - POST staff and ai analysis
 - PUT staff by id
 - DELETE staff by id
- Error handling includes missing records (404), missing required fields (400) and database constraint violations (400).

Academic Staff Management - AI Mode

- The model used for the generated AI summary is qwen2.5:0.5b.
- The prompt for the AI is built from a staff_analysis_system_prompt.txt and staff_analysis_task_prompt.txt which provides the staff data, model behaviour and

guardrails. The prompt is then sent to Ollama runtime through `llm.client.py` with the OpenAI compatible chat completions API.

- The AI derives its summary response from a staff's qualification and expertise.
- Suitability score is rule based and computed using average skill level multiplied by 2, which is prone to future change.
- Generated summaries are persisted to the `staff_ai_analysis` table.

Academic Staff Management - Frontend Microservice

- Built with HTML, CSS, JS and served as a static page.
- The backend is called with `fetch()` at `http://127.0.0.1:5001`.
- Displays the staff records with their details including expertise, department, etc.
- Contains a view button to see an individual staff member's details
- Contains search and filter buttons to filter staff.
- Contains an Add/Edit form and delete with confirmation.
- Share CSS theme (`http://localhost:8000/css/styles.css`) and links back to the unified home page.

Academic Staff Management - DevOps

- Each container or layer (frontend, backend, database) contains a Dockerfile which is integrated into the shared `docker-compose.yml`.
- The CI/CD is implemented in `student-1.yml` in `.github/workflows/`, where this microservice is built and validated independently.

Teaching, Subject and Classroom Allocation (Hein Htet Aung)

Teaching, Subject and Classroom Allocation - Database Microservice

- SQLite database named `allocation.db` consisting of 4 tables: `subjects`, `subject_offers`, `classrooms` and `teaching_allocations`
- Runs through a Flask database microservice on port 6002
- At least 10 records are seeded in each table.
- Relationships between local entities such as `subjects` and `subject_offers` are enforced through the use of foreign keys.
- `Assigned_staff_member` is not stored as a local foreign key but as an external staff reference due to the staff records being owned by a different microservice.
- The backend accesses the database through its REST API rather than accessing SQLite directly.

Teaching, Subject and Classroom Allocation - Backend Microservice

- Built with Python and Flask
- Exposed on port 5002
- Supply CRUD REST endpoints for `subjects`, `subject offers`, `classrooms` and `teaching allocations`.
- Validates input before records are persisted.

- Enforces deterministic teaching-allocation business rules including:
 - required fields
 - valid date and time formats
 - start time before end time
 - valid day, class type and allocation status
 - classroom capacity
 - classroom scheduling conflicts
 - staff existence and availability where applicable
- Communicates with student 1 Academic Staff Management REST API for getting staff identity and expertise.
- Gets workload and availability information from the Student 3 Workload and Availability REST API.
- Business rule validation is separated from database persistence.

Teaching, Subject and Classroom Allocation - AI Mode

- Utilizes qwen2.5:0.5b through the communal Ollama runtime.
- It produces evaluated personnel recommendations for teaching assignments requiring staff distribution.
- Constructs AI context based on subject specifications, resource distribution, personnel proficiency, availability, and workload.
- Prompt directives are stored in external files located in Student 2 prompts/allocation/ instead of being entirely embedded within the application logic.
- The model is instructed to generate a maximum of three prioritized suggestions in a formatted JSON.
- AI-generated suggestions are treated as unreliable output and assessed by fixed backend algorithms.
- Submissions featuring duplicate, unidentified, inaccessible, or invalid staff identification numbers are dismissed.
- Transmit prior suggestion identifiers and failure rationales to the ensuing recommendation phase to facilitate the Adapt stage.
- The AI recommendations serve merely as guidance, and the user retains the authority to determine the ultimate distribution of workers.

Teaching, Subject and Classroom Allocation - Frontend Microservice

- Built using HTML, CSS & JavaScript Served by nginx on port 8002.
- Uses the standard CSS theme of the Faculty Management System Links back to the combined main page.
- Provides unique interfaces for
 - Subjects
 - Subject Offers
 - Classrooms
 - Teaching Allocations
 - AI Recommendations
- Uses JavaScript fetch() calls to communicate with the backend REST API for Student 2.
- It provides create, view, update and delete interaction on the managed resources.

- Shows classroom capacity and does supporting client side checks at time of entering allocation
- Displays AI recommendations along with staff details, experience, work context and reasoning for recommendation.
- The last assignment of the recommended personnel is still user controlled activity.

Teaching, Subject and Classroom Allocation - DevOps

- The frontend, backend, and database each possess their own Docker configurations and operate within separate containers.
- All three services are implemented in the common docker-compose.yml.
- Student 2 is validated independently using `.github/workflows/student-2-HeinHtetAung.yml`.
- The CI pipeline autonomously constructs all three Docker images and does live runtime smoke tests for the Student 2 services.
- Database entries prepared Subject Create, Read, Update, Delete RESTful services.
- Runtime CI checks include seeded database records, REST endpoints, Subject CRUD, invalid-input handling, classroom capacity validation, back-to-back scheduling and frontend asset delivery.
- A device is produced that records CI and provides a summary of the validation.

Workload and Availability Management (Tristan Lim)

The workload and availability management feature consists of CRUD functions over seven tables describing the staff workload, availability and leave. There is also a rules engine that computes overload / underload status and detects scheduling clashes and an AI-Mode to propose workload redistribution. The implementation follows the project's microservices architecture of a frontend, backend and database integrated with the shared Ollama runtime.

Workload and Availability Management - Database Microservice

- SQLite database called workload.db containing 7 tables: staff_workload_profile, workload_entry, availability_slot, leave_record, workload_rule, workload_alert, rebalance_recommendation.
- Single generic CRUD route layer (app.py) that serves every table
- All tables seeded with at least 10 records each, with staff_id, staff_name, and department aligned with Student 1's roster

Workload and Availability Management – Backend Microservice

- Built using Flask, exposing a REST API on port 5003
- APIs has GET, POST, PUT, DELETE for every resource (entryies, availability, leave, profiles, alers, recommendations. As well as GET: analysis, clashes, calendar and Post: alerts and recompute.
- GET: staff/directory, staff/<id> and staff/reconcile read staff identity live from Student 1 services
- Error handling includes missing records (404), missing or invalid fields (400), and a dependent service being unreachable (503, with a message)

Workload and Availability Management – AI-Mode

- The model used for rebalancing and Q&A is configurable via the OLLAMA_MODEL environment variable. Both llama and qwen have been tested against this feature.
- The rebalance agent is built from five prompt files (system, context, task, review and a separate Q&A system prompt) loaded from the prompts directory
- A specific staff member's activities is gathered and is sent to the LLM. It validates each proposal against the rules engine before its shown, rejecting any non-existent reference activity or would push a target over their cap. It would re-prompt the model with rejection reasons for a corrected attempt.
- Accepted proposals can be saved as pending recommendations for a human to accept, reject or override.

Workload and Availability Management – Frontend Microservice

- Built with HTML, CSS, and JavaScript served as a static site with two tabs: Dashboard and AI Mode
- Displays the computed workload overview, staff detail and calendar, staff directory, conflicts, alerts, and full CRUD forms, for entries, availability and leave.
- Provides CSV exports and a separate printable report page for PDF view.
- Uses the teams collective CSS.

Workload and Availability Management – DevOps

- Each container has its own Dockerfile and is integrated into the shared docker-compose.yml
- The CI/CD workflow is implemented in student-3TristanLim.yml in the github directory where the microservice is build and validated independently.

Research and Grant Management (Nicholas Hatzidimitriou)

The research and grant management feature consists of CRUD operations tailored towards research projects, grants, publications and staff assignments. Alongside this, it includes three AI mode functions: Research Summary generation, staff-fit rationale and cross-service staff recommendation. This implementation follows the project's microservice structure split into Frontend, backend and a dedicated database microservice. All of this is integrated with a shared Ollama AI-Mode runtime with Staff Management and Workload and Availability management for cross data retrieval.

Research and Grant Management - Database Microservice

- SQLite database named student4.db, containing 6 tables: ResearchProjects, Grants, Publications, ProjectStaff, GrantAlerts and ResearchAIAnalysis.
- This database is containerised in its own Flask Service (database-service), exposing data over REST API rather than direct access via the backend.

- CHECK constraints enforce valid values on status and type fields (e.g. Grants must have one of the following statuses: Applied, Rejected, Awarded, Completed or In Progress)
- Cross-service references (leadStaffID, staffID) are not stored as foreign keys but as integers since they reference records owned by a different database.
- All tables are seeded with 10 records each

Research and Grant Management - Backend Microservice

- Backend built using flask, exposing a REST API on port 5004
- Full CRUD for 6 tables and nested routes for /projects/id/grants, /projects/id/publications, /projects/id/staff, and /grants/id/alerts.
- Backend never queries SQLite directly. Data access goes through HTTP calls to the database microservice, encapsulating the database and isolating ownership to that service.
- Error handling includes missing records (404), missing required fields (400) and invalid foreign references like grant being created against a project that does not exist (400).
- CORS enabled so the frontend that runs on a separate port can call the backend from the browser.

Research and Grant Management - AI Mode

- The AI model uses qwen2.5:0b, called through the shared Ollama runtime
- Each feature has its own system, task and context prompts: (prompts/generate_summary/, prompts/match_staff/, prompts/recommend_staff/), loaded via prompt_loader.py and sent through llm_client.py using the openai API
- Generate Research summary builds its context from a project's details and publications.
- Explain Current Staff Fit reasons over staff already assigned to the project from the ProjectStaff table
- Recommend New Staff: retrieves the full staff roster and individual expertise via the Staff Management microservice and workload and availability status from the Workload and Availability microservice before asking the LLM to recommend the best fit candidates who are not assigned.

Research and Grant Management - Frontend Microservice

- Build with Flask serving HTML templates, using HTML for AI mode interactions.
- Backend is called at https://localhost:5004 for JSON endpoints
- Projects and grants tab display all research projects, add project form and detail view showing a project's linked grants, publications and assigned staff (staff names are fetched live from the Staff Management service)
- AI insights tab: selector with three buttons that correspond to three AI mode features, and their displayed results when run.
- Shared team CSS theme that like back to the unified homepage.

Research and Grant Management - DevOps

- Three Dockerfiles, located in the Frontend, Backend and Database microservices. Each file is integrated into the shared Docker-Compose.yml file as student4-frontend, student4-backend and student4-database.
- The CI/CD workflow is implemented in student-4-NicholasHatzidimitriou.yml in .github/workflows/. It validates the microservices python syntax, confirms that the database initialises and is seeded correctly and builds all docker images.

Performance and Professional Development (Matthew Barnard)

The Performance and Professional Development feature manages performance reviews, development goals, training programs, staff training and AI-assisted professional development recommendations. The implementation follows the project's frontend, backend and database microservice structure and integrates with the shared Ollama runtime.

Performance and Professional Development- Database Microservice

- SQLite database called performance_dev.db containing five tables: PerformanceReviews, DevelopmentGoals, TrainingPrograms, StaffTraining and DevelopmentRecommendations.
- Runs through its own Flask database API on port 5105.
- Uses CHECK, UNIQUE and foreign-key constraints to maintain data integrity.
- All five tables are seeded with 10 records each.

Performance and Professional Development - Backend Microservice

- Built using Flask and exposed on port 5005.
- Provides CRUD functionality for all five resources.
- Applies input and cross-field validation before persistence.
- Retrieves staff profiles and expertise through Student 1's Staff Management REST API.

Performance and Professional Development - AI Mode

- Uses qwen2.5:1.5b through the shared Ollama runtime.
- Builds context from staff details, performance reviews, development goals, training enrolments and available training programs.
- Validates model output before saving a recommendation.
- Retries invalid output once before using a database-grounded fallback.
- Valid recommendations are persisted to DevelopmentRecommendations and can be Accepted, Rejected or Modified by the user.

Performance and Professional Development - Frontend Microservice

- Served through Nginx on port 8005.

Step	Status	Duration
Set up job	Completed	0s
Run actions/checkout@v4	Completed	0s
Set up Python	Completed	0s
Install backend dependencies	Completed	5s
Check backend imports correctly	Completed	1s
Build database Docker image	Completed	6s
Build backend Docker image	Completed	8s
Build frontend Docker image	Completed	2s
Post Set up Python	Completed	0s
Post Run actions/checkout@v4	Completed	0s
Complete job	Completed	0s

Student 2 (Hein Htet Aung)

Student 2 - Teaching, Subject and Classroom Allocation

student-2-HeinHtetAung.yml

Filter workflow runs

...

118 workflow runs

Event Status Branch Actor

This workflow has a workflow_dispatch event trigger.

Run workflow

✓ Student 2 - Teaching, Subject and Classroom Allocation

Student 2 - Teaching, Subject and Classroom Allocation #122: Manually run by Hein-Htet-Aung08

main

1 minute ago
1m 20s

...

✓ Merge pull request #34 from Hein-Htet-Aung08/student-2-hein-teaching-...

Student 2 - Teaching, Subject and Classroom Allocation #120: Commit 7a452c9 pushed by Hein-Htet-Aung08

main

Today at 5:32 PM
1m 56s

...

✓ Student 2 hein teaching subject and classroom allocation

Student 2 - Teaching, Subject and Classroom Allocation #119: Pull request #34 opened by Hein-Htet-Aung08

student-2-hein-teaching-

Today at 5:32 PM
1m 15s

...

Student 2 - Teaching, Subject and Classroom Allocation

Student 2 - Teaching, Subject and Classroom Allocation #122

Re-run all jobs

...

Summary

All jobs

Unit and code validation

Build Docker images

Runtime and API smoke tests

Run details

Usage

Workflow file

Annotations

1 warning

Unit and code validation

succeeded 1 minute ago in 14s

Search logs

...

Set up job

1s

Check out repository

1s

Set up Python 3.11

2s

Install Student 2 dependencies

5s

Validate Python syntax

8s

Validate backend import

1s

Verify AI prompt files

8s

Run Student 2 automated tests

8s

Post Set up Python 3.11

8s

Post Check out repository

8s

Complete job

8s

Student 2 - Teaching, Subject and Classroom Allocation

Student 2 - Teaching, Subject and Classroom Allocation #122

Re-run all jobs

...

Summary

All jobs

Unit and code validation

Build Docker images

Runtime and API smoke tests

Run details

Usage

Workflow file

Annotations

1 warning

Build Docker images

succeeded 1 minute ago in 25s

Search logs

...

Set up job

8s

Check out repository

1s

Build database Docker image

18s

Build backend Docker image

8s

Build frontend Docker image

3s

Confirm Docker images exist

8s

Post Check out repository

8s

Complete job

8s

Build microservice images

Run and validate services

Generate CI evidence

Run details

Usage

Workflow file

Build microservice images

succeeded last week in 23s

Search logs

> Set up job

> Run actions/checkout@v4

> Validate Python syntax

> Build database-service

> Build backend-service

> Build frontend-service

> List built images

> Post Run actions/checkout@v4

> Complete job

8s

1s

1s

7s

7s

3s

8s

8s

8s

Summary

All jobs

Build microservice images

Run and validate services

Generate CI evidence

Run details

Usage

Workflow file

Annotations

1 warning

Run and validate services

succeeded last week in 54s

Search logs

> Set up job

> Run actions/checkout@v4

> Build images

> Start services

> Wait for services

> Verify seed data meets the ten record minimum

> Validate workload analysis endpoints

> Validate CRUD operations

> Validate alert recomputation is idempotent

> Validate AI-Mode degrades cleanly without an LLM runtime

> Verify prompts are packaged in the backend image

> Verify frontend assets

> Service logs

> Stop services

> Post Run actions/checkout@v4

> Complete job

1s

1s

22s

8s

2s

8s

1s

1s

8s

8s

1s

1s

8s

8s

8s

8s

Summary

All jobs

Build microservice images

Run and validate services

Generate CI evidence

Run details

Usage

Workflow file

Annotations

1 warning

Generate CI evidence

succeeded last week in 6s

Search logs

> Set up job

> Run actions/checkout@v4

> Generate reports

> Run actions/upload-artifact@v4

> Post Run actions/checkout@v4

> Complete job

1s

8s

8s

2s

8s

8s

Student 4 (Nicholas Hatzidimitriou)

Student 4 - Nicholas Hatzidimitriou (Research & Grant Management)

student-4-NicholasHatzidimitriou.yml

Filter workflow runs

88 workflow runs

Event

Status

Branch

Actor

added shared agentic loop

Student 4 - Nicholas Hatzidimitriou (Research & Grant Management) #91: Commit 68c367f pushed by NicholasHatzidimitriou

student-4-nicholas-research...

Sep 5, 17:06 GMT+10

57s

Merge pull request #21 from Hein-Htet-Aung08/student-5-matthew-perfor...

Student 4 - Nicholas Hatzidimitriou (Research & Grant Management) #90: Commit 69439c3 pushed by MattBarnard4

main

Sep 3, 16:47 GMT+10

53s

Improve feature navigation

Student 4 - Nicholas Hatzidimitriou (Research & Grant Management) #89: Pull request #21 opened by MattBarnard4

student-5-matthew-performan...

Sep 3, 16:46 GMT+10

58s

Student 4 - Nicholas Hatodimnou (Research & Grant Management)

added shared agentic loop #91

Re-run all jobs

Summary

All jobs

Annotations

1 warning

Validate Python syntax

succeeded 3 days ago in 12s

Search logs

Set up job

13s

Checkout repository

13s

Set up Python

8s

Install backend dependencies

4s

Compile check backend

8s

Install database-service dependencies

13s

Compile check database-service

8s

Install frontend dependencies

8s

Compile check frontend

8s

Post Set up Python

13s

Post Checkout repository

8s

Complete job

8s

Student 5 (Matthew Barnard)

Matthew - Performance and Professional Development

student-5-MatthewBarnard.yml

Filter workflow runs

100 workflow runs

Event

Status

Branch

Actor

This workflow has a workflow_dispatch event trigger.

Run workflow

Merge pull request #33 from Hein-Htet-Aung08/student-5-matthew-perfor...

Matthew - Performance and Professional Development #103: Commit 37643ee pushed by MattBarnard4

main

Today at 14:17

57s

Simplify Student 5 dashboard and retry metric loading

Matthew - Performance and Professional Development #102: Pull request #33 opened by MattBarnard4

student-5-matthew-performan...

Today at 14:16

58s

Simplify Student 5 dashboard and retry metric loading

Matthew - Performance and Professional Development #101: Commit 255fd58 pushed by MattBarnard4

student-5-matthew-performan...

Today at 14:15

1m 3s

Merge pull request #32 from Hein-Htet-Aung08/student-5-matthew-perfor...

Matthew - Performance and Professional Development #100: Commit a54c3f4 pushed by MattBarnard4

main

Today at 14:11

54s

Student 5 matthew performance and professional development

Matthew - Performance and Professional Development #99: Pull request #32 opened by MattBarnard4

student-5-matthew-performan...

Today at 14:10

52s

Completed student 5 agentic ai loop feature

Matthew - Performance and Professional Development #98: Commit 877cd08 pushed by MattBarnard4

student-5-matthew-performan...

Sep 8, 20:54 GMT+10

59s

couple fixes

Matthew - Performance and Professional Development #97: Commit c893251 pushed by MattBarnard4

student-5-matthew-performan...

Sep 8, 19:32 GMT+10

53s

student-5-MatthewBarnard.yml

on: push

Unit and API tests

12s

Build and smoke-test conta...

33s

Package CI evidence

5s

Docker Compose Evidence

Docker-compose up command

```
#147 [allocation-backend] resolving provenance for metadata file
#147 DONE 0.0s
[+] up 34/34
✓ Image faculty-management-system-asd-student4-frontend      Built      5.6s
✓ Image faculty-management-system-asd-student4-database      Built      5.6s
✓ Image faculty-management-system-asd-student3-database      Built      5.6s
✓ Image faculty-management-system-asd-student5-database      Built      5.6s
✓ Image faculty-management-system-asd-student3-backend      Built      5.6s
✓ Image faculty-management-system-asd-student4-backend      Built      5.6s
✓ Image faculty-management-system-asd-student5-frontend      Built      5.6s
✓ Image faculty-management-system-asd-staff-backend          Built      5.6s
✓ Image faculty-management-system-asd-staff-database          Built      5.6s
✓ Image faculty-management-system-asd-allocation-database      Built      5.5s
✓ Image faculty-management-system-asd-staff-frontend          Built      5.5s
✓ Image faculty-management-system-asd-student5-backend      Built      5.5s
✓ Image faculty-management-system-asd-allocation-frontend      Built      5.5s
✓ Image faculty-management-system-asd-student3-frontend      Built      5.5s
✓ Image faculty-management-system-asd-allocation-backend      Built      5.5s
✓ Network faculty-management-system-asd_faculty-network      Created     0.0s
✓ Container faculty-management-system-asd-allocation-database-1 Created     0.1s
✓ Container student4-database                                Created     0.1s
✓ Container unified-home                                      Created     0.1s
✓ Container faculty-management-system-asd-staff-database-1    Created     0.1s
✓ Container ollama                                             Created     0.1s
✓ Container student5-database                                  Created     0.1s
✓ Container student3-database                                  Created     0.1s
✓ Container ollama-init                                         Created     0.1s
✓ Container student5-backend                                    Created     0.1s
✓ Container faculty-management-system-asd-staff-backend-1     Created     0.1s
✓ Container student3-backend                                    Created     0.1s
✓ Container student4-backend                                    Created     0.1s
✓ Container student5-frontend                                    Created     0.1s
✓ Container student3-frontend                                    Created     0.1s
✓ Container faculty-management-system-asd-staff-frontend-1    Created     0.1s
✓ Container faculty-management-system-asd-allocation-backend-1 Created     0.1s
✓ Container student4-frontend                                    Created     0.1s
✓ Container faculty-management-system-asd-allocation-frontend-1 Created     0.1s
```

Docker ps command

PS C:\Users\kring\Documents\Github\faculty-management-system-ASD> docker ps

CONTAINER ID	NAME	IMAGE	COMMAND	CREATED	STATUS	PORTS
87acf5eb16a6	faculty-management-system-asd-allocation-frontend	faculty-management-system-asd-allocation-frontend-1	"/docker-entrypoint..."	About a minute ago	Up About a minute	0.0.0.0:3002->80/tcp, [::]:3002->80/tcp
81ce39f98642	faculty-management-system-asd-staff-frontend	faculty-management-system-asd-staff-frontend-1	"/docker-entrypoint..."	About a minute ago	Up About a minute	0.0.0.0:8001->80/tcp, [::]:8001->80/tcp
a7dccccdbcd3	faculty-management-system-asd-allocation-backend	faculty-management-system-asd-allocation-backend-1	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp
6bc2daef944e	faculty-management-system-asd-student5-frontend	faculty-management-system-asd-student5-frontend	"/docker-entrypoint..."	About a minute ago	Up About a minute	0.0.0.0:8005->80/tcp, [::]:8005->80/tcp
9ef692bcf47f	faculty-management-system-asd-student3-frontend	faculty-management-system-asd-student3-frontend	"/docker-entrypoint..."	About a minute ago	Up About a minute	0.0.0.0:8003->80/tcp, [::]:8003->80/tcp
781614c2dcdbd	faculty-management-system-asd-student4-frontend	faculty-management-system-asd-student4-frontend	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:8004->8004/tcp, [::]:8004->8004/tcp
5ef8ab48aacc	faculty-management-system-asd-staff-backend	faculty-management-system-asd-staff-backend-1	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
ff822faa8bcc	faculty-management-system-asd-student4-backend	faculty-management-system-asd-student4-backend	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:5004->5004/tcp, [::]:5004->5004/tcp
ad916255b365	faculty-management-system-asd-student5-backend	faculty-management-system-asd-student5-backend	"python app.py"	About a minute ago	Up About a minute (healthy)	0.0.0.0:5005->5005/tcp, [::]:5005->5005/tcp
e16697b8aa85	faculty-management-system-asd-student3-backend	faculty-management-system-asd-student3-backend	"python backend.serv..."	About a minute ago	Up About a minute	0.0.0.0:5003->5003/tcp, [::]:5003->5003/tcp
b38986c9287c	faculty-management-system-asd-student4-database	faculty-management-system-asd-student4-database	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:5104->5104/tcp, [::]:5104->5104/tcp
fae2453a1a3b	faculty-management-system-asd-staff-database	faculty-management-system-asd-staff-database-1	"tail -f /dev/null"	About a minute ago	Up About a minute	
971bc298dec3	faculty-management-system-asd-student3-database	faculty-management-system-asd-student3-database	"python app.py"	About a minute ago	Up About a minute	0.0.0.0:5103->5103/tcp, [::]:5103->5103/tcp
fa5a9577c40c	faculty-management-system-asd-student5-database	faculty-management-system-asd-student5-database	"python app.py"	About a minute ago	Up About a minute (healthy)	0.0.0.0:5105->5105/tcp, [::]:5105->5105/tcp
1525cab7934e	faculty-management-system-asd-allocation-database	faculty-management-system-asd-allocation-database-1	"python app.py"	About a minute ago	Up About a minute	6002/tcp
f47d0d255413	ollama/ollama:latest	ollama/ollama:latest	"/bin/ollama serve"	3 days ago	Up About a minute	11434/tcp
ad52ec2f850d	nginx:alpine	nginx:alpine	"/docker-entrypoint..."	3 days ago	Up About a minute	0.0.0.0:8000->80/tcp, [::]:8000->80/tcp

Containers visual in Docker Desktop (visual alternative to docker ps)

☐		Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk rea	Actions
☐	●	ollama	f47d0d255413	ollama/oll...		0%	36.61MB / 7.61G	0.47%	400MB	
☐	●	unified-home	ad52ec2f850d	nginx:alpin	8000:80	0%	12.75MB / 7.61G	0.16%	8.47MB	
☐	●	allocation-frc	87acf5eb16a6	faculty-ma	3002:80	0%	9.92MB / 7.61GB	0.13%	0B / 12...	
☐	●	staff-fronten	81ce39f98642	faculty-ma	8001:80	0%	9.93MB / 7.61GB	0.13%	586KB /	
☐	●	allocation-ba	a7dccccdbcd3	faculty-ma	5002:5002	0.28%	117.9MB / 7.61G	1.51%	57.3KB	
☐	●	student5-fror	6bc2daef944e	faculty-ma	8005:80	0%	10.21MB / 7.61G	0.13%	9.34MB	
☐	●	student3-fror	9ef692bcf47f	faculty-ma	8003:80	0%	10.07MB / 7.61G	0.13%	0B / 12...	
☐	●	student4-fror	781614c2dcdbd	faculty-ma	8004:8004	0.08%	45.3MB / 7.61GB	0.58%	7.23MB	
☐	●	staff-backen	5ef8ab48aacc	faculty-ma	5001:5001	0.29%	113.6MB / 7.61G	1.46%	22.2MB	
☐	●	student4-bac	ff822faa8bcc	faculty-ma	5004:5004	0.34%	97.06MB / 7.61G	1.25%	22.6MB	
☐	●	student5-bac	ad916255b365	faculty-ma	5005:5005	5.19%	64.89MB / 7.61G	0.83%	23.1MB	
☐	●	student3-bac	e16697b8aa85	faculty-ma	5003:5003	0.3%	94.43MB / 7.61G	1.21%	21.4MB	
☐	●	student4-dat	b38986c9287c	faculty-ma	5104:5104	0.09%	48.96MB / 7.61G	0.63%	6.89MB	
☐	●	staff-databa	fae2453a1a3b	faculty-ma		0.01%	464KB / 7.61GB	0.01%	98.3KB	
☐	●	student3-dat	971bc298dec3	faculty-ma	5103:5103	0.07%	46.91MB / 7.61G	0.6%	6.96MB	
☐	●	student5-dat	fa5a9577c40c	faculty-ma	5105:5105	5.45%	26.27MB / 7.61G	0.34%	14.1MB	
☐	●	allocation-da	1525cab7934e	faculty-ma		0.11%	47.48MB / 7.61G	0.61%	553KB /	

Docker-compose down showing the entire stack stops cleanly.

```

PS C:\Users\kring\Documents\GitHub\faculty-management-system-ASD> docker-compose down
[+] down 19/19
✓ Container student4-frontend Removed
✓ Container student3-frontend Removed
✓ Container faculty-management-system-asd-staff-frontend-1 Removed
✓ Container faculty-management-system-asd-allocation-frontend-1 Removed
✓ Container student5-frontend Removed
✓ Container unified-home Removed
✓ Container ollama-init Removed
✓ Container faculty-management-system-asd-allocation-backend-1 Removed
✓ Container student5-backend Removed
✓ Container student4-backend Removed
✓ Container faculty-management-system-asd-staff-backend-1 Removed
✓ Container student3-backend Removed
✓ Container faculty-management-system-asd-allocation-database-1 Removed
✓ Container student4-database Removed
✓ Container student3-database Removed
✓ Container faculty-management-system-asd-staff-database-1 Removed
✓ Container student5-database Removed
✓ Container ollama Removed
✓ Network faculty-management-system-asd_faculty-network Removed

```

Agentic Loop Workflow Record

Student 1 (Andy Lam)

Contribution

- Assessment of AI mode integration (initially db, endpoints and architecture)
- The AI-mode integration review consists of:
 - collectors/ai_mode_collector.py - A static analysis collector which scans a students backend source for Ollama integration, checking for error handling, explicit timeouts for the LLM call and detects externalised prompt template files. Additionally it checks whether AI-generated output is persisted to a database table.
 - pipelines/ai_mode_pipeline.py - Prompt assembly pipeline for new mode.
 - Three prompt assets (system_prompt.txt, task_prompt.txt, context_prompt.txt) instructing a reviewer model to critique AI-mode integration quality from collected evidence.

Evidence

```
✓ # AI_MODE review - student-1-Andy-Lam

Generated: 20260907-203925

✓ ## OBSERVE (real evidence)

...
AI-MODE INTEGRATION EVIDENCE for student-1-Andy-Lam:
- backend\llm_client.py: calls Ollama; error handling MISSING; explicit timeout MISSING.
- prompt templates found: staff_analysis_system_prompt.txt, staff_analysis_task_prompt.txt (externalised, not hardcoded).
- AI output appears to be persisted to a database table (INSERT into an *ai* table found).
...

✓ ## REVIEW (model output)

- **Error handling**: Wrap the Ollama call in `llm_client.py` in a try/except so a model timeout returns a clean error instead of a 500.
- **Prompt externalisation**: Move the inline prompt string in `app.py` into an external prompt template file, consistent with the rest of the codebase.
```

Findings and Validation

- The first advice about error handling was accurate as my llm_client.py was lacking a try/except block or timeout around the call (confirmed against evidence and source file).
- The second finding however was an issue with the model, qwen2.5:0.5b partly copied the example content instead of treating it as a template for formatting. Hence, giving a response contradictory to the evidence.

Action Taken

- Fixed the issue with try/except block for the llm_client.py around the Ollama call.
- Did not act on the prompt externalisation finding as it was incorrect against my code.
- The Plan -> Act -> Observe -> Adapt Loop has been demonstrated through identifying a lack of review in AI-mode integration quality (plan), building the reviewer for the AI-mode (act), collecting evidence and model's review output (observe) and making adjustments according to their validity (adapt).

Student 2 (Hein Htet Aung)

Contribution

- Ran the shared AI-Mode reviewer against personal features and checked its findings against the source code.
- Review all written prompts for the agentic loop as well as the generated reviewer output for my personal features
- The validity of the conclusions was checked by comparing the evidence gathered and the AI suggestions to the real Student 2 implementation.

Evidence

```
ai-services > agentic_loop > reports > al_mode-student-2-20260909-195351.md > cat # AI_MODE review - student-2Hein-Htet-Aung
1 # AI_MODE review - student-2Hein-Htet-Aung
2
3 Generated: 20260909-195351
4
5 ## OBSERVE (real evidence)
6
7 ```
8 AI-MODE INTEGRATION EVIDENCE for student-2Hein-Htet-Aung:
9 - backend/services/llm_client.py: calls Ollama; error handling MISSING; explicit timeout MISSING.
10 - no external prompt template files found (prompt likely hardcoded inline, if present).
11 - no evidence of AI-generated output being persisted to a database table.
12 ```
13
14 ## REVIEW (model output)
15
16 - **Error handling**: Wrap the Ollama call in 'llm_client.py' in a try/except block so a model timeout returns a clean error instead of a 500.
17 - **Prompt externalisation**: Move the inline prompt string in 'app.py' into an external prompt template file, consistent with the rest of the codebase.
18
```

Findings and Validation

- The reviewer pointed out that the Ollama call inside backend/services/llm_client.py lacks an explicit timeout or exception handling. I checked the source code and this finding was correct.
- The reviewer also mentioned that there were no external prompt templates, and advised to move an inline prompt from the app. This was a false finding. Student 2 already records its ai prompts externally in prompts/allocation/system_prompt.txt and prompts/allocation/staff_recommendation_prompt.txt which are loaded by llm_client.py.
- The reason for the finding of the improper prompt was because the Agentic Loop collector did not pick up the nested Student 2 prompt directory, and not due to the Student 2 implementation.

Action taken

- Explicitly handle timeouts in backend/services/llm_client.py so an ollama request can't wait forever
- Added exception handling around ollama chat completion call. When an ollama request fails, the error is caught by the current AI recommendation endpoint.
- No action was taken on the prompt externalization advice since the finding was wrong. Student 2 already stores and loads its prompts outside from the directory prompts/allocation/.
- The review then illustrated the Adapt stage by implementing the legitimate error-handling recommendation and discarding the unsupported prompt recommendation after testing it against the source code.

Student 3 (Tristan Lim)

Contribution

- Ran the shared AI-Mode reviewer against my own feature and checked its findings against the source code.

Evidence

```
# AI_MODE review - student-3Tristan-Lim

Generated: 20260909-150040

## OBSERVE (real evidence)
...
AI-MODE INTEGRATION EVIDENCE for student-3Tristan-Lim:
- backend-service\routes\ai_mode.py: calls Ollama; error handling present; explicit timeout MISSING.
- backend-service\services\llm_client.py: calls Ollama; error handling present; explicit timeout set.
- backend-service\services\rebalance_agent.py: calls Ollama; error handling present; explicit timeout MISSING.
- no external prompt template files found (prompt likely hardcoded inline, if present).
- no evidence of AI-generated output being persisted to a database table.
...

## REVIEW (model output)

- **Error handling**: Wrap the Ollama call in `llm_client.py` in a try/except so a model timeout returns a clean error instead of a 500.
- **Prompt externalisation**: Move the inline prompt string in `app.py` into an external prompt template file, consistent with the rest of the codebase.
```

Findings and Validation

- The tool check for the word “try” appears somewhere in the file. It does not check if it's actually protecting the Ollama call. There is once shared timeout for every AI call set inside llm_client.py and the error handling for a failed AI calls happens in routes/ai_mode.py
- No external prompt templates is false. It only scans the top level of prompts, however mine is two folders deeper in prompts/service/implementation, so it does not find them.

Action taken

- No changes to the code. My feature already does what the review claimed I didn't.
- The bug is within the evidence collector.
- Demonstrated Plan, spot a suspicious result. Act, read the collector's source. Observe, confirm false negatives against real files. Adapt, concluded no code change was needed.

Student 4 (Nicholas Hatzidimitriou)

Contribution

- Built the core shared agentic loop architecture based off the labs.
- It consists of:
 - core/orchestrator.py: resolves a review mode via config/review_config.py and runs the Observe → Prompts → LLM → Done.
 - core/prompt_registry.py: resolves and validates prompt files, raising clear errors if a file is missing
 - core/ai_runner.py: openai compatible client that is directed towards the shared Ollama runtime, supporting two modes: implementation model and architecture mode
 - collectors/db_collector.py, endpoints_collector.py, architecture_collector.py: three evidence collectors, live HTTP endpoint testing against each service

```

1 # ENDPOINTS review - student-4Nicholas-Hatzidimitriou
5 ## OBSERVE (real evidence)
   not exercised (non GET/POST); DELETE /project_staff/1 was discovered but not exercised (non GET/POST);
   GET /publications responded 500 (6 ms); GET /publications/1 responded 500 (8 ms); POST /publications
   responded 400 (3 ms); PUT /publications/1 was discovered but not exercised (non GET/POST); DELETE /
   publications/1 was discovered but not exercised (non GET/POST).
9   ...
10
11 ## REVIEW (model output)
12
13 - Endpoint testing: Several endpoints returned 404 or 500 status codes, indicating potential issues with
   route implementation or error handling. For example, POST /projects/1/generate_summary and others like
   POST /grant_alerts responded with 404 or 500 errors.
14 - Response formatting: Inconsistent response formats across endpoints, particularly for error responses
   which often returned 404 or 500 without clear message structures.
15
16 Recommendations:
17 - Error handling: Implement proper error messages and consistent status codes for all endpoints that
   currently return 404 or 500. For instance, ensure POST /projects/1/generate_summary returns a meaningful
   404 or 500 with an appropriate message.
18 - Response formatting: Standardize the structure of error responses to include clear messages and status
   codes across all endpoints. This will improve user understanding and debugging capabilities.
19 |

```

Finding and Validation

- GET /publications and GET /publications/1 returned an unhandled 500. Unlike the AI model clients that all wrap their HTTP calls in try/except and degrade gracefully, database_api.py doesn't have that.
- The second finding was incorrect. The generate_summary 404 isn't in the evidence collected. Most likely since the small model made a generalised review rather than having the evidence behind it.

Action Taken

- Added try/except handling around database_api.py HTTP calls to match what was done in the other AI model clients.
- Didn't do anything on the generate_sumamry finding. Left it as it was.

Student 5 (Matthew Barnard)

Contribution

Built an individual development-goal integrity review using the group's shared agentic loop architecture.

It consists of:

- `development_integrity_collector.py`: loads my real schema and validation modules, performs a test using a disposable in-memory SQLite database, and makes read-only requests to my running database API.
- `development_integrity_pipeline.py`: places the collected evidence into the external prompt templates before sending it to the model.
- Three external prompt files: `system_prompt.txt`, `task_prompt.txt`, and `context_prompt.txt`.
- A configuration entry that makes the review available as Student 5 option 6 in the shared loop.
- Reporting support that saves the PLAN, ACT, OBSERVE and ADAPT stages in Markdown and JSON.
- Automated tests for the collector, pipeline, prompt configuration and compatibility with the existing review modes.

My review focuses on whether a development goal can be marked as Completed while its progress is below 100%. This is separate from the existing AI-Mode review, which checks how each feature connects to Ollama.

```

"""text
DEVELOPMENT RECORD INTEGRITY EVIDENCE for student-5Matthew-Barnard:
PLAN concern: development-goal lifecycle integrity between status and progress.
Source files inspected: database-service/schema.py, resource_config.py, validation.py.
DevelopmentGoals status values: Cancelled, Completed, In Progress, On Hold, Planned; progress range: 0 to 100.
Disposable validation + in-memory SQLite probes (the real database was not modified):
- REJECTED: Completed development goal with progress=25 (ValueError: progress must be 100 when status is Completed).
Read-only live API check at http://localhost:5105:
- GET /health -> HTTP 200.
- GET /development-goals -> HTTP 200.
- Live rows: 10 development goals; 1 Completed goal(s) have progress other than 100.
"""

## OBSERVE (model recommendations)

Finding: REJECTED
Evidence: REJECTED: Completed development goal with progress=25 (ValueError: progress must be 100 when status is Completed).
Recommendation: Retain the cross-field validation and its regression test.
Validation: Run the API test that submits status=Completed with progress=25 and expects HTTP 400.
```

Finding and Validation

- The initial review found that a development goal could be saved with `status=Completed` and `progress=25`.
- This was demonstrated using my real `validate_payload` function followed by a real SQLite insert in a disposable in-memory database. It did not modify the application database.
- The read-only API check also found one previously stored completed goal with progress below 100%.
- Qwen recommended adding cross-field validation so a completed goal must have exactly 100% progress.
- I checked the recommendation against my validation code and accepted it because the status and progress were previously validated separately, but their relationship was not checked.

Action Taken

- Added `validate_record_consistency()` to `database-service/validation.py`.
- The new validation rejects a development goal when its status is Completed but its progress is not 100.
- Updated the database API's partial-update path so it combines the existing record with the requested changes before checking consistency. This prevents an incomplete goal from being marked completed through a status-only update.
- Added tests covering:
 - Rejection of a completed goal at 25% progress.
 - Acceptance of a completed goal at 100% progress.
 - Rejection of a partial update that marks an incomplete goal as completed.
- All 12 database tests and all four agentic-loop tests passed.
- I ran the review again after implementing the change. The same test record was rejected with: `ValueError: progress must be 100 when status is Completed`

Known Issues and Limitations

Academic Staff System (Andy Lam)

- Skill level and expertise are not editable after creation, the edit form intentionally disables where the only way to access them is through direct changes to the database. Future releases can include an endpoint for this.
- UI only allows one qualification and expertise per staff and the AI analysis features aggregates multiple rows. In addition, the add/edit form only allows one input of expertise and qualification per staff.
- The suitability score is a basic heuristic (skill level multiplied by 2). Unlike the AI generated summary, it does not account for qualifications, expertise and availability. This was a deliberate decision for a deterministic calculation in the short span of release 0.
- If a user navigates away from the AI analysis page while it is in progress, it is not cancelled. Not prioritised given time constraints of release 0.
- The frontend `api_base` is hard coded for local development (`http://127.0.0.1:5001`). This needs to be environment configurable before release 2 cloud deployment.

Teaching, Subject and Classroom Allocation (Hein Htet Aung)

- AI staff proposals depend on the availability of Student 1 Staff Management and Student 3 Workload and Availability services. If either service is unavailable, the recommendation functionality cannot get full staff context.
- AI answers are expected to follow a structured JSON format. Imperfect or incomplete model output can still fail the recommendation request, even though erroneous recommendations are filtered out in the final result.
- Student 1 owns the staff records. That means that the `assigned_staff_member` is not a local foreign key but rather an external staff reference. A staff record can therefore

become stale if it is removed from Student 1 after an allocation has already been stored.

- The recommendation approach uses a compact local Qwen model, which leads to inconsistent quality and reasoning of advice across multiple executions even if the contexts are similar.
- Release 0 uses Docker networking and local service URLs. Additional preparation will be required for future cloud implementation.

Workload and Availability Management (Tristan Lim)

- Small local models frequently produce rebalance proposals that reference activities that do not exist or that would push a staff over their cap. These are rejected by the rules engine. Although, the rebalance cycle can return zero accepted proposals.
- The free-text is returning json only.
- Staff_id is a soft to Student 1 service instead of a local foreign key. If that service roster changes without this feature's seed data being updated, reconcile will report the mismatch and will not resolve automatically.

Research and Grant Management (Nicholas Hatzidimitriou)

- Recommend New Staff and Explain Staff Current Fit AI features are very dependent on the Staff Management and Workload and Availability microservices being reachable. If either are unavailable, a response will still be generated, but placeholders will be in place, not giving a correct result.
- Deleting a research project that still has a grant, publication or staff assignment to it will fail and not cleanly remove the record.

Performance and Professional Development (Matthew Barnard)

- Staff IDs are stored as external references and are not enforced as foreign keys, so locally stored records can reference a staff member who no longer exists.
- Student 5 currently has no integration with Student 3 Workload and Availability, so AI development recommendations do not consider staff workload or availability.
- Staff training validation does not currently require a completion date when training is marked Completed or ensure that it occurs after the enrolment date.
- AI validation does not completely prevent duplicate or already-enrolled training recommendations, which may result in repeated suggestions.

GitHub Commit Logs

Student 1 (Andy Lam)

Commits

main	Alamb-beyonder	All time
Commits on Sep 7, 2026		
Add try/except block to llm_client.py and timeout for ollama call		
Alamb-beyonder committed 29 minutes ago · 2 / 2		
6b147ea		
Merge pull request #29 from Hein-Htet-Aung08/student-1-andy-staff-management		
Alamb-beyonder authored 1 hour ago · 1 / 1		
Verified		
36c68bb		
Merge branch 'main' of https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD into student-1-andy-staff-management		
Alamb-beyonder committed 1 hour ago · 1 / 1		
33bc06b		
Fixed bugs with styling and button		
Alamb-beyonder committed 1 hour ago · 1 / 1		
25bf768		
Add port to shared ollama		
Alamb-beyonder committed 2 hours ago · 1 / 1		
5a7e44b		
Merge branch 'main' of https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD into student-1-andy-staff-management		
Alamb-beyonder committed 2 hours ago · 4 / 4		
835da84		
Commits on Sep 3, 2026		
Merge pull request #12 from Hein-Htet-Aung08/student-3-tristan-workload-and-availability-management		
Alamb-beyonder authored 4 days ago · 6 / 6		
Verified		
7b6f5a6		
Merge pull request #10 from Hein-Htet-Aung08/student-3-tristan-workload-and-availability-management		
Alamb-beyonder authored 5 days ago · 3 / 3		
Verified		
895b6de		
linked css to student 1 microservice		
Alamb-beyonder committed 5 days ago		
ca4b524		
Linked shareed css to student 1 microservice		
Alamb-beyonder committed 5 days ago · 2 / 2		
8c26c9d		
Commits on Sep 2, 2026		
Merge pull request #8 from Hein-Htet-Aung08/student-4-nicholas-research-and-grant-management		
Alamb-beyonder authored 5 days ago		
Verified		
fc951cd		
Fix CORS for frontend-backend communication in Docker		
Alamb-beyonder committed 5 days ago · 1 / 1		
fee2c92		
Merge pull request #4 from Hein-Htet-Aung08/student-4-nicholas-research-and-grant-management		
Alamb-beyonder authored 5 days ago		
Verified		
6b8581e		
added task and system prompts for ai, adjusted dockerfile and fixed bug with edit and add staff		
Alamb-beyonder committed 5 days ago · 2 / 2		
9eaf712		
Add docker-compose.yml skeleton with student-1 services		
Alamb-beyonder committed 5 days ago · 1 / 1		
4037c5f		
Add dockerfiles for database/backend (didnt push before) and github actions CI workflow		
Alamb-beyonder committed 5 days ago · 1 / 1		
a3590fa		
add dockerfiles for database, frontend and backend + db_path env var support		
Alamb-beyonder committed 5 days ago		
6a76645		

Commits on Sep 1, 2026

Add/edit form + ai analysis generation  Alamb-beyond committed last week	e111fac		
add staff list, detail view, filter and delete for frontend  Alamb-beyond committed last week	af22fdb		

Commits on Aug 31, 2026

Add ai analysis route with ollama integration (gwen2.5:0.5b)  Alamb-beyond committed last week	fe4fb77		
Add search and filter for staff api  Alamb-beyond committed last week	eb18153		

Commits on Aug 29, 2026

Added endpoints for getting staff availability,expertise and qualifications  Alamb-beyond committed last week	6bd0613		
--	---------	---	---

Commits on Aug 28, 2026

Added staff CRUD routes and moved manual test JSON files into tests/manual  Alamb-beyond committed last week	7cd036c		
---	---------	---	---

Commits on Aug 26, 2026

seed.py complete, database layer finished  Alamb-beyond committed 2 weeks ago	15cfadf		
--	---------	---	---

Commits on Aug 25, 2026

Created all the databases required for staff management system  Alamb-beyond committed 2 weeks ago	c1a3a14		
---	---------	---	---

Student 2 (Hein Htet Aung)

student-2-hein-t...	All users	All time
Commits on Sep 9, 2026		
Adaptation to the agentic loop.	Hein-Htet-Aung08 committed 12 minutes ago · ✓ 9 / 9	a6caf84
Merge pull request #34 from Hein-Htet-Aung08/student-2-hein-teaching-subject-and-classroom-allocation	Hein-Htet-Aung08 authored 2 hours ago · ✓ 3 / 3	Verified7a452c9
Workflow error is fixed.	Hein-Htet-Aung08 committed 2 hours ago · ✓ 6 / 6	1199f75
New workflow is added.	Hein-Htet-Aung08 committed 2 hours ago · ✗ 2 / 3	0591a0d
Minor typo fixes.	Hein-Htet-Aung08 committed 3 hours ago	9605048
Merge pull request #33 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development	MattBarnard4 authored 6 hours ago · ✓ 3 / 3	Verified37643ee
Simplify Student 5 dashboard and retry metric loading	Mattonut committed 6 hours ago · ✓ 6 / 6	255fd58
Merge pull request #32 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development	MattBarnard4 authored 6 hours ago · ✓ 3 / 3	Verifieda54c3f4
Commits on Sep 8, 2026		
Completed student 5 agentic ai loop feature	Mattonut committed yesterday · ✓ 6 / 6	877cd08
couple fixes	Mattonut committed yesterday · ✓ 3 / 3	c893251
Commits on Sep 7, 2026		
Merge pull request #31 from Hein-Htet-Aung08/student-2-hein-teaching-subject-and-classroom-allocation	Hein-Htet-Aung08 authored 2 days ago · ✓ 1 / 1	Verified5d02646
Delete docs/architecture/Integrated Architecture	Alamb-beyonder authored 2 days ago	Verified7a4c299
Add files via upload	Alamb-beyonder authored 2 days ago	Verified6f6ef36
Tests added and some frontend and backend features fixed.	Hein-Htet-Aung08 committed 2 days ago · ✓ 2 / 2	0b69f85
Create report0_placeholder	Alamb-beyonder authored 2 days ago	Verified8922009
Create docs/architecture/ directory	Alamb-beyonder authored 2 days ago	Verified2d7c823
Update README.md	Alamb-beyonder authored 2 days ago	Verified0c0b5bf
Merge pull request #30 from Hein-Htet-Aung08/student-1-andy-staff-management	TLim05 authored 2 days ago · ✓ 1 / 1	Verified630d96a
Add try/except block to llm_client.py and timeout for ollama call	Alamb-beyonder committed 2 days ago · ✓ 2 / 2	6b147ea
Merge pull request #29 from Hein-Htet-Aung08/student-1-andy-staff-management	Alamb-beyonder authored 2 days ago · ✓ 1 / 1	Verified36c68bb
Merge branch 'main' of https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD into student-1-andy-staff-management	Alamb-beyonder committed 2 days ago · ✓ 1 / 1	33bc06b
Fixed bugs with styling and button	Alamb-beyonder committed 2 days ago · ✓ 1 / 1	25bf768
Merge pull request #28 from Hein-Htet-Aung08/student-1-andy-staff-management	TLim05 authored 2 days ago · ✓ 1 / 1	Verified8733d00
Add port to shared ollama	Alamb-beyonder committed 2 days ago · ✓ 1 / 1	5a7e44b

Merge branch 'main' of <https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD> into student-1-andy-staff-management

Alamb-beyonder committed 2 days ago ·

4 / 4

Commits on Sep 5, 2026

Merge pull request #27 from Hein-Htet-Aung08/student-4-nicholas-research-and-grant-management

Verified

30618f8

NicholasHatzi authored 4 days ago

removed unnecessary file

8739b17

NicholasHatzi committed 4 days ago

added shared agentic loop

88c367f

NicholasHatzi committed 4 days ago ·

6 / 6

Commits on Sep 3, 2026

Merge pull request #26 from Hein-Htet-Aung08/student-2-hein-teaching-subject-and-classroom-allocation

Verified

ab38b78

Hein-Htet-Aung08 authored last week ·

1 / 1

Merge branch 'student-2-hein-teaching-subject-and-classroom-allocation' of <https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD> into student-2-hein-teaching-subject-and-classroom-alloc...

e6040e7

Hein-Htet-Aung08 committed last week ·

2 / 2

AI recommender bug fixed.

20c28e8

Hein-Htet-Aung08 committed last week

Merge pull request #25 from Hein-Htet-Aung08/student-2-hein-teaching-subject-and-classroom-allocation

Verified

b0bbd71

Hein-Htet-Aung08 authored last week ·

1 / 1

Merge branch 'main' into student-2-hein-teaching-subject-and-classroom-allocation

Verified

f79a712

Hein-Htet-Aung08 authored last week ·

2 / 2

Align Student 2 Docker build contexts

6bbd874

Hein-Htet-Aung08 committed last week ·

1 / 1

Fix database Docker build context

98cc9ee

Hein-Htet-Aung08 committed last week ·

0 / 1

Commits on Sep 3, 2026				
Fix Student 2 CI Paths.			a5907c1	
Hein-Htet-Aung08 committed last week · 0 / 1				
Update CI workflow to include its own path		Verified	5d5e136	
Hein-Htet-Aung08 authored last week · 0 / 1				
Merge pull request #24 from Hein-Htet-Aung08/main		Verified	4bf9ced	
Hein-Htet-Aung08 authored last week				
Merge branch 'student-2-hein-teaching-subject-and-classroom-allocation' into main		Verified	ada1425	
Hein-Htet-Aung08 authored last week				
CI/CD workflow added.			4560699	
Hein-Htet-Aung08 committed last week				
Add CI workflow for Student 2 project		Verified	33da98e	
Hein-Htet-Aung08 authored last week				
Merge pull request #23 from Hein-Htet-Aung08/student-2-hein-teaching-subject-and-classroom-allocation		Verified	a9c6882	
Hein-Htet-Aung08 authored last week				
Commit			11df9cf	
Hein-Htet-Aung08 committed last week				
Merge pull request #22 from Hein-Htet-Aung08/student-2-hein-teaching-subject-and-classroom-allocation		Verified	d3525b1	
Hein-Htet-Aung08 authored last week				
Merge branch 'main' into student-2-hein-teaching-subject-and-classroom-allocation		Verified	8613c50	
Hein-Htet-Aung08 authored last week · 3 / 3				
Merge branch 'student-2-hein-teaching-subject-and-classroom-allocation' of https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD into student-2-hein-teaching-subject-and-classroom-alloc...			cb4f52c	
Hein-Htet-Aung08 committed last week				
Merge pull request #21 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development		Verified	69439c5	
MattBarnard4 authored last week · 7 / 7				
All frontends are fixed			77746bf	
Hein-Htet-Aung08 committed last week				
Improve feature navigation			6601e53	
Mattonut committed last week · 7 / 7				

Subject offers CRUD fixed Hein-Htet-Aung08 committed last week	b401879		
Subjects frontend fixed. Hein-Htet-Aung08 committed last week	72eb994		
Merge pull request #19 from Hein-Htet-Aung08/student-2-hein-teaching-subject-and-classroom-allocation Hein-Htet-Aung08 authored last week	Verified c9f91ea		
Before a risky change Hein-Htet-Aung08 committed last week	9ab3389		
Merge pull request #20 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development MattBarnard4 authored last week · 3 / 3	Verified ce0429b		
Remove AI mode scope disclaimer Mattonut committed last week · 6 / 6	b054ce2		
Merge branch 'main' into student-2-hein-teaching-subject-and-classroom-allocation Hein-Htet-Aung08 authored last week · 3 / 3	Verified 7da167b		
Commit for video Hein-Htet-Aung08 committed last week	5dd1bb9		
Before a risky change Hein-Htet-Aung08 committed last week	5836d42		
Merge pull request #18 from Hein-Htet-Aung08/student-3-tristan-workload-and-availability-management TLim05 authored last week · 3 / 3	Verified 2868830		
Edited Text TLim05 committed last week · 6 / 6	95b5cd9		
Merge pull request #17 from Hein-Htet-Aung08/student-3-tristan-workload-and-availability-management TLim05 authored last week · 3 / 3	Verified 0779ff4		
Changed LLM TLim05 committed last week · 6 / 6	7cdeb8e		
Merge pull request #16 from Hein-Htet-Aung08/student-3-tristan-workload-and-availability-management TLim05 authored last week	Verified 69917dd		
Enforced LF Line Endings TLim05 committed last week	c92f9b3		

Merge pull request #15 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development TLim05 authored last week · 3 / 3	Verified 791a0f9		
Merge pull request #14 from Hein-Htet-Aung08/student-4-nicholas-research-and-grant-management NicholasHatzl authored last week · 3 / 3	Verified 2beccb8		
Merge branch 'main' into student-4-nicholas-research-and-grant-management NicholasHatzl committed last week	24980bb		
added CI workflow to student-4 NicholasHatzl committed last week · 3 / 3	5f8caac		
Add grounded fallback for invalid AI recommendations Mattonut committed last week · 6 / 6	b473ee0		
Merge remote-tracking branch 'origin/main' into student-5-matthew-performance-and-professional-development Mattonut committed last week	0b9ec0d		

Commits on Sep 3, 2026		
Remove student 5 evidence guide	8dd9769	
Mattonut committed last week		
Rename student 5 frontend and backend service folders	9e83d36	
Mattonut committed last week · ✓ 3 / 3		
Merge branch 'student-2-hein-teaching-subject-and-classroom-allocation' of https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD into student-2-hein-teaching-subject-and-classroom-alloc... new	5949f4a	
Hein-Htet-Aung08 committed last week		
Merge pull request #13 from Hein-Htet-Aung08/main new	Verified a494122	
Hein-Htet-Aung08 authored last week		
Merge pull request #12 from Hein-Htet-Aung08/student-3-tristan-workload-and-availability-management new	Verified 7b6f5a6	
Alamb-beyonder authored last week · ✓ 6 / 6		
Integrate student 5 with shared Release 0 application	eeb95ac	
Mattonut committed last week		
Merge remote-tracking branch 'origin/main' into student-5-matthew-performance-and-professional-development	2c65891	
Mattonut committed last week		
Complete student 5 Release 0 base feature	fac8780	
Mattonut committed last week		
Changed Textbox into Dropdown for clarity	84b0b27	
TLim05 committed last week · ✓ 6 / 6		
Changed departments to match DB	5e13d91	
TLim05 committed last week · ✓ 3 / 3		
Changed Pins	8012803	
TLim05 committed last week · ✓ 3 / 3		
Merge pull request #11 from Hein-Htet-Aung08/student-4-nicholas-research-and-grant-management new	Verified 159bfc7	
TLim05 authored last week · ✓ 3 / 3		
Merge branch 'main' into student-4-nicholas-research-and-grant-management	e0ce32a	
NicholasHatzl committed last week · ✓ 3 / 3		
Added /profiles/<staff_id>/json endpoint to workload_ui.py to allow for me to get workload for recommended staff	1a29246	
NicholasHatzl committed last week		

Merge pull request #10 from Hein-Htet-Aung08/student-3-tristan-workload-and-availability-management 🔒  Alamb-beyonder authored last week ·  3 / 3	Verified 895b6de  
Fixed ollama port conflict  TLim05 committed last week ·  3 / 3	127e304  
Read staff records from student 1 and align seed data to its roster  TLim05 committed last week ·  3 / 3	192e337  
Migrated frontend classes to the shared team stylesheet  TLim05 committed last week ·  3 / 3	5ab9a35  
Added student 3 to shared docker compose file  TLim05 committed last week	56f3722  
Frontend is partially finished. Waiting for Student 3 to run testings.  Hein-Htet-Aung08 committed last week	1a082f0  
Merge pull request #9 from Hein-Htet-Aung08/main 🔒  Hein-Htet-Aung08 authored last week	Verified b7a3b8d  
Merge branch 'main' of https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD into student-2-hein-teaching-subject-and-classroom-allocation 🔒  Hein-Htet-Aung08 committed last week	a7d0fa3  
All backend services finished. Waiting on Student 3 for Testing.  Hein-Htet-Aung08 committed last week	9ead584  
linked css to student 1 microservice  Alamb-beyonder committed last week	ca4b524  
Linked shared css to student 1 microservice  Alamb-beyonder committed last week ·  2 / 2	8c26c9d  

Commits on Sep 2, 2026

Merge pull request #8 from Hein-Htet-Aung08/student-4-nicholas-research-and-grant-management 🔒  Alamb-beyonder authored last week	Verified fc951cd  
Merge branch 'main' of https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD into student-2-hein-teaching-subject-and-classroom-allocation 🔒  Hein-Htet-Aung08 committed last week	d507807  

Merge branch 'main' into student-4-nicholas-research-and-grant-management  NicholasHatzl committed last week	36fa77a  
add recommended staff button  NicholasHatzl committed last week	7741dfd  
integrated staff management sevice for ai mode. fixed match-staff service to respond with real names from andy's database and recommend staff function to search the roster  NicholasHatzl committed last week	52e7dda  
added shared homepage with CSS and updated docker-compose.yml with home services  NicholasHatzl committed last week	ba4794e  
Fix CORS for frontend-backend communication in Docker  Alamb-beyonder committed last week ·  1 / 1	fee2c92  
Merge branch 'main' of https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD into student-2-hein-teaching-subject-and-classroom-allocation  Hein-Htet-Aung08 committed last week	b72823c  
Database setup with get routes finsihed.  Hein-Htet-Aung08 committed last week	9fdb3de  
Merge pull request #6 from Hein-Htet-Aung08/student-1-andy-staff-management 🔒  TLim05 authored last week ·  5 / 5	Verified 454810e  

Project structure already accomplished. Hein-Htet-Aung08 committed last week	06d9e3a		
Merge pull request #4 from Hein-Htet-Aung08/student-4-nicholas-research-and-grant-management Alamb-beyonder authored last week	Verified 6b8581e		
added task and system prompts for ai, adjusted dockerfile and fixed bug with edit and add staff Alamb-beyonder committed last week - 2 / 2	9ea7f12		
Merge branch 'main' into student-4-nicholas-research-and-grant-management NicholasHatzl committed last week	c26385e		
Added student4 services into yml file and added shared ollama NicholasHatzl committed last week	fc47612		
basic tests for backend NicholasHatzl committed last week	cca2302		
Merge pull request #2 from Hein-Htet-Aung08/student-3-tristan-workload-and-availability-management TLim05 authored last week - 3 / 3	Verified 6940c1e		
Merge branch 'main' into student-3-tristan-workload-and-availability-management TLim05 authored last week - 3 / 3	Verified 0b3db80		
Merge pull request #3 from Hein-Htet-Aung08/student-1-andy-staff-management TLim05 authored last week - 1 / 1	Verified 3bf3e4d		
Add docker-compose.yml skeleton with student-1 services Alamb-beyonder committed last week - 1 / 1	4037c5f		
Add dockerfiles for database/backend (didnt push before) and github actions CI workflow Alamb-beyonder committed last week - 1 / 1	a3590fa		
Added CSV and PDF export TLim05 committed last week - 6 / 6	4150600		
Changed ports to remove conflicts from other modules TLim05 committed last week - 3 / 3	8fa6f9b		
add dockerfiles for database, frontend and backend + db_path env var support Alamb-beyonder committed last week	6a76645		

Added CI workflow TLim05 committed last week - 3 / 3	d54966e		
Completed frontend. Have ot move individual css into shared folder when ready NicholasHatzl committed last week	b632675		
Added update and load support to complete CRUD in the frontend TLim05 committed last week	7b4e152		

Student 3 (Tristan Lim)

student-3-tristan-...		All users	All time
Commits on Sep 3, 2026			
Edited Text	TLim05 committed last week · 6 / 6	95b5cd9	
Changed LLM	TLim05 committed last week · 6 / 6	7cdeb8e	
Enforced LF Line Endings	TLim05 committed last week	c92f9b3	
Changed Textbox into Dropdown for clarity	TLim05 committed last week · 6 / 6	84b0b27	
Changed departments to match DB	TLim05 committed last week · 3 / 3	5e13d91	
Changed Pins	TLim05 committed last week · 3 / 3	8012803	
Fixed ollama port conflict	TLim05 committed last week · 3 / 3	f27e304	
Read staff records from student 1 and align seed data to its roster	TLim05 committed last week · 3 / 3	192e337	
Migrated frontend classes to the shared team stylesheet	TLim05 committed last week · 3 / 3	5ab9a35	
Added student 3 to shared docker compose file	TLim05 committed last week	56f3722	
Linked shareed css to student 1 microservice	Alamb-beyondr committed last week · 2 / 2	8c26c9d	

Commits on Sep 2, 2026		
Merge pull request #8 from Hein-Htet-Aung08/student-4-nicholas-research-and-grant-management	Verified fc951cd	
Alamb-beyonder authored last week		
Merge branch 'main' into student-4-nicholas-research-and-grant-management	36fa77a	
NicholasHatzl committed last week		
add recommended staff button	7741dfd	
NicholasHatzl committed last week		
integrated staff management sevice for ai mode. fixed match-staff service to respond with real names from andy's database and recommend staff function to search the roster	52e7dda	
NicholasHatzl committed last week		
added shared homepage with CSS and updated docker-compose.yml with home services	ba4794e	
NicholasHatzl committed last week		
Fix CORS for frontend-backend communication in Docker	fee2c92	
Alamb-beyonder committed last week - ✓ 1 / 1		
Merge pull request #6 from Hein-Htet-Aung08/student-1-andy-staff-management	Verified 454810e	
TLim05 authored last week - ✓ 5 / 5		
Merge pull request #4 from Hein-Htet-Aung08/student-4-nicholas-research-and-grant-management	Verified 6b8581e	
Alamb-beyonder authored last week		
added task and system prompts for ai, adjusted dockerfile and fixed bug with edit and add staff	9eaf712	
Alamb-beyonder committed last week - ✓ 2 / 2		
Merge branch 'main' into student-4-nicholas-research-and-grant-management	c26385e	
NicholasHatzl committed last week		
Add student4 services into yml file and added shared ollama	fc47612	
NicholasHatzl committed last week		
basic tests for backend	cca2302	
NicholasHatzl committed last week		
Merge pull request #2 from Hein-Htet-Aung08/student-3-tristan-workload-and-availability-management	Verified 6940c1e	
TLim05 authored last week - ✓ 3 / 3		
Merge branch 'main' into student-3-tristan-workload-and-availability-management	Verified 0b3db80	
TLim05 authored last week - ✓ 3 / 3		
Merge pull request #3 from Hein-Htet-Aung08/student-1-andy-staff-management	Verified 3b3e4d	
TLim05 authored last week - ✓ 1 / 1		
Add docker-compose.yml skeleton with student-1 services	4037c5f	
Alamb-beyonder committed last week - ✓ 1 / 1		
Add dockerfiles for database/backend (didnt push before) and github actions CI workflow	a3590fa	
Alamb-beyonder committed last week - ✓ 1 / 1		
Added CSV and PDF export	4150600	
TLim05 committed last week - ✓ 6 / 6		
Commits on Sep 1, 2026		
Changed ports to remove conflicts from other modules	8fa6f9b	
TLim05 committed last week - ✓ 3 / 3		
add dockerfiles for database, frontend and backend + db_path env var support	6a76645	
Alamb-beyonder committed last week		
Added CI workflow	d54966e	
TLim05 committed last week - ✓ 3 / 3		
Completed frontend. Have ot move individual css into shared folder when ready	b632675	
NicholasHatzl committed last week		
Add update and load support to complete CRUD in the frontend	7b4e152	
TLim05 committed last week		
Commits on Sep 1, 2026		
removed local css	fe431b5	
TLim05 committed last week		

Commits on Sep 1, 2026		
Added frontend	TUm05 committed last week	0b83dcc
Added AI mode to help with workload rebalancing	TUm05 committed last week	7c4a652
Workload rules for overload, underload and clash detection	TUm05 committed last week	77ba76f
removed dockerfile from the first folder structure. docker files are now contained in backend and database-service as per specification	NicholasHatzl committed last week	ff84e97
fixed indentation error	NicholasHatzl committed last week	bd6a5dc
created and finalised dockerfile for backend	NicholasHatzl committed last week	831269b
implemented cors support (referenced from labs) and fixed backend dependencies to allow it to be containerised	NicholasHatzl committed last week	d63b54e
Add/edit form + ai analysis generation	Alamb-beyondr committed last week	eff1fac
fixed naming and added title comments	NicholasHatzl committed last week	c6e8ad6
updated database_api.py to call database-service using http instead of sqlite	NicholasHatzl committed last week	04cdk77
add staff list, detail view, filter and delete for frontend	Alamb-beyondr committed last week	af22fdb
moved sqlite data into a separete database-service flask app	NicholasHatzl committed last week	a06a6e1
Added AI mode into HTML endpoints and created generic prompt templates	NicholasHatzl committed last week	030b4f1

Commits on Aug 31, 2026		
Finished route functions, tested and changed mismatched name functions across endpoints	NicholasHatzl committed last week	549846a
Added database_api functions, llm_client and prompt_loader to use openai client the same as in the labs	NicholasHatzl committed last week	59043e4
Add ai analysis route with ollama integration (gwen2.5:0.5b)	Alamb-beyondr committed last week	fe4fb77
Add search and filter for staff api	Alamb-beyondr committed last week	eb18153
Finished backend routes and created prompt and service folder structure for AI integration on the web app. Modified backend/app.py to integrate ai_mode_bp.	NicholasHatzl committed last week	8294de4
Commits on Aug 30, 2026		
Finished grant and project routes code.	NicholasHatzl committed last week	9044f61
Commits on Aug 29, 2026		
Fixed app.py structure. Created routes folder to hold each route (grant, grant alerts etc). Created services and views folders	NicholasHatzl committed 2 weeks ago	ff5708d
Added endpoints for getting staff availability,expertise and qualifications	Alamb-beyondr committed 2 weeks ago	6bd0613
Commits on Aug 28, 2026		
Added staff CRUD routes and moved manual test JSON files into tests/manual	Alamb-beyondr committed 2 weeks ago	7cd036c
Database and Backend service	TUm05 committed 2 weeks ago	804071f

Commits on Aug 27, 2026		
Began development on backend app.py. Connected DB to backend and began on projects route development.	NicholasHatz committed 2 weeks ago	30b54cf
Added in seed data for the database. Fixed init_db.py to allow for docker container later	NicholasHatz committed 2 weeks ago	8f4e09b
Commits on Aug 26, 2026		
seed.py complete, database layer finished	Alamb-beyond committed 2 weeks ago	15cfadf
Created database fields for research and grant management. Created initial requirements.txt file for backend development	NicholasHatz committed 2 weeks ago	125d738
Commits on Aug 25, 2026		
Created all the databases required for staff management system	Alamb-beyond committed 2 weeks ago	c1a3a14
Commits on Aug 24, 2026		
Setup release 0 project structure based on the assignment specification	NicholasHatz committed 2 weeks ago	044cb1c
Commits on Aug 21, 2026		
Initial commit	Hein-Htet-Aung08 authored 3 weeks ago	Verified 08b1dee

Student 4 (Nicholas Hatzidimitriou)

Commits		
student-4-nichola...		NicholasHatz All time
Commits on Sep 5, 2026		
removed unnecessary file	NicholasHatz committed 3 days ago	8739b17
added shared agentic loop	NicholasHatz committed 3 days ago · 6 / 6	88c367f
Commits on Sep 3, 2026		
Merge pull request #14 from Hein-Htet-Aung08/student-4-nicholas-research-and-grant-management	NicholasHatz authored 5 days ago · 3 / 3	Verified 2beccb8
Merge branch 'main' into student-4-nicholas-research-and-grant-management	NicholasHatz committed 5 days ago	24980bb
added CI workflow to student-4	NicholasHatz committed 5 days ago · 3 / 3	5f8caac
Merge branch 'main' into student-4-nicholas-research-and-grant-management	NicholasHatz committed 5 days ago · 3 / 3	e0ce32a
Added /profiles/<staff_id>/json endpoint to workload_ui.py to allow for me to get workload for recommended staff	NicholasHatz committed 5 days ago	1a29246
Commits on Sep 2, 2026		
Merge branch 'main' into student-4-nicholas-research-and-grant-management	NicholasHatz committed last week	36fa77a
add recommended staff button	NicholasHatz committed last week	7741dfd
integrated staff management sevice for ai mode. fixed match-staff service to respond with real names from andy's database and recommend staff function to search the roster	NicholasHatz committed last week	52e7dda
added shared homepage with CSS and updated docker-compose.yml with home services	NicholasHatz committed last week	ba4794e

Merge branch 'main' into student-4-nicholas-research-and-grant-management	c26385e	<>
NicholasHatzl committed last week		
Added student4 services into yml file and added shared ollama	fc47612	<>
NicholasHatzl committed last week		
basic tests for backend	cca2302	<>
NicholasHatzl committed last week		
Completed frontend. Have ot move individual css into shared folder when ready	b632675	<>
NicholasHatzl committed last week		
Commits on Sep 1, 2026		
removed dockerfile from the first folder structure. docker files are now contained in backend and database-service as per specification	ff84e97	<>
NicholasHatzl committed last week		
fixed indentation error	bd6a5dc	<>
NicholasHatzl committed last week		
created and finalised dockerfile for backend	831269b	<>
NicholasHatzl committed last week		
implemented cors support (referenced from labs) and fixed backend dependencies to allow it to be containerised	d63b54e	<>
NicholasHatzl committed last week		
fixed naming and added title comments	c6e8ad6	<>
NicholasHatzl committed last week		
updated database_api.py to call database-service using http instead of sqlite	04cdc77	<>
NicholasHatzl committed last week		
moved sqlite data into a seperate database-service flask app	a06a6e1	<>
NicholasHatzl committed last week		
Added AI mode into HTML endpoints and created generic prompt templates	030b4f1	<>
NicholasHatzl committed last week		
Commits on Aug 31, 2026		
Commits on Aug 31, 2026		
Finished route functions, tested and changed mismatched name functions across endpoints	549846a	<>
NicholasHatzl committed last week		
Added database_api functions, llm_client and prompt_loader to use openai client the same as in the labs	59043e4	<>
NicholasHatzl committed last week		
Finished backend routes and created prompt and service folder structure for AI integration on the web app. Modified backend/app.py to integrate ai_mode_bp.	8294de4	<>
NicholasHatzl committed last week		
Commits on Aug 30, 2026		
Finished grant and project routes code.	9044f61	<>
NicholasHatzl committed last week		
Commits on Aug 29, 2026		
Fixed app.py structure. Created routes folder to hold each route (grant, grant alerts etc). Created services and views folders	ff5708d	<>
NicholasHatzl committed last week		
Commits on Aug 27, 2026		
Began development on backend app.py. Connected DB to backend and began on projects route development.	30b54cf	<>
NicholasHatzl committed 2 weeks ago		
Added in seed data for the database. Fixed init_db.py to allow for docker container later	8f4e09b	<>
NicholasHatzl committed 2 weeks ago		
Commits on Aug 26, 2026		
Created database fields for research and grant management. Created initial requirements.txt file for backend development	125d738	<>
NicholasHatzl committed 2 weeks ago		
Commits on Aug 24, 2026		
Setup release 0 project structure based on the assignment specification	044cb1c	<>
NicholasHatzl committed 2 weeks ago		

Student 5 (Matthew Barnard)

Commits on Sep 9, 2026		
Merge pull request #33 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development	Verified	37643ee
MattBarnard4 authored 1 hour ago · 3 / 3		
Merge pull request #32 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development	Verified	a54c3f4
MattBarnard4 authored 1 hour ago · 3 / 3		
Commits on Sep 3, 2026		
Merge pull request #21 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development	Verified	69439c5
MattBarnard4 authored last week · 7 / 7		
Merge pull request #20 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development	Verified	ce0429b
MattBarnard4 authored last week · 3 / 3		
Commits on Sep 1, 2026		
Merge pull request #1 from Hein-Htet-Aung08/student-5-matthew-barnard	Verified	d611a02
MattBarnard4 authored last week		

Commits on Sep 3, 2026		
Merge pull request #21 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development	Verified	69439c5
MattBarnard4 authored last week · 7 / 7		
Merge pull request #20 from Hein-Htet-Aung08/student-5-matthew-performance-and-professional-development	Verified	ce0429b
MattBarnard4 authored last week · 3 / 3		
Commits on Sep 1, 2026		
Merge pull request #1 from Hein-Htet-Aung08/student-5-matthew-barnard	Verified	d611a02
MattBarnard4 authored last week		

Commits on Sep 9, 2026		
Simplify Student 5 dashboard and retry metric loading		255fd58
Mattonut committed 1 hour ago · 6 / 6		
Commits on Sep 8, 2026		
Completed student 5 agentic ai loop feature		877cd08
Mattonut committed 18 hours ago · 6 / 6		
couple fixes		c893251
Mattonut committed 20 hours ago · 3 / 3		
Commits on Sep 3, 2026		
Improve feature navigation		6601e53
Mattonut committed last week · 7 / 7		
Remove AI mode scope disclaimer		b054ce2
Mattonut committed last week · 6 / 6		
Add grounded fallback for invalid AI recommendations		b473ee0
Mattonut committed last week · 6 / 6		
Merge remote-tracking branch 'origin/main' into student-5-matthew-performance-and-professional-development		0b9ec0d
Mattonut committed last week		
Remove student 5 evidence guide		8dd9769
Mattonut committed last week		
Rename student 5 frontend and backend service folders		9e83d36
Mattonut committed last week · 3 / 3		
Integrate student 5 with shared Release 0 application		eeb95ac
Mattonut committed last week		
Merge remote-tracking branch 'origin/main' into student-5-matthew-performance-and-professional-development		2c65891
Mattonut committed last week		
Complete student 5 Release 0 base feature		fac8780
Mattonut committed last week		

Contribution Logs

Student Number	Name	Contribution
25249309	Andy Lam	- Academic Staff Management Microservice - Agentic Loop (AI-mode Integration quality reviewer) - Integrated Architecture - Docker compose evidence - Repository Structure - Other shared sections - Individual Report Sections - README.md
25026511	Hein Htet Aung	- Teaching, Subject and Classroom Allocation Microservice - Student 2 GitHub Actions CI/CD Workflow - Overall Project Plan - Sprint Backlog - Student 2 Individual Report Sections - Ran and reviewed the shared Agentic Loop against Student 2 - Reviewed Agentic Loop prompts/output and validated findings against the implementation
25299672	Tristan Lim	- Workload and Availability Management Microservice - Individual Report Sections - Docker Compose Architecture diagram - DevOps Pipeline Architecture diagram - Edited Video
25471398	Nicholas Hatzidimitriou	- Research and Grant Management Microservice - Agentic Loop - Project Overview - Agentic AI workflow diagram - Individual Report Sections - Individual Functional and non-functional requirements
25563457	Matthew Barnard	- Performance and Professional Development Microservice - Development Goal Integrity Agentic AI reviewer - Student 5 individual report sections - Agile Team Project Approach

Showcase video URL

https://www.youtube.com/watch?v=jHCUCQG_3oE

GitHub Repository Link

<https://github.com/Hein-Htet-Aung08/faculty-management-system-ASD>